SOCIAL NETWORKS

NPTEL Online Course | Prof. Sudarshan Iyengar, IIT Ropar

Week 5 - Lecture Transcripts

Lectures 54-74 | 21 lectures | 165 pages (source transcript pages 532-695)

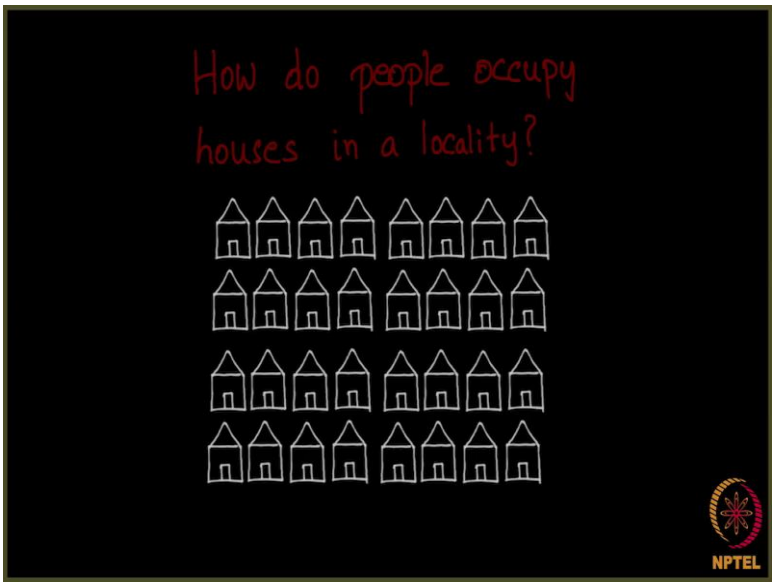
INDEX

LECTURE	TOPIC	PAGE IN THIS FILE
L54	Spatial Segregation: An Introduction	2
L55	Spatial Segregation: Simulation of the Schelling Model	9
L56	Spatial Segregation: Conclusion	14
L57	Schelling Model Implementation-1 (Introduction)	16
L58	Schelling Model Implementation-2 (Base Code)	21
L59	Schelling Model Implementation-3 (Visualization, Boundary and Internal Nodes)	35
L60	Schelling Model Implementation-4 (Getting a List of Unsatisfied Nodes)	50
L61	Schelling Model Implementation-5 (Shifting Unsatisfied Nodes, Final Graph)	58
L62	Chapter 5 - Positive and Negative Relationships (Introduction)	65
L63	Structural Balance	67
L64	Enemy's Enemy is a Friend	75
L65	Characterizing the Structure of Balanced Networks	83
L66	Balance Theorem	88
L67	Proof of Balance Theorem	94
L68	Introduction to Positive and Negative Edges	99
L69	Outline of Implementation	105
L70	Creating Graph, Displaying It and Counting Unstable Triangles	112
L71	Moving a Network from an Unstable to Stable State	124
L72	Forming Two Coalitions	145
L73	Forming Two Coalitions (contd.)	156
L74	Visualizing Coalitions and the Evolution	160

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture - 54
Homophily (Continued) & Positive and Negative Relationships
Spatial Segregation: An Introduction

(Refer Slide Time: 00:16)



So, let us discuss an interesting question on how do people occupy houses in a locality. So, assume you had a few houses and you want to choose a house for yourself what kind of house will you like living in that you think who your neighbors are matters to some extent would you like to be in a locality.

(Refer Slide Time: 00:44)



Where your neighbors are someone with whom you cannot have a conversation or your neighborhood is full of people who are unlike you would you like staying in such locality.

Now, here is an observation made several times in the literature. The observation is people are known to go and occupy a place a locality or a city uniformly at random, but which time they sort of start segregated by segregating I mean they all get together. You will see clusters of different types of people here is a type of people let us call it type green they all are together, they all are together and there is this type red as well they all are together too. Then you can see the clustering happening here they wish to stay with each other people have observed why exactly this happens as I told you this is not observed initially they come and occupy the place uniformly at random, but which time you see this kind of segregation that I am showing here.

They get segregated into different classes. So, assume in the united states you will see a lot of Indians, Chinese and people from other countries, but you will see that Indian tend to sort of stay with each other in colonies and there will be colonies of Chinese as well although there will be some sort of mingling, but there are places where you will see only people of one type this is observed in many places. So, assume you were to live in a foreign country or even in your own country would you not like to live in a place where you are surrounded by people who are of your type.

(Refer Slide Time: 02:41)



Now these is observed and let us assume you are surrounded by people who are not like you it is observed that people then they came to migrate.

This is you and you are surrounded by people who are unlike you what will happen?

(Refer Slide Time: 03:03)



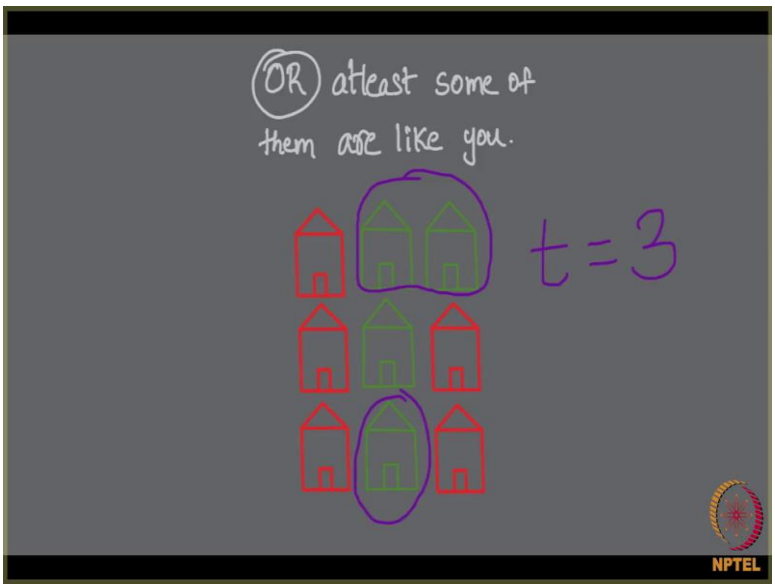
Nobody would like to stay in such a place would you like to stay in such a place absolutely not mostly not.

(Refer Slide Time: 03:14)



But then would you not prefer to stay they your type of people are present right. So, people who are around you are all like you anybody would like to stay in such a place not just this. In fact, at least some of them should be like you right at least some of them should be like you for example, you are here.

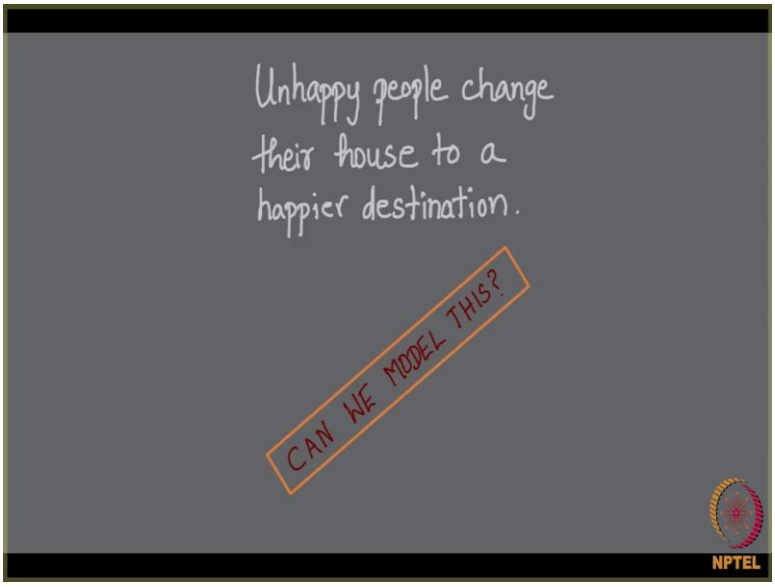
(Refer Slide Time: 03:34)



And you are surrounded by at least some 3 people who are like and the other one 2 3 4 five people are not like you would not care much.

So, let us call this the threshold value t equals 3 which means whenever take a pos and understand what I am saying this is the important concept in this lecture wherever you are surrounded by at least t number of people out of 8 people who can surround you in a neighborhood 3 behind 3 in the front and 2 people on a sideways. Let us say at least when you have t number of people who are like you then you will stay there if they are not if you are not surrounded by people at least t people who are like you then you will migrate here are 3 people who are like me and my threshold is actually 3.

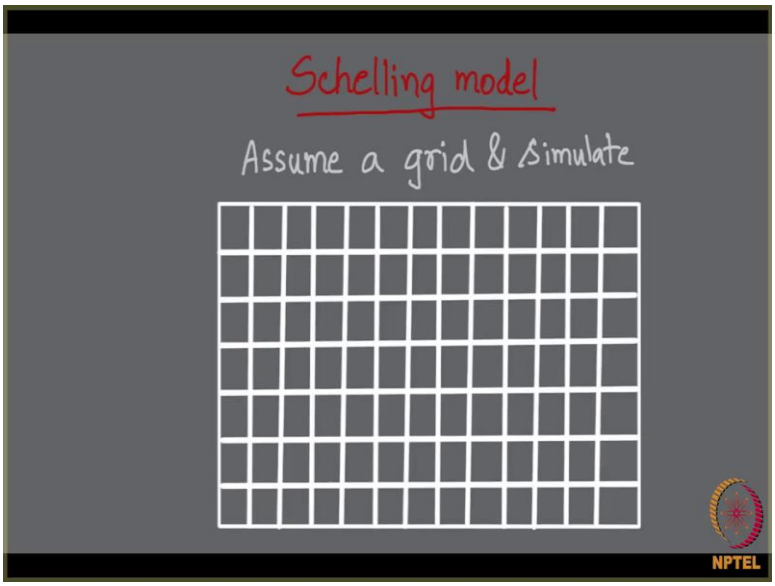
(Refer Slide Time: 04:46)



So, I like living in such a neighborhood.

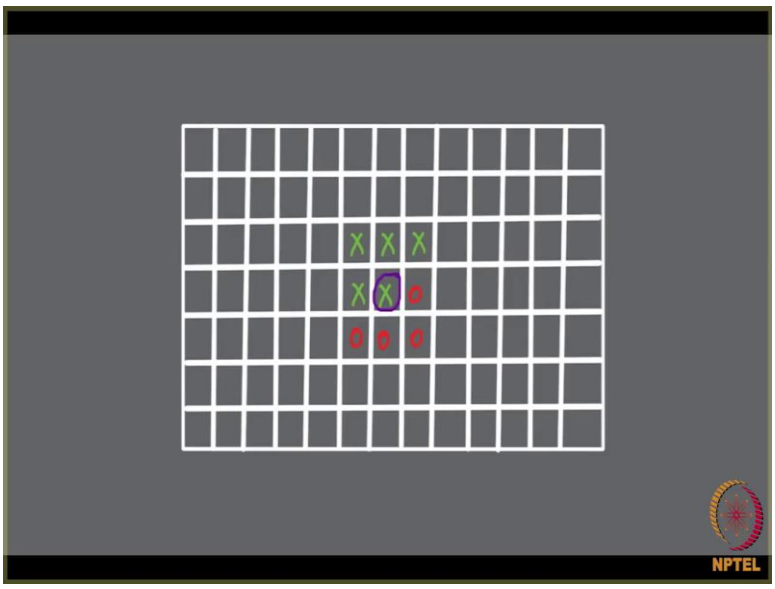
So, it is believed that unhappy people change their house to a happier destination by unhappy people I mean people who are surrounded by less than t number of people in the previous case we saw t was 3 which means this fellow was happy if in case this particular thing instead of being green was red. Then he would be unhappy because is neighbors would be less than 3 can we model this as a question can this be modeled can we model and see what exactly happens when people are unhappy and they migrate they come and occupy place uniformly at random a house uniformly at random in a city and then later they realize that because of the neighborhood they want to move to some other place.

(Refer Slide Time: 05:43)



How do we model this there is a very popular model called the Schelling model it is a very interesting model very simple and straight forward simulation it goes like this we assume that a grid simulates city grid is a analog to a city and we simulate the behavior of people.

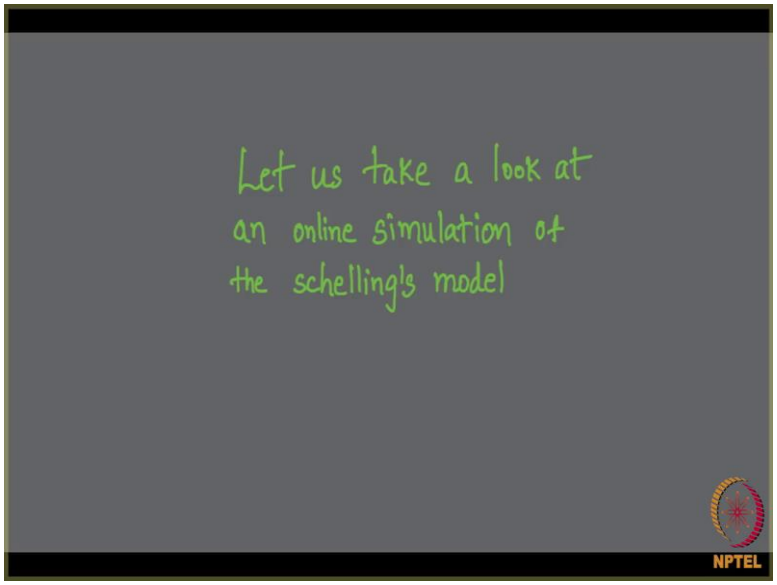
(Refer Slide Time: 06:13)



How do we do that we take a grid like this and then we put into for 1 parity that is let us say type green and then oh; for a red parity type red. So, this particular person x is

surrounded by the 4 red people and 4 green people as I told you if the threshold is 3 then this green fellow is actually happy.

(Refer Slide Time: 06:45)

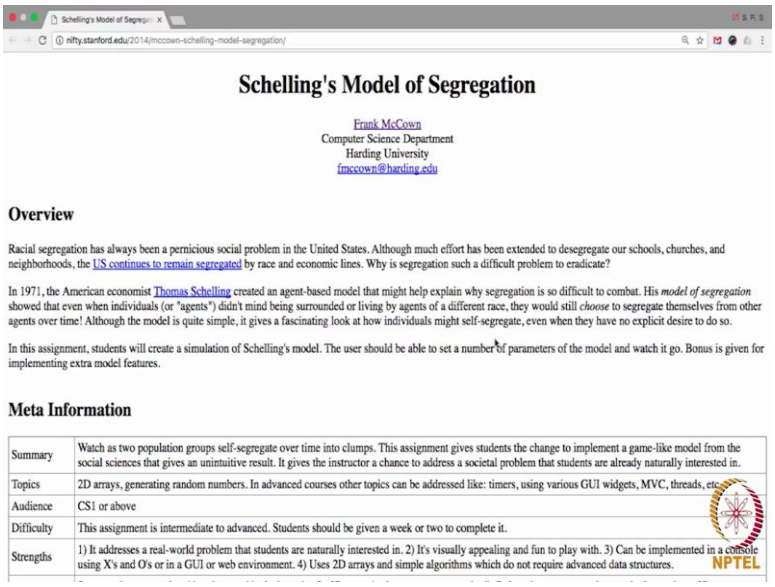


Now, let us take a look at on online simulation of the schillings model what is schillings model say schillings model simply says that as and when people are unhappy because they are surrounded by neighbors below their threshold value what do I mean by that sounds a little (Refer Time: 07:02) all I am trying to say is my threshold is 3 or may be 2 if I have 2 people in my neighborhood who are of my type I will stay there. Otherwise I will move let me take a big grid and then try simulating this and seeing. Let us see what one can observe. There are a lot of online simulations of this model I am just going to open a browser right now and then I am going to show you one such simulation.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

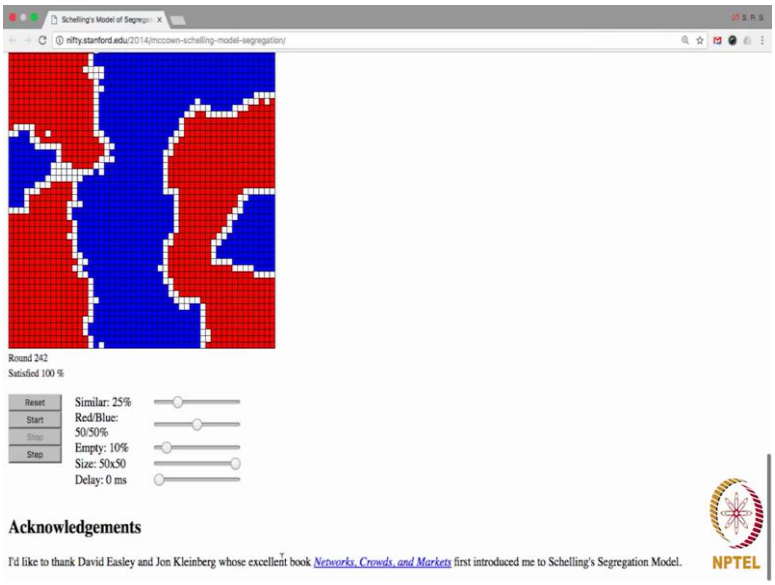
Lecture – 55
Homophily (Continued) & Positive and Negative Relationships
Spatial Segregation: Simulation of the Schelling Model

(Refer Slide Time: 00:05)



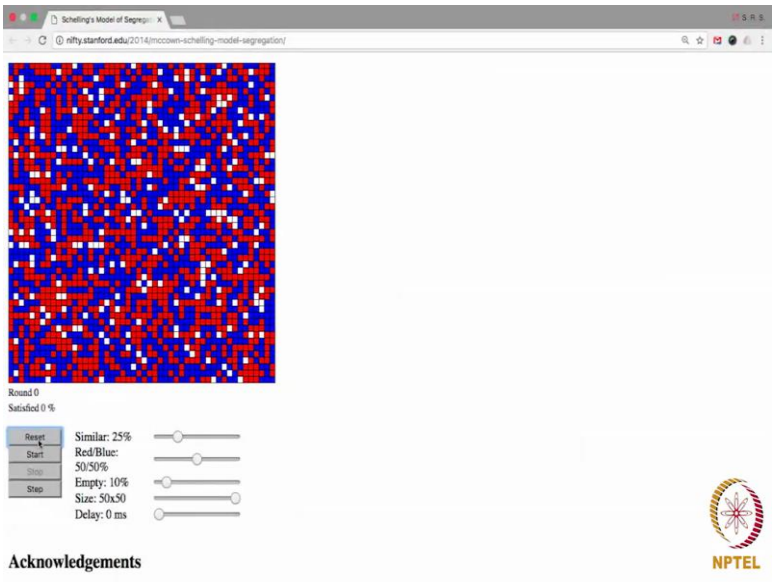
So, here is a website that I have opened, it has a very nice animation of the concept, it is by Frank McCown as you can see it there is a beautiful description of the model and a very nice simulation of the module as well.

(Refer Slide Time: 00:21)



If you can see in the acknowledgement in fact, the creator of this simulation acknowledges the text book that we have we are using for the course network stores in markets, alright.

(Refer Slide Time: 00:41)



So, here is the simulation facility let me hit the reset button and as you would have guessed that red and blue here denote 2 different types of people who have populated this city and white represents blank where anybody can shift. So, here have some parameters

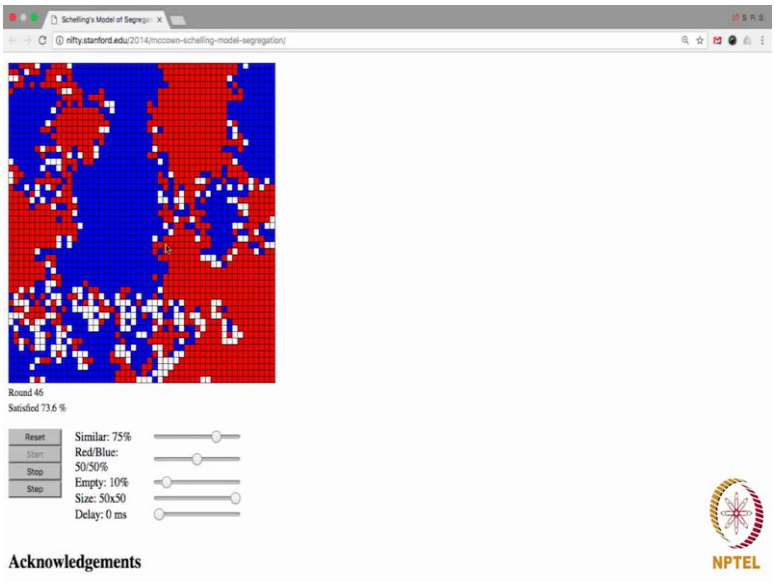
which are again, which are again quite self explanatory all though I will try explaining them for you.

So, this is this time for how similar do you expect the how similar do each node expect their neighbours if I put 25 percent here it is the t value that I explained just now, the t value I say t = 3. If its 25 percent I am surrounded by as you know 8 people 25 percent of 8 people is how much - 8 * 1/4 which is 2. So, if any person has 2 neighbours of the same colour then he is deemed to be happy and satisfied he will not shift and anyone who is not surrounded by at least 2 people of the same colour that person will try to shift to a blank space. And the rest is the distribution of red and blue it is the same when I say 50-50 you can in fact change it. Empty space I have kept some ten percent you can in fact, change these 2 sizes is 50 ka 50 grid delay is the delay between 2 steps of the animation I have kept it is 0.

So, let me reset it once again and then let me start the simulation as I start you will see that the number of people who are satisfied will be displayed here as of now nobody is satisfied it says 0 percent people are satisfied which means nobody is surrounded by at least 25 percent of people who are happy. Now basically once you start the animation only this will start counting as you can see I hit the start button and you see 100 percent people are satisfied it took just 8 rounds as you can see every single person is surrounded by at least 2 neighbours of its type. That is the similarity percentage that I gave t equals basically 2 now it will make t equals 4 which is 50 percent out of 8 people who my know I would expect half of them to be of my type if the reset and then start the simulation and see the beautiful convergence of segregation that happens here right now.

I hit the start button now please observe look at this they are getting segregated and you see there is some beautiful segregation happening here right. Then you expect half the people to be of your type you observe segregation happening right let me try increasing it all the more reset and increase this to let us say 75 percent what you expect now whether lets pause for a minute and then see what I am doing here I am saying 75 percent of my neighbourhood should be of my type which means if you take a blue red cell here I should have at least more than what is 75 percent of 8 neighbours it is 6. This red person should have 6 people surrounding him only then he is redeemed happy otherwise you would want to shift which means as you can see none of them are happy here, right. Let me start the simulation and see what happens.

(Refer Slide Time: 04:10)

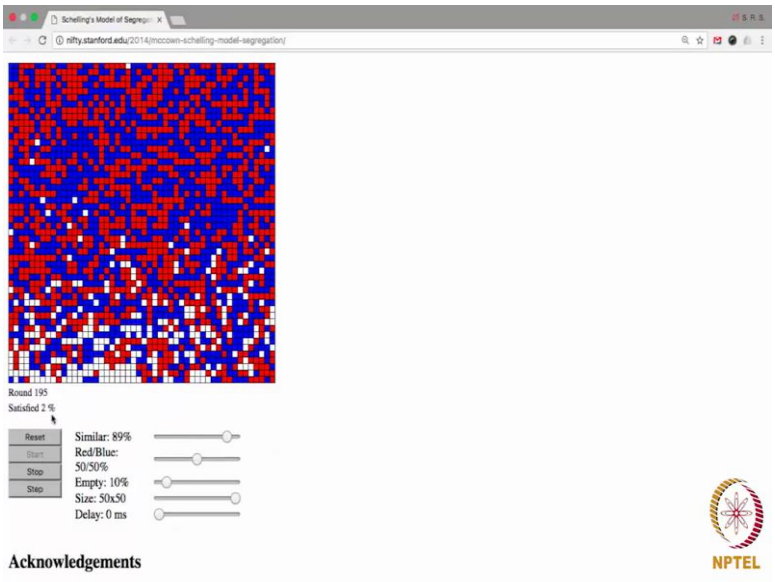


See, they are now moving to the empty spaces now roughly if 50 percent of the people are happy and then it goes on and on look at the beautiful segregation that is happening.

See more and more people get segregated and there are clusters that are emerging you see of huge blue cluster here and a red cluster here and you see 80 percent people are happy you see the stick more and more correct you see that is the segregation is beautifully going towards 100 percent. Now very soon you will see all of them are happy as you can see the number of rounds here in this case when similarity percentage is high when t is high seems to take a lot of time. You see the some problem in the boundary now with converges beautifully you see there is basically simply three clusters cluster 1; I am sorry 4 clusters cluster 1 cluster 2 cluster 3 and cluster 4 of course, you have clusters here as well, but I think we can ignore it, right.

So, basically there are 4 big clusters. So, the simulation says that more the similarity that you expect better is the segregation. If you actually I am sure you would know what we can guess what will happen if I make it 89 percent it will probably take a long time, but let us see what happens from here.

(Refer Slide Time: 05:29)



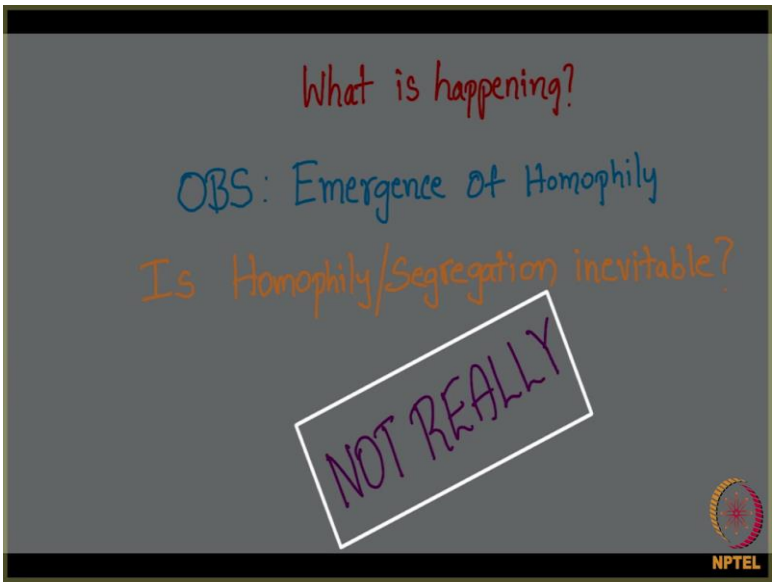
You see you see the people become less and less satisfied because 89 percent is what it is not achievable as you know 89 percent is more than 7 people should be of your type I mean that is not possible right not even more than 7 almost 8 people all your neighbour should be of your type that is not possible. So, it is a sort of simulation gets confused this might even go forever.

So, if you sort of reduce it you will observe that the convergence happens very quickly. Reset start for 30 percent which very quick yeah thanks to the create of this simulation we can explain this very easily with the simulation than verbal explanation, also it is interesting to note that the people have not understood this mathematically really well how exactly this happens how is it a function of this similarity percentage and how does the clustering combined although it is common sensical, but the exact mathematical details of it is not very well known.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 56
Homophily (Continued) and Positive and Negative Relationships
Spatial Segregation: Conclusion

(Refer Slide Time: 00:05)

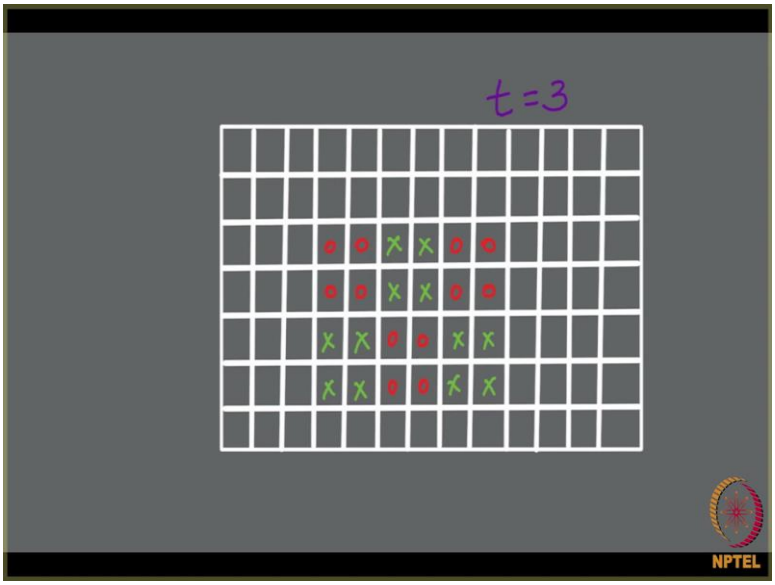


We saw the simulation; what it we just observe we observed that homophily sort of emerges, now please note the chapter the discussion has been of homophily. So, do you observe that there is some sort of a homophily here right. There is a lot of people who sort of get together and they cluster together now do you observe something. In fact, it is startling to see that local behaviour results in a global structure. What do I mean by that? A local behaviour such as one person trying to decide about his locality and trying to shift to a better locality person who is trying to see if he has a good neighbours if he does not have the required number of neighbours he shifts to a different locality he is just trying to do something locally, but each and every person doing this it results in a global phenomena correct.

So, we observe that there is this emergence of homophily here, but that is this homophily or segregation inevitable, what do I mean by this, probably my next figure that I am

going to show you we will make it clear is this homophily inevitable will it always happen not really and why do I say this.

(Refer Slide Time: 01:35)



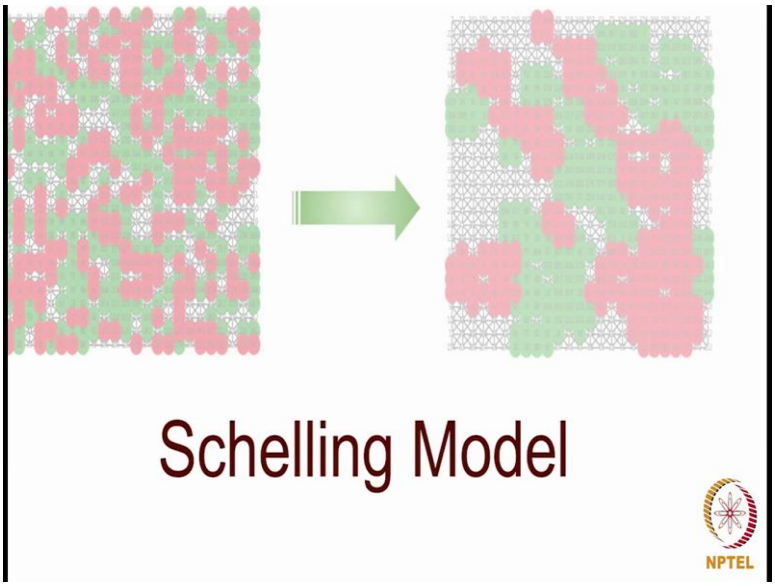
Look at this beautiful example. Here are a few people right green and red. In fact, for t = 3 you observe that everybody is actually happy everybody has 3 neighbours who are like them correct. So, observe it carefully every green has 3 green neighbours every red has 3 red neighbours you can filled the entire grid like this; this is not a lot of clustering here all though there is some kind of clustering there is lot of clustering here people. In fact, are living harmoniously with each other there are both types of people and there is no high because segregation here and still they do satisfy this property of the threshold being 3 which means they have the required number of neighbours, but segregation is not happening.

Now, this is a surprising factor that all though there is a possibility that segregation need it happen segregation does happen most of the time. It is observed that in on the map of some countries people basically are distributed evenly rather I mean uniformly at random, but later on this slowly drift and you see this patterns of segregation emerging. All in all Schelling model beautifully captures the this notion of a people getting segregated and rises many questions like can we come out to the nice mathematical model to explain this which remains to be an open question till date.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

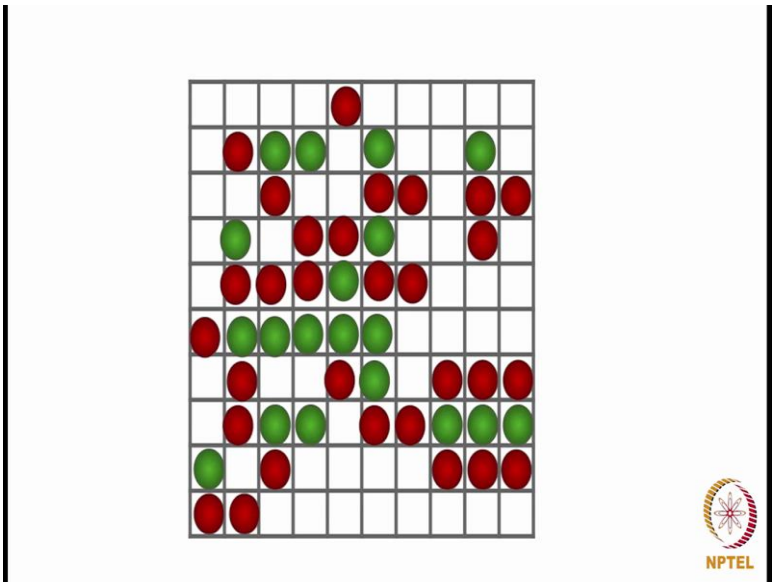
Lecture – 57
Homophily (Continued) & Positive and Negative Relationships
Schelling Model Implementation – Introduction

(Refer Slide Time: 00:05)



Hey everyone, Schelling model shows how the effect of homophily that operates at a local level leads to some interesting global patterns in the network. In this video we are going to implement this model and we will observe the patterns that emerge. Before we start the implementation I will give you a brief of the model, let us assume that there is a population of people and these people are one of the 2 types that is every person is either of type 0 or type 1.

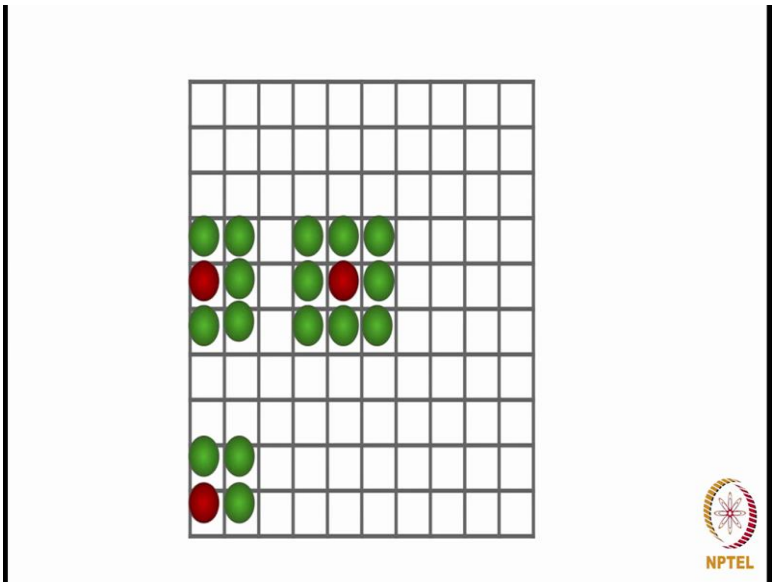
(Refer Slide Time: 00:44)



Further assume that these people live in a city which is like a grid that is there are cells in this grid where these people live just like this. Also, some cells of this grid have people whereas, some cells will be empty, in other words every cells cell will either have a person of type 0 or a person of type 1 or it will be empty. For example, like this as you can see every cell either has a red node or you can say red person or a green person or it is empty, correct.

Now, cells neighbours are the cells that touch it including the diagonal cells. So, you can imagine that the cell that is not on the boundary will have 8 neighbours.

(Refer Slide Time: 01:24)



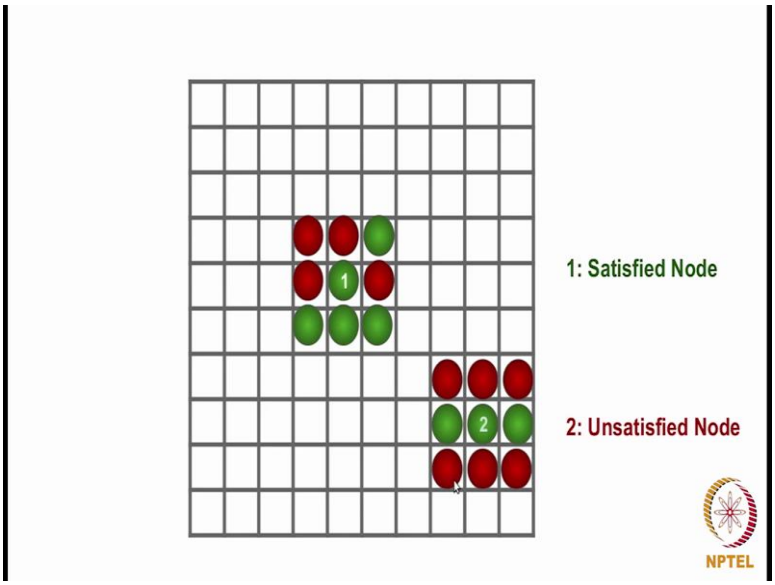
For example, if you look at this scenario this red node is an internal node that is not on the boundary. So, every node which is not on the boundary will have 8 neighbours like this the red person has basically the cell that has red person has 8 cells which are its neighbouring cells. Similarly, if you look at the boundary cells they can be 2 cases first case could be where the cell is a corner cell like here the red person is staying in a cell which is a corner cell. So, this cell has 3 neighbours as you can see here the other kinds of boundary cells can be like this which are not the corners.

So, as you can see this red person which is residing in the cell has five neighbours I think it is not difficult to understand that we can convert this whole scenario into a graph where the cells are the nodes and there is an edge between 2 cells if they are neighbours on this grid. So, that is what we are going to do we are going to analyze this grid using a network now if you look at a metropolitan city where there are people from different states of the country everyone tends to or rather prefers to stay in an area where there are more people from their own state. So, that practically happens we are going to apply the same scenario in our given grid. So, the people in our model wish to have at least some other people of their own type as neighbours.

Now, as an analogy if you look at a metropolitan city for example, where there are people from different states of the country everyone tends to or rather prefers to stay in an area where there are more people from their own state in a similar fashion the people

in our model wish to have at least some other people of their own type as neighbours. So, what we will do is we will assume a common threshold t for each person if a person discovers that it is surrounded by fewer than t people of its own type he tends to move to a new cell such a version is called unsatisfied with his current location.

(Refer Slide Time: 03:48)



As an example if we set the threshold t to be 3 for instance and if you look at this node a labelled one it has 8 neighbours as you can see and out of these 8 neighbours 4 neighbours are of its own type and the rest 4 neighbours are of different type types. So, since the threshold is 3 it has more than 3 neighbours of its own type. So, we will call this node one to be a satisfied node if you look at the other scenario.

For example if you look at this node labelled one it is of green type and it has 8 neighbours out of these 8 neighbours 4 neighbours are of the different type and 4 neighbours are of the same type as the neighbour as the node one.

Now, since as an example if we set the threshold t to be equal to 3 let us say and we look at the node which is labelled one here as you can see there are 8 neighbours and out of the 8 neighbours 4 neighbours are of different type and 4 neighbours are of the same time as the given node one since the threshold t = 3. And this node one has more than 3 neighbours of its own time we will call this node one to be a satisfied node, right and if you take one more example and if you look at this node which is labelled 2 here it again has 8 neighbours out of these 8 neighbours 6 neighbours are of different types and 2

neighbours are of the same type as the node 2 and threshold t we have said to be 3 since this node 2 has lesser than t neighbours of its own type this node 2 will be called an unsatisfied node.

Now, as I already explained the unsatisfied nodes tend to move to new locations which are already empty and where they are likely to be to be satisfied their different versions of Schelling model in this context. For example, in some of the versions does the unsatisfied nodes we will move to those locations where of where there are more likely to become satisfied or in other versions the unsatisfied nodes will move to any random location. So, so there are different versions which you can implement.

Ah another thing is that whenever a node moves from one cell to the other cell it affects other neighbouring nodes as well for example, in this process some other nodes might become satisfied some other nodes might become unsatisfied. So, it basically changes the scenario of the cell of the grid, right. So, in every iteration, we choose some unsatisfied nodes and we move them to new locations and the cell and the grid structure changes and this keeps on repeating and we can analyze the structure that we get to in the end that is after certain number of iterations.

So, this is what we are going to implement we are going to implement the version of Schelling model where the unsatisfied nodes move to the random locations and we are going to run several iterations of this process we are we are basically going to recalculate the unsatisfied nodes in every iteration because that is; obviously, required because one shifting affects the rest of the nodes as well. So, let us get started with implementation now.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 58
Homophily (Continued) & Positive and Negative Relationships
Schelling Model Implementation – Base Code

Before we start the implementation in the editor I want to show you certain commands in the ipython console, so that it becomes easier for you to understand these commands when I write there in the editor. I basically want to show you some intermediate results. So, I will be easier for you to understand. So, let me ask you what is our aim our aim is to how a grid then we have to put people on the cells of the script then we have to identity the nodes which are the people which are unsatisfied and then we have to move these are unsatisfied people to different locations that is what is the cracks. Now what is the first step? The first step is you want to grid, right.

So, how do we get the grid is your any network expansion that can help us to get the grid. So, apparently, we have this function grid to be graph which we can make use of although not to suit our complete purpose, but we can make changes into that and then basically we can manipulate that.

(Refer Slide Time: 01:16)

```
anamika@anamika-Inspiron-5423:~$ ipython
Python 2.7.6 (default, Oct 26 2016, 20:30:19)
Type "copyright", "credits" or "license" for more information.

IPython 1.2.1 -- An enhanced Interactive Python.
?         -> Introduction and overview of IPython's features.
%quickref -> Quick reference.
help      -> Python's own help system.
object?   -> Details about 'object', use 'object??' for extra details.

In [1]: import networkx as nx

In [2]: N = 10

In [3]: G = nx.grid_2d_graph(N,N)

In [4]: import matplotlib.pyplot as plt

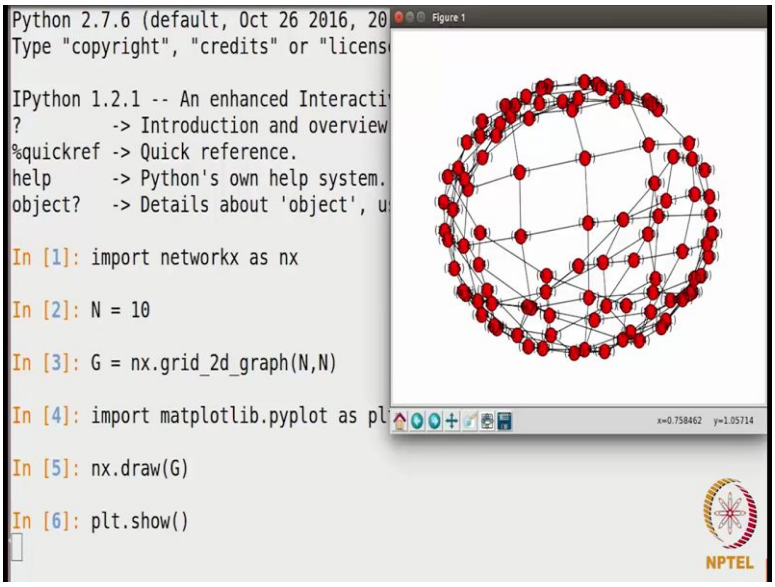
In [5]: nx.draw(G)

In [6]: plt.show()
```

So, let us get started I will start my ipython console and lets import. So, let us create a 10 cross 10 grid. So, I will take N to be 10 and then I will use the function. So, I will write n x.grid 2 d graph and we want to N to be N cross and that is a 10 cross 10 in this case.

So, this is the function that we will use let me show how it looks like. So, I need to show I need to show that. So, I need to import matplot lib.pyplot as plt. So, I will write n x.draw(G) and plt.show.

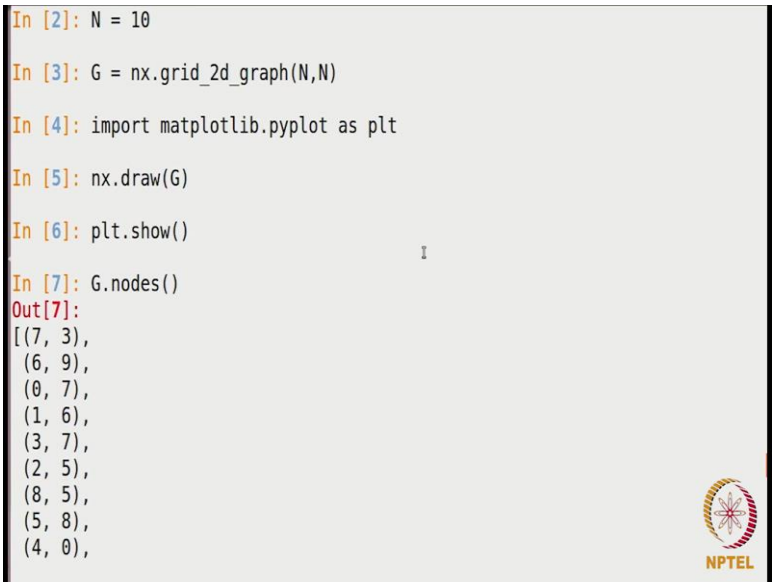
(Refer Slide Time: 02:17)



So, this is how it looks like; obviously, it is not looking. So, good it is not even looking like a grid of, but it is a grid although it is not looking like the way we want to show it. So, we might need to make some changes into it let us see what kind of changes we can make here I am going to close it the graph is not looking. So, nice because the nodes are not displayed in the form of a grid apparently, we do not have an in grid layout a networkx.

So, we might need to manually assign the positions to the grid to the nodes now how can we do that before I tell you that let me show you the nodes of this graph.

(Refer Slide Time: 03:05)



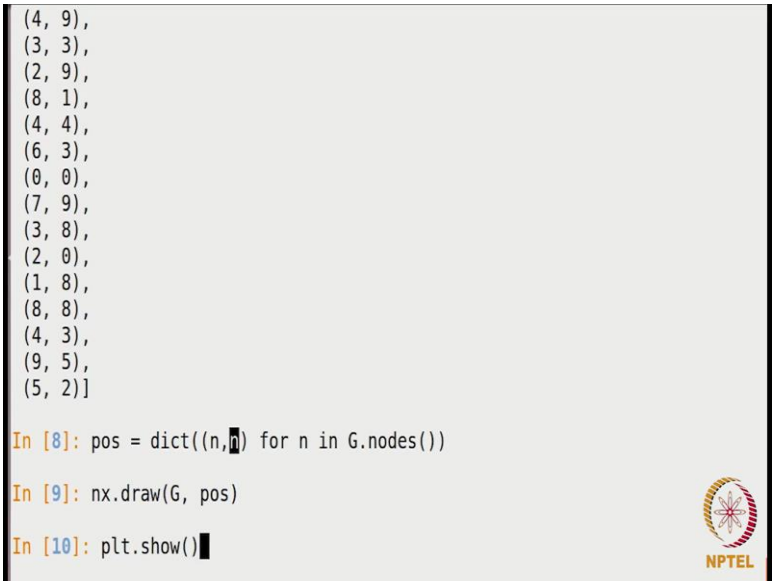
So, I will write G.nodes now this something different here in the other kinds of graphs we get either some number or some use a define string as the label of the nodes, but here see I wrote G.nodes and what I am getting is and getting a tuple for every node and the tuple consist in 2 values which is basically the row number and the column number of that node. So, basically it is like a grid, but it just that is not looking like a grid.

So, we might need to display it accordingly we might need to assign appropriate positioning to every node. So, I will show you how we can do that please note that we are getting both x and y positions which are relative positions of these nodes with respect to each other.

(Refer Slide Time: 03:58)



(Refer Slide Time: 04:00)



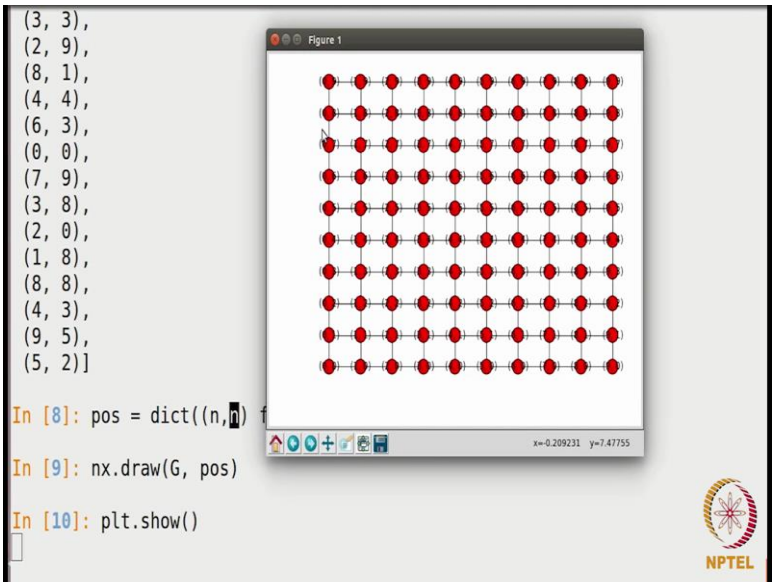
So, what we can do is I will write ((N, N)) for N in G.nodes or since we have used N capital N already maybe I should use small n. So, what I will write is I will convert this into a dictionary, and I will assign that dictionary to a variable say pos.

Let me explain you what I have done. So, these are G.nodes as you can see please note the pattern of the nodes are we getting here. So, these (8, 8) for example, will be N will be 1 N. So, what I am doing here for N in G.nodes that is for all these tuples I am writing (N, N). So, I am creating tuple where there are 2 Ns (N, N). So, it will be like 8 8 comma

8 8 what I am doing is for node I am fixing its position which is the same as its label, right so, and after that I am converting that into dictionary because that is how we pos the parameter that we pos should be in the form of dictionary. So, this dict function this is the function in python what it does is it converts a tuple into a dictionary where the first value of the tuple will be the key and the second value of the tuple will be the value of the dictionary.

So, this pos this is the positioning that we have fixed and we can assign the these positions should a nodes and how we can do that is we call this function and it is not draw G we can pos this positioning here and then we can write plt.show.

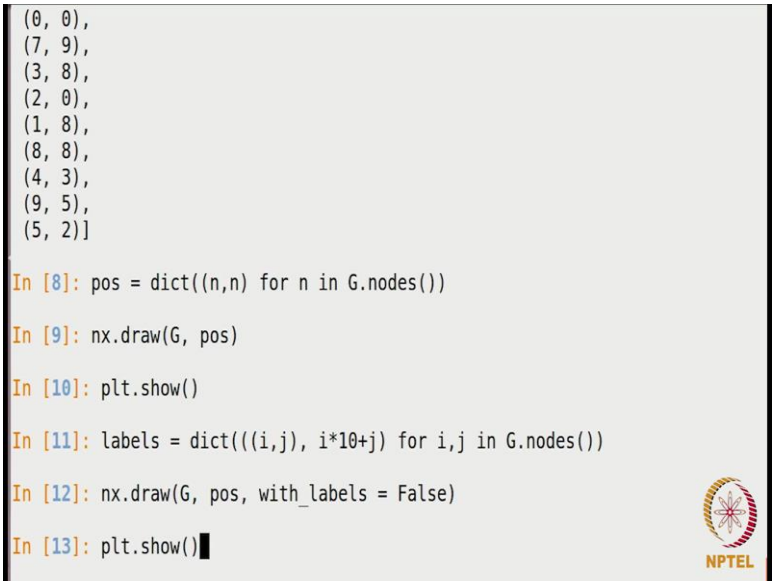
(Refer Slide Time: 05:47)



Alright, this looks beautiful now listen it right. So, as you can see, we are getting the grid and now other thing that we have to node here is that the labels are not in. So, nice one thing that you should note here is that 0 0 is here and the bottom left corner and it goes like this 0 0 0 1 0 2 0 3 and then comes back from 1 0 1 1 and so on. So, this is the default sequencing of the nodes that this function provides that we used what grid 2 d graph if you want you can change this label in such a way that 0 0 pairs here, but I do not think we need that. So, we will let it be like this, but one thing that you would want to do is that we would want to change the label of basically we have 10 cross 10 that is hundred nodes here and it will be nice if we assign the no labels like 0 2 0 1 2 3 up to 99.

So, that is one thing that we would like to do here now let me show you how we can do that again we will assign different label to every node now the name of the node itself carries the row and column number that is what we will be using in order to assign the label. So, the 0 0 should be having the label 0 0 1 should be having label 1 and so on 0 1 2 3 4 up to 10; now 0 up to 9 and then 10 should be here than 11 and so on. So, this is how we have to assign the labels how can we do that if you know a little math you should be able to understand what I am doing.

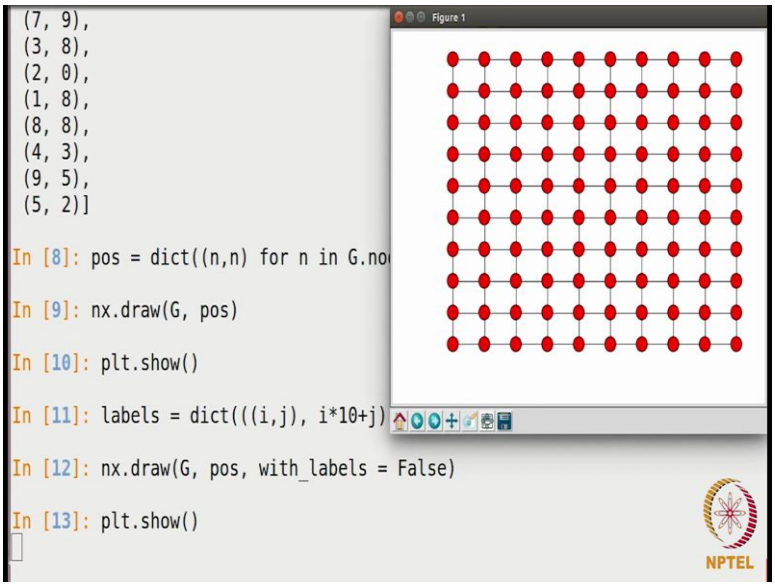
(Refer Slide Time: 07:35)



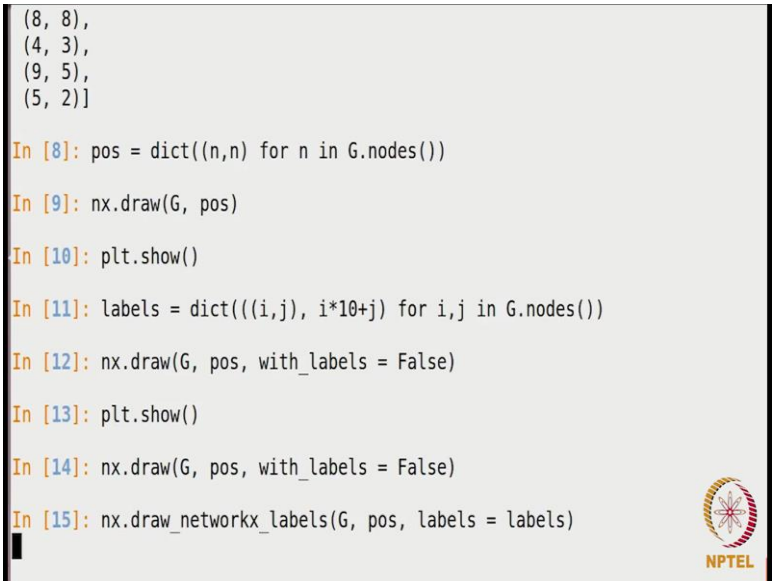
So, what I will do is (i, j) to (i, j) what I am assigning is i * 10 + j this is what I am assigning to first node and to the (i, j)th node this I will do for every node. So, I will write for (i, j) in G.nodes and again I will convert this into dictionary. So, I will use the same function and let me call this dictionary labels or when they should go.

So, what I am doing here is that to every node which is (i, j) like here like 4 3. I am assigning label which is i * 10 + j in the case (4, 3) it will be 43. So, that label I am assigning to every node and that information I am keeping in the form of a dictionary in this dictionary called labels which I am going to use. So, let me show you how we can do that. So, we used nx.draw a previously which displace some labels by default which is the labels that I showed you just now we do not want those labels to be displayed. So, what I will write is with labels = false right. So, the labels that it will showing it will not be visible you can check that is well if you I can pretty show you.

(Refer Slide Time: 09:02)



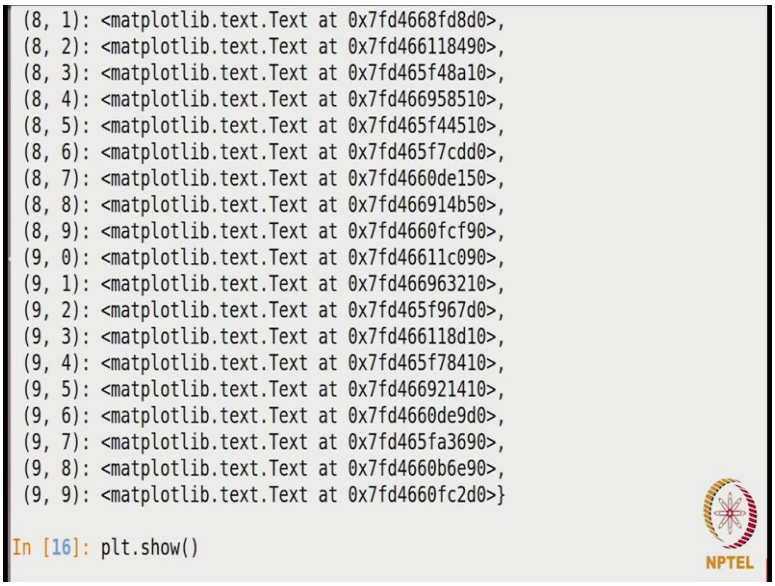
(Refer Slide Time: 09:04)



So, there are no label displayed here; here now we want to display our own labels for that we have a function nx.draw_networkx_labels. Now again we have to pos the graph we have to pos the positioning and then we have to pos the labels dictionary labels dictionary.

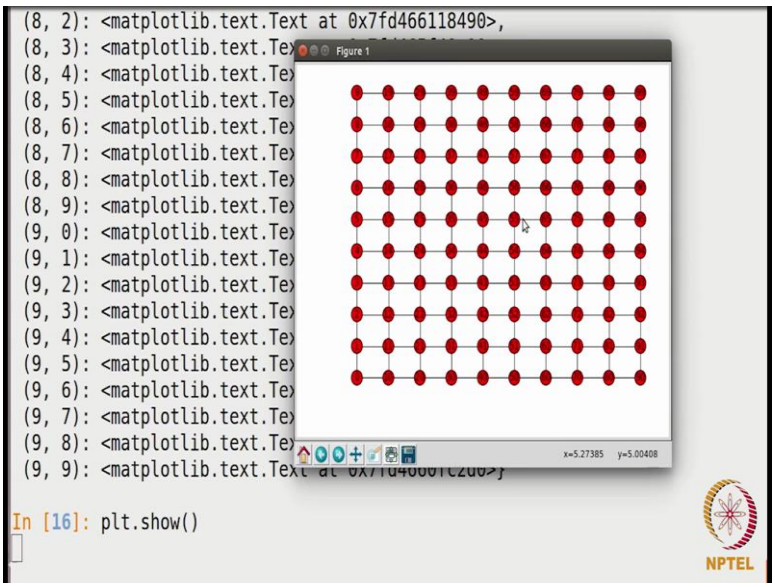
So, this is how we write labels is it would labels this should work now the labels have the new labels have been assigned that is display them oh, yeah, this looks nice.

(Refer Slide Time: 09:28)



Now, So, we have the labels from 0 1 2 3 up to 99. So, it will be nice to refer to them when we do the later manipulations.

(Refer Slide Time: 09:35)



Now, one thing that is important to note here is that I told in the explanation of scaling model that every node which is an internal node that is node on the boundary should have 8 neighbours right. Now here if you look at this node number 45 it has 3; 35, 46, 55, 44; it has only 4 neighbours it is not neighbours with the diagonal nodes as you can see. So, this is the kind of grid that the function provides as by default.

But this is not serving our purpose our purpose is that this 45 should be connected to 56 as well as an all these 4 nodes 36, 34, 54 and 56 now how do we do that. So, we would have to add them the edges manually which are not already present here now how can we do that I want to show you.

(Refer Slide Time: 10:40)

```
(8, 8): <matplotlib.text.Text at 0x7fd466914b50>,
(8, 9): <matplotlib.text.Text at 0x7fd4660fcf90>,
(9, 0): <matplotlib.text.Text at 0x7fd46611c090>,
(9, 1): <matplotlib.text.Text at 0x7fd466963210>,
(9, 2): <matplotlib.text.Text at 0x7fd465f967d0>,
(9, 3): <matplotlib.text.Text at 0x7fd466118d10>,
(9, 4): <matplotlib.text.Text at 0x7fd465f78410>,
(9, 5): <matplotlib.text.Text at 0x7fd466921410>,
(9, 6): <matplotlib.text.Text at 0x7fd4660de9d0>,
(9, 7): <matplotlib.text.Text at 0x7fd465fa3690>,
(9, 8): <matplotlib.text.Text at 0x7fd4660b6e90>,
(9, 9): <matplotlib.text.Text at 0x7fd4660fc2d0>}

In [16]: plt.show()

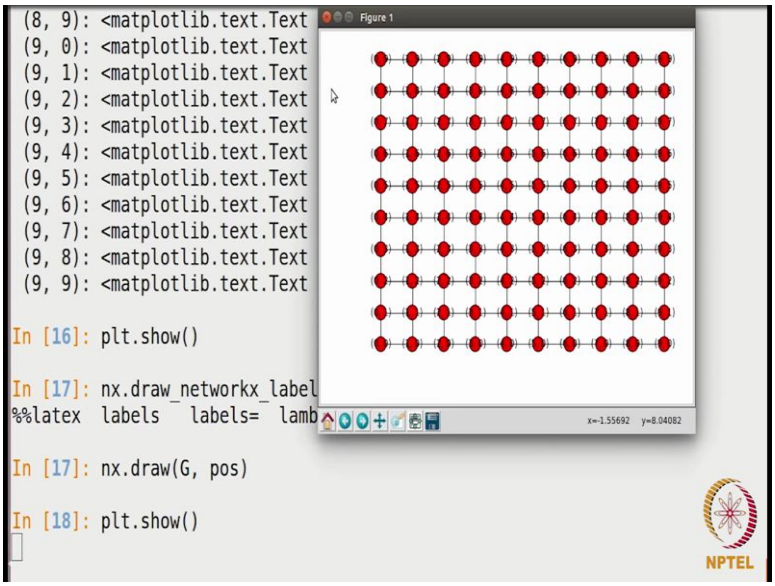
In [17]: nx.draw_networkx_labels(G, pos, labels = labels)
%%latex  labels   labels=  lambda

In [17]: nx.draw(G, pos)

In [18]: plt.show()
```

The previous representation of the graph because that will help us understand how we can add the edges, so, what I am going to do is I am not going to use this command nx.draw the usual function that we will using and this position we were using I would like to display this only.

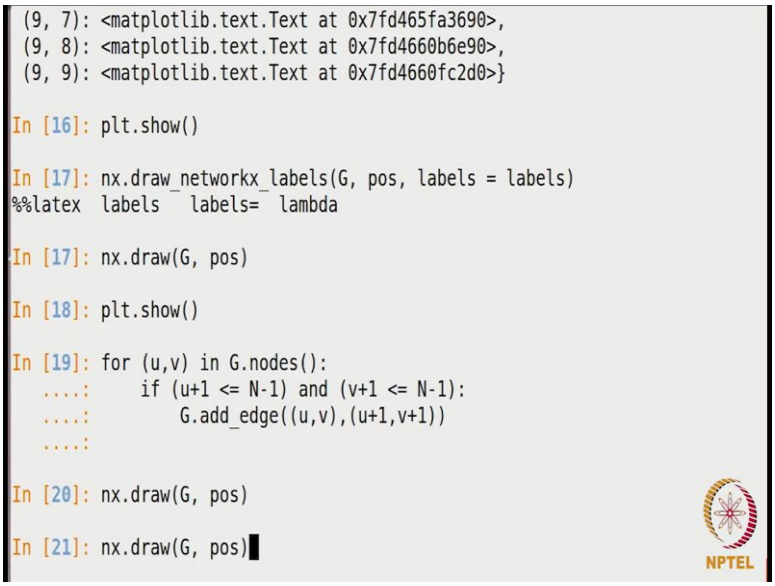
(Refer Slide Time: 11:06)



I am showing you the previous version of the graph for a reason because I want to show you how can are the edges with diagonal edges now if you look at this 1 1 node we have to connected to 2 2 this 2 2 has to be connected to 3 3 and if you look at some other node this 2 6 has to be connected to 3 7 4 8 3 7 has to be connected to 4 8.

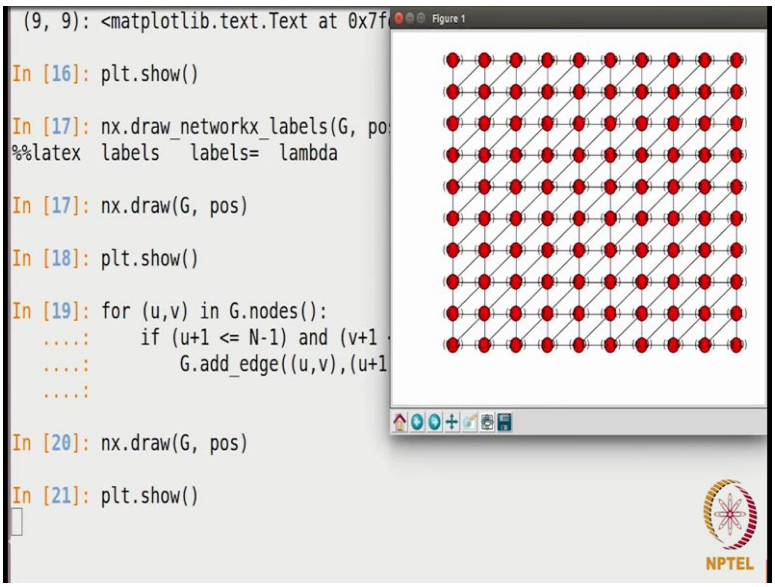
So, if you observe carefully the node (i, j) has to be connected to (i + 1, j + 1) if you look at 6 7 it has to be connected to 7 8 which is again (i + 1, j + 1). So, we will take all the nodes (i, j) and we will connect them to the nodes (i + 1, j + 1) that should be half of the work done. So, let us see one thing that we have to node here again is that when we are doing (i + 1, j + 1) it should never cross the 9 right we cannot connect 9 9 to 10 10. So, that limit we have to take care of, so that we will take care of that thing.

(Refer Slide Time: 12:15)



So, I will write for (u, v) let say for (u, v) in G.nodes what do we want to do (u, v) has to be connected to (u + 1, v + 1), but again we have to take care that u + 1 and v + 1 does not exist does not exceed N - 1. So, I will write if u + 1 <= N - 1 and we say to be done for v. So, I will write v + 1 <= N - 1 then only we will do this otherwise you will not add that edge. So, I will write G.add edge which edge (u, v) edge to (u + 1, v + 1) edge this should work let us see let me show you in the graph at this stage I will write nx.draw and then (Refer Time: 13:16) plt.show, yeah.

(Refer Slide Time: 13:16)



So, we have 1 p of half of the diagonal edges in corporate already the other direction of the diagonal edges is still yet to be added for example, we must connect this 3 9 2 4 8 4 8 to 5 7. So, that must be done.

Now, if you a observe carefully 4 6 has to be connected to 5 5 which means (u, v) has to be connected to (u + 1, b – 1) yeah that that wholes true for every node. Now again one thing that we have to take care of here is that while doing v - 1 it should not we come less than 0, so that part we will take care of let see how we can do that.

(Refer Slide Time: 14:01)

```
In [18]: plt.show()

In [19]: for (u,v) in G.nodes():
   ....:     if (u+1 <= N-1) and (v+1 <= N-1):
   ....:         G.add_edge((u,v),(u+1,v+1))
   ....:

In [20]: nx.draw(G, pos)

In [21]: plt.show()

In [22]: for (u,v) in G.nodes():
    if (u+1 <= N-1) and (v-1 >= 0):
        G.add_edge((u,v),(u+1,v-1))
   ....:

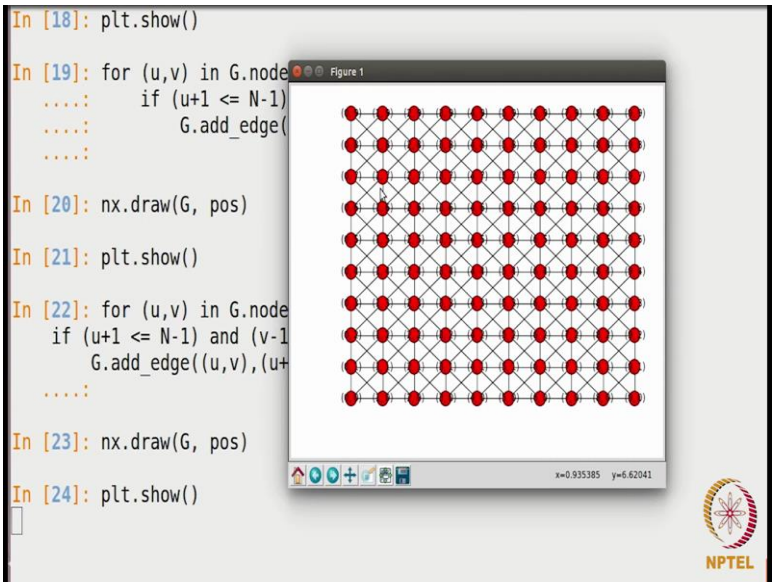
In [23]: nx.draw(G, pos)

In [24]: for (u,v) in G.nodes():
    if (u+1 <= N-1) and (v-1 >= 0):
        G.add_edge((u,v),(u+1,v-1))
```

NPTEL

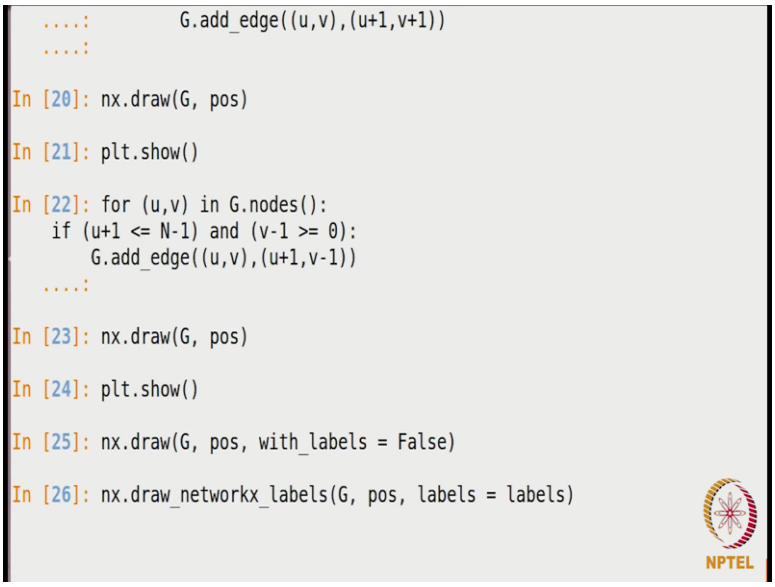
I am going to make changes here itself or yeah. So, what I will do is I will go back here and if u + 1. So, (u, v) must be connected to (u + 1, v – 1). So, u + 1 should be this and v - 1 should be less than equal to now should be greater than equal to 0 it should not be less than equal to. So, this we must check and if this is true than we will have N h from (u, v) to (u + 1, v – 1); they should work.

(Refer Slide Time: 14:46)



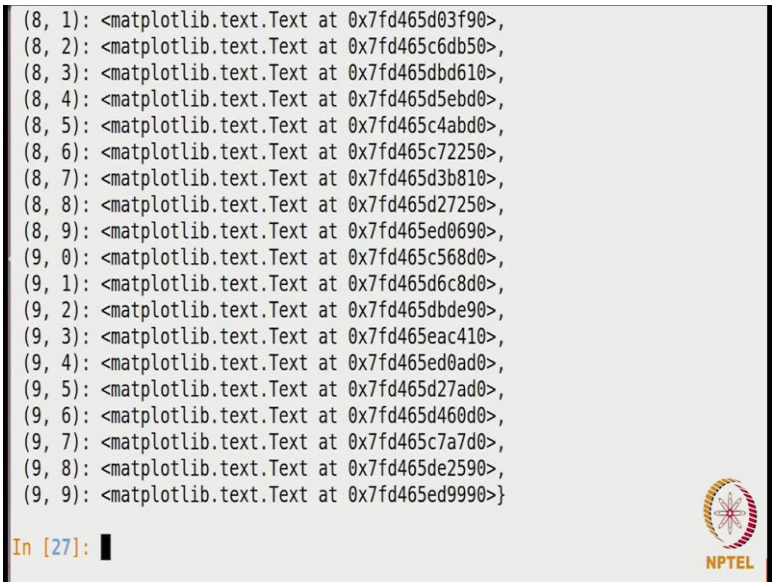
Doubtful about the in indentation but let me see if there is work if it works fine, yeah. So, here you see all the edges have been in incorporated all the diagonal edges are there now the great seems complete.

(Refer Slide Time: 14:58)

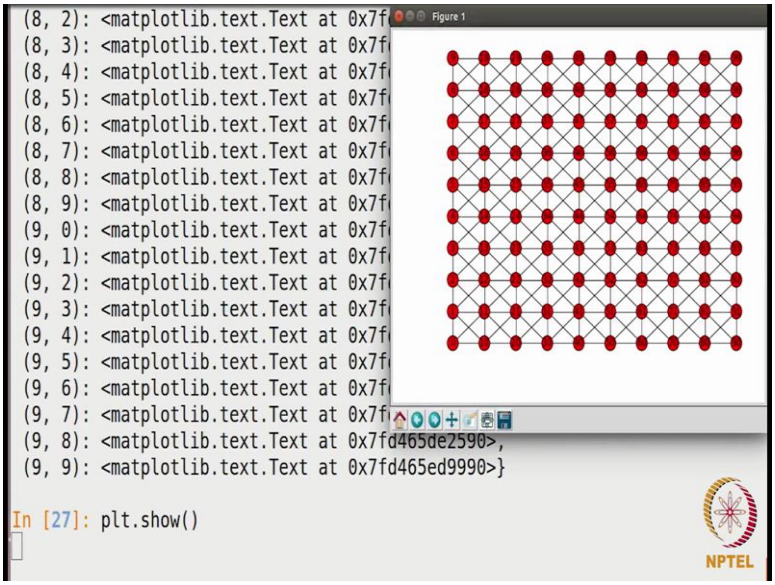


Let me just show you the labels they updated labels that we recently showed which command it will be use yeah, we use this and then we used this one yeah let us see.

(Refer Slide Time: 15:11)



(Refer Slide Time: 15:14)

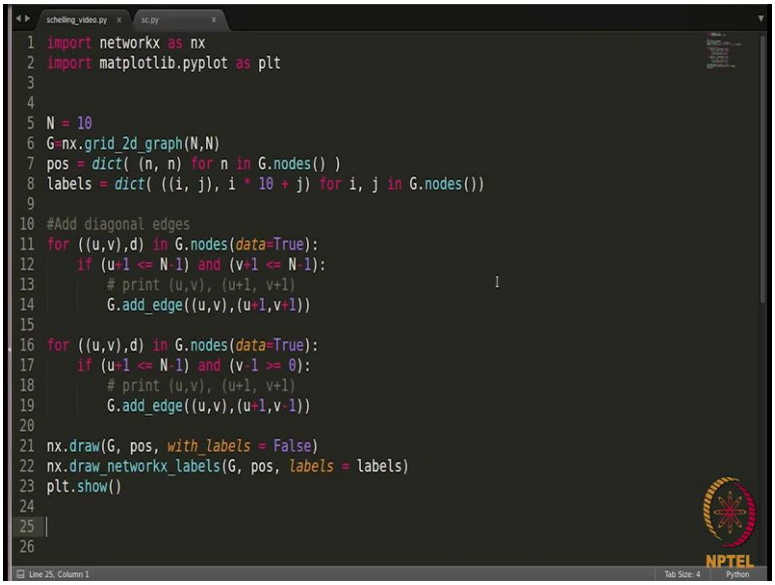


This looks nice. So, we have the great we have the diagonal edges we have the nodes now we have said the playground for playing with these nodes where we will put the people of different types there will be some people of type 0, there will be some people of type 1; there will be some nodes its will be empty we will not put any people there. So, that is how we are going start in the next video I will be using all these commands that I would show shown you already in the ipython console. So, I will use them in the editor and then we will get started with manipulating the nodes.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

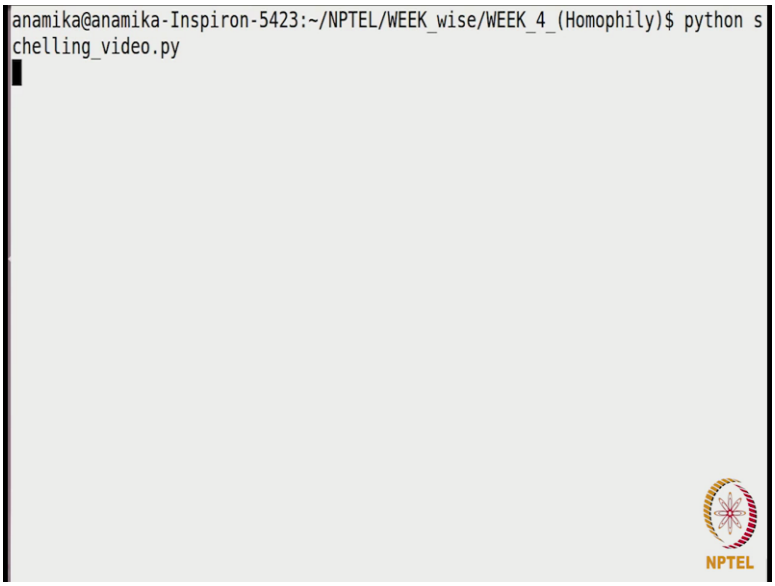
Lecture – 59
Homophily (Continued) & Positive and Negative Relationships
Schelling Model Implementation – Visualization and Getting a list of boundary & Internal nodes

(Refer Slide Time: 00:05)

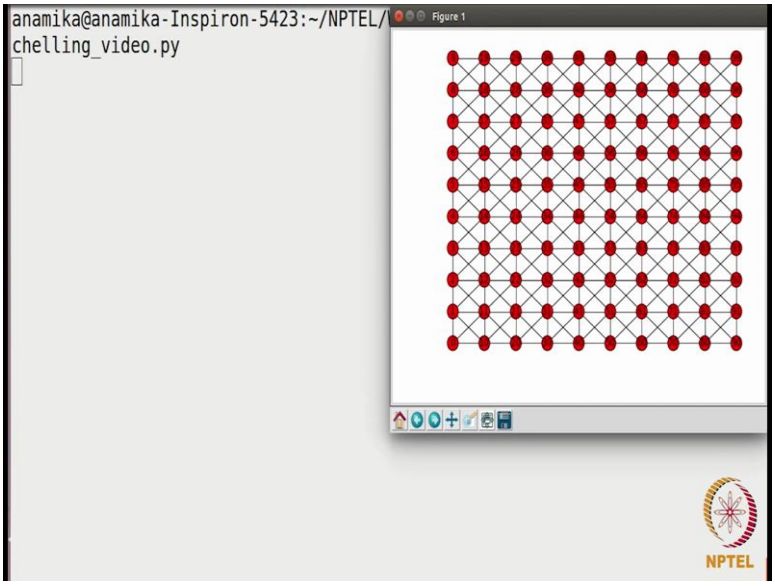


So, I have created this file and whatever we did in ipython console, I copied all that all those commands. So, here; so, there is nothing new here.

(Refer Slide Time: 00:20)

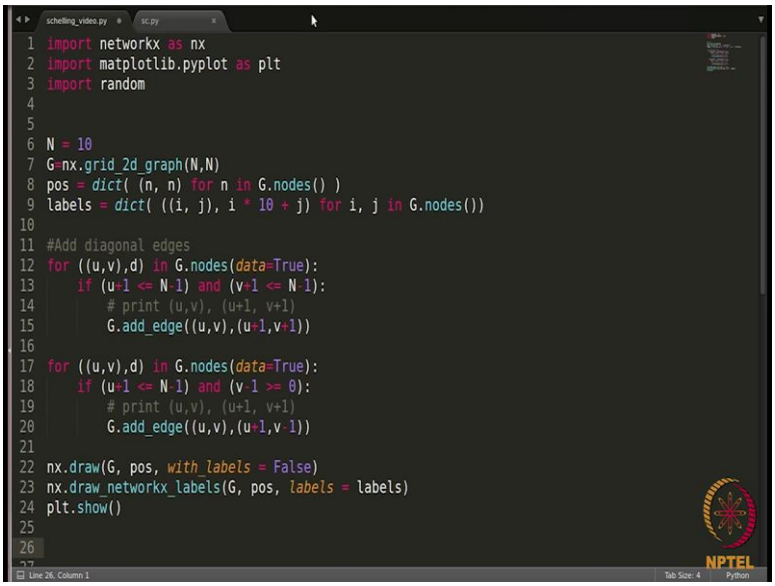


(Refer Slide Time: 00:27)



Let me show you the output of this. So, this is where we left of in the previous video and this is the graph that we were getting now we will continue with from here after having created the network we have to assign people to the nodes. So, as I explained we have to assign people of 2 kinds people of type one people of type 2 and there will be some nodes which will be empty. So, what we can do is we can make use of package random.

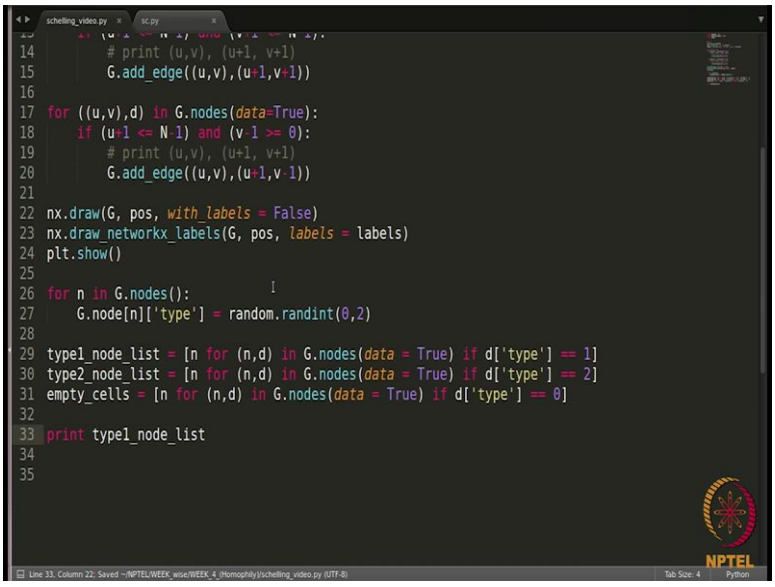
(Refer Slide Time: 01:00)



So, I will write both random to assign people to nodes what we can do here is that we can assign and attribute that is type to every node in the network.

So, this attribute type will have values from 0 1 or 2 if the type is equal to 0 the node will be considered empty if the type is equal to 1, the node will be considered a person of type 1 if the type = 2 the node will be considered a person of type 2. So, we will be following this convention.

(Refer Slide Time: 01:41)



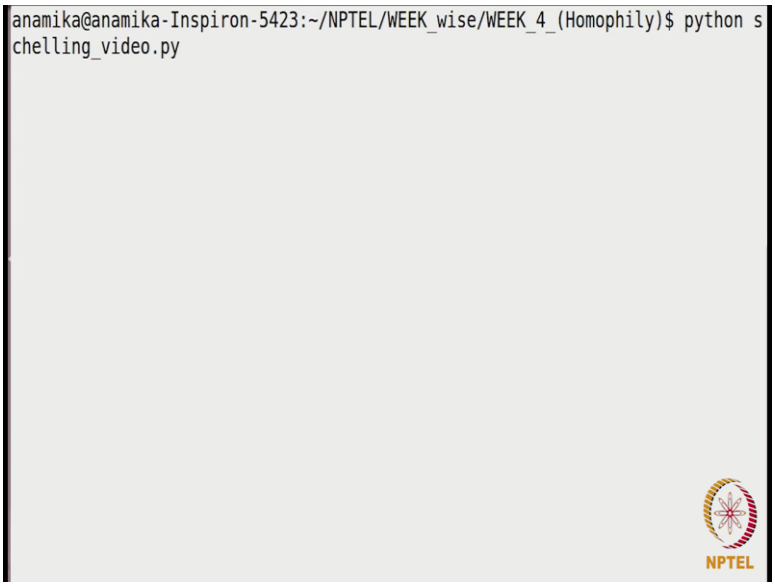
Let me assign the type to the nodes. So, what we can do is for n in G.nodes, G.node. So, they must assign a type to n. So, I will write type I want to assign a type randomly. So, I will write random.randint and this should re this should go from 0 to 2. So, 0 1 2 any random value could be assigned to the type of to the type of node n. Now after we have assign the type to every node, we should maintain a list of type one people 2 people and the list of empty nodes or you can say empty cells.

So, we will create three lists for that purpose. So, I will write type one node list sorry is equal to. So, this has to be list and what we can do here is n for (n, d) now I will explain what this means G.nodes (data = true) if the type of t = 1. So, let me tell you what I have done here. So, when you when you can G.nodes you get only a list of nodes, but in case we have assigned an attribute to the nodes and still we write G.nodes still we get only the list of nodes we do not get the get the tie we do not get the value of that attitude in case you want the value that attribute you have to explicitly write data = true.

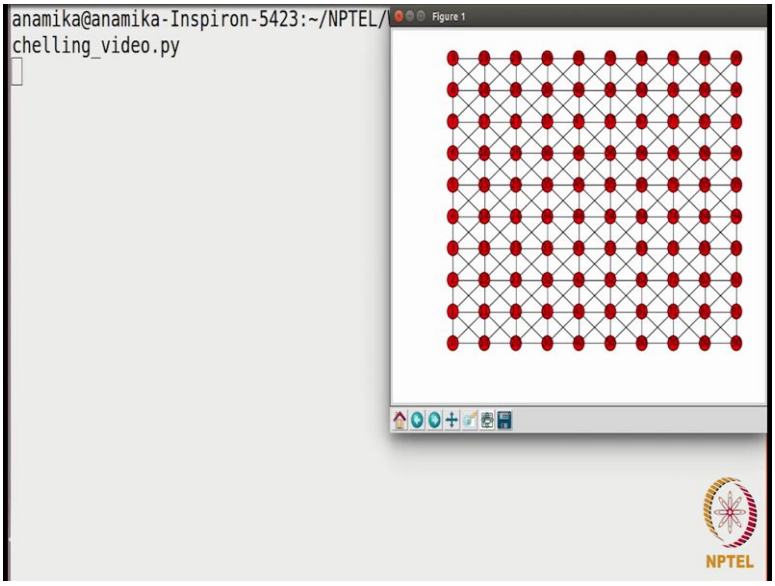
So, when you write G.nodes (data = true) you get n and d where n is a node and d is a dictionary which will contain the attribute and its value. So, since we want to get the type of every node we will write G.nodes (data = true) and that will write us (n, d) where n is a node and d will give us the type. So, we have written if d[‘type’] = 1. So, if d[‘type’] = 1 the node will be of type 1 sorry please pattern by back though out. So, this is for type one node list similarly we will get the type 2 list it just copies paste I am sorry. So, I will write type 2 node list is equal to n for (n, d) in G.nodes that as will true if d[‘type’] = 2, right.

So, similarly we can get a list of empty nodes. So, we can write empty cells is equal to, I will just copy paste where the type = 0. So, they should work we can also verify let me try printing the type one node list type one node list let me check whether it works find.

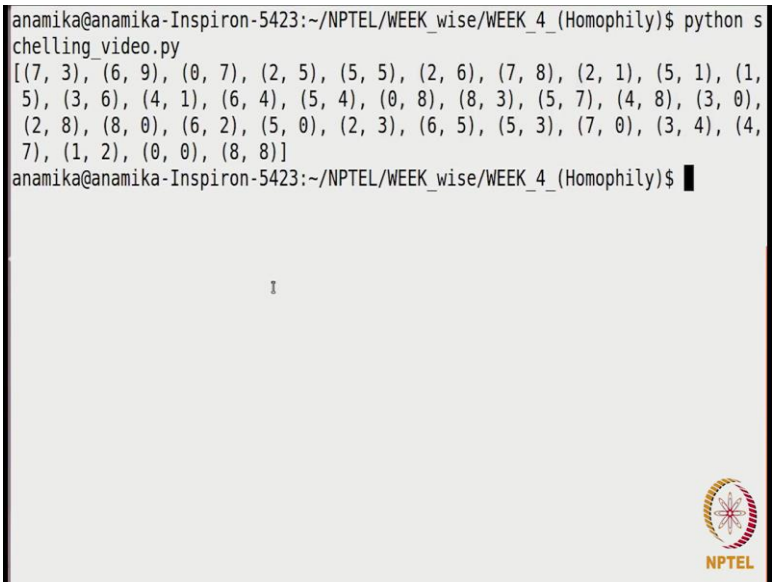
(Refer Slide Time: 05:07)



(Refer Slide Time: 05:10)

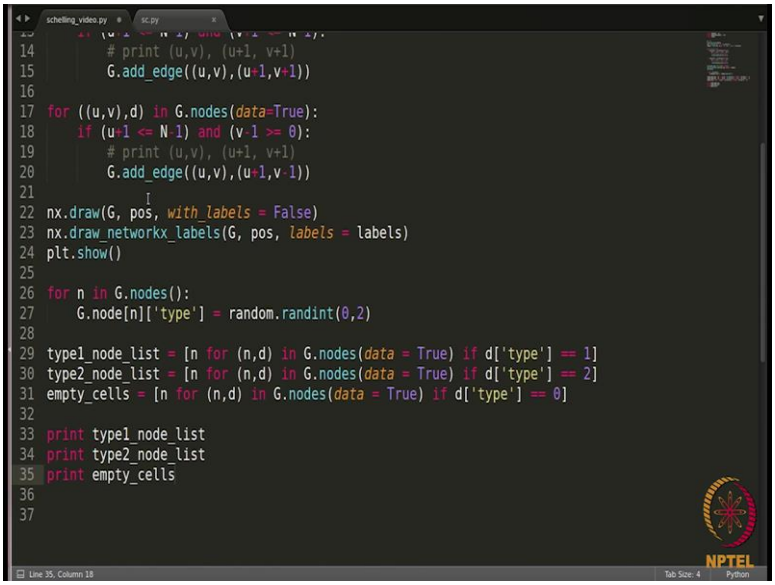


(Refer Slide Time: 05:11)



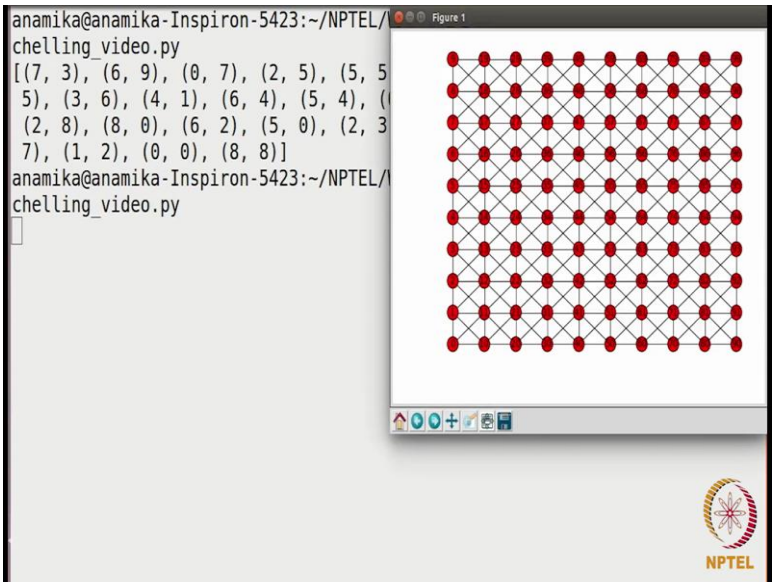
So, this is the list of the list of nodes which are n type 1 similarly we can print the other lists as well let me just verify and then we will continue empty cells yeah.

(Refer Slide Time: 05:20)

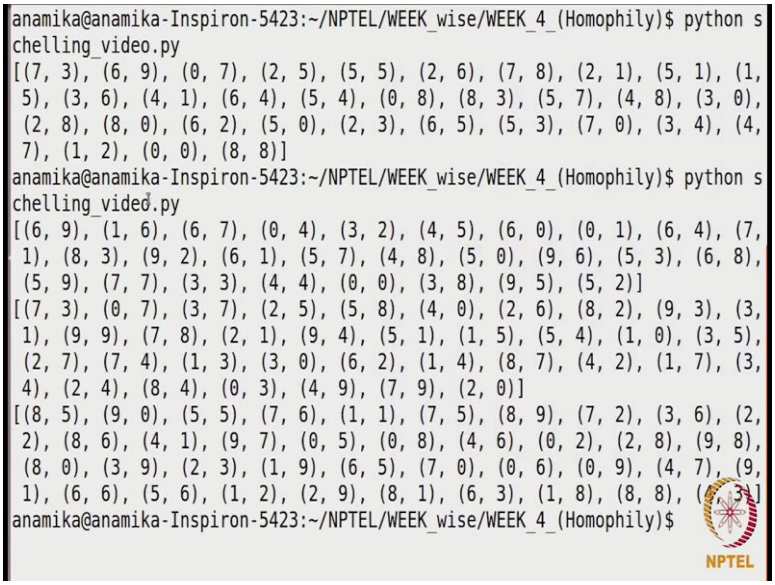


So, it is giving all the three lists we are going to make use of these lists. So, I am going to comment this because we do not need to print.

(Refer Slide Time: 05:32)



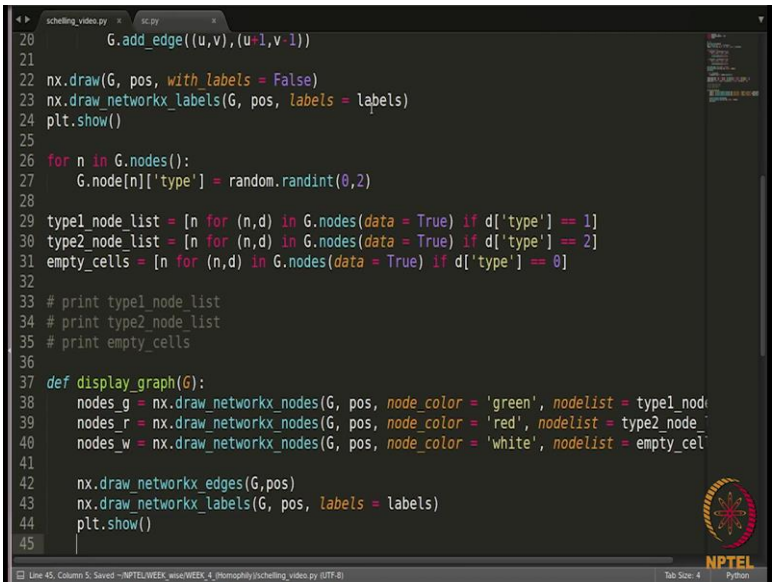
(Refer Slide Time: 05:34)



Now, what we can do is now that we have assigned type to every node. So, way we have 2 different kinds of people in our network it will be nice if he represents them with a different colour.

So, that it is easy to visualise the kind of people that are present in a given cell.

(Refer Slide Time: 06:09)



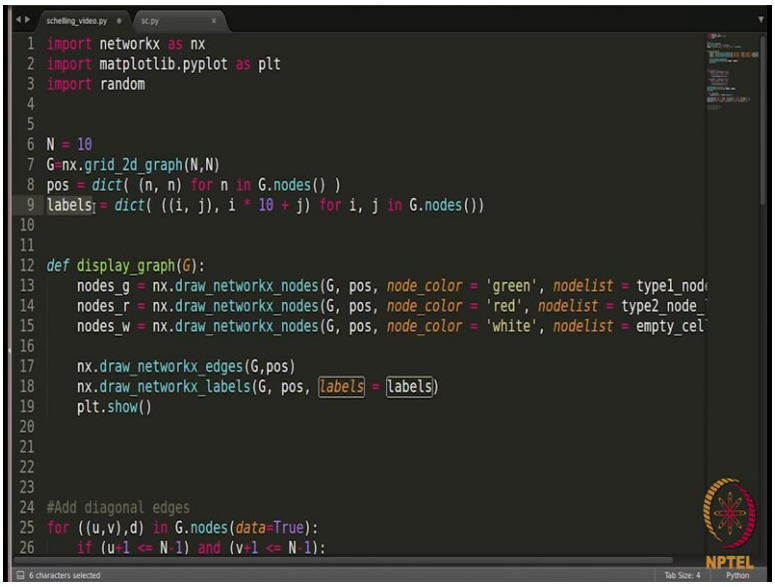
So, let me create a function for that I will create a function say display graph. So, I am just going to decorate the graph here it is up to you how much you want to do here how much you want t, how you want to display how much you want to decorate it is up to you can skip some of the a steps here. If you want who is a what I want is type one node should be displayed in say green colour type 2 nodes should be displayed in say red colour and empty cell should be say display displayed in white colour. So, the function that we will use and it is not draw network x nodes where we can specify a list of nodes that should be displayed by a given colour. So, what we can do is nx.draw networkx nodes the parameters will be G and this pos parameter that we posed already.

And what is to be want I everyone the node colours. So, I will write node colour is equal to green I wanted to be green and then what should be the nodes for which we want to as for whom we want to assign the pink colour. So, I will write node list is equal to type one node list right and going to copy the next kind of nodes we want in red colour. So, I will write nodes are I want in red colour and then type to we are doing this all this here because it will be nice to visualise the kind of transitions that take place. So, all though this is optional step it will be in it will help us to visualise the things I have just to go out the yeah. So, we want empty cells here I will write empty cells and I want why it here.

So, now we have drawn the nodes and, but the edges have not been drawn. So, we have to draw the edges. So, we will write nx.draw networkx edges and there again the 2

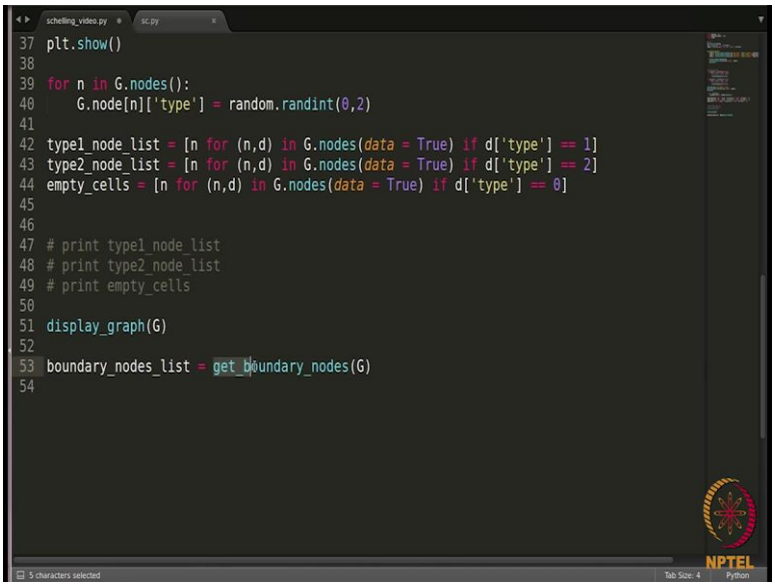
parameters we will pos G and pos the positioning since we are since we are manually draw in the nodes and edges labels we will not come. So, again we have to manually drop the labels as well. So, you write nx.draw_networkx labels and again we I think we used at already. So, yeah this one we can just copy this, they should work and anything that we skipped let us see plt.show. So, now, we have to call this function let me put this function on the top so that we can call it.

(Refer Slide Time: 09:37)



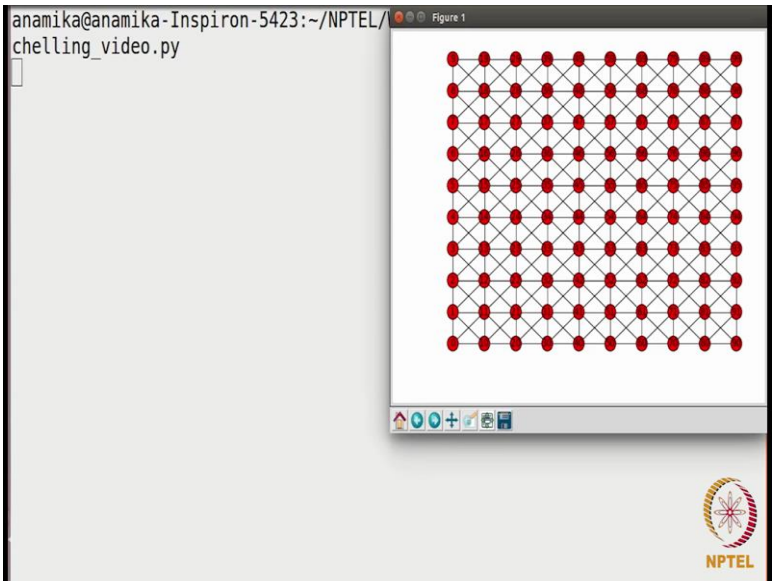
So, I will cut it from here and I will right it here this is because we want to make use of labels. So, I want to define the labels dictionary before I declare this function.

(Refer Slide Time: 09:54)

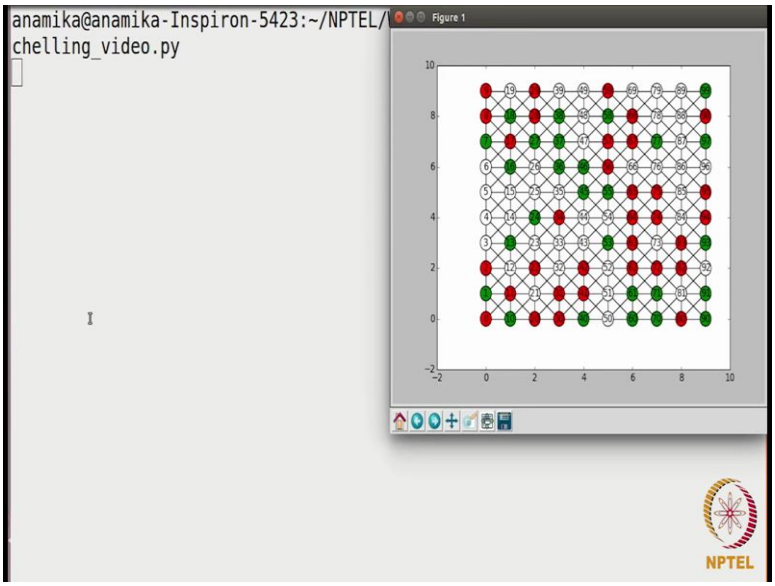


So, I will call this function here a write display graph G let me execute it.

(Refer Slide Time: 10:01)



(Refer Slide Time: 10:05)

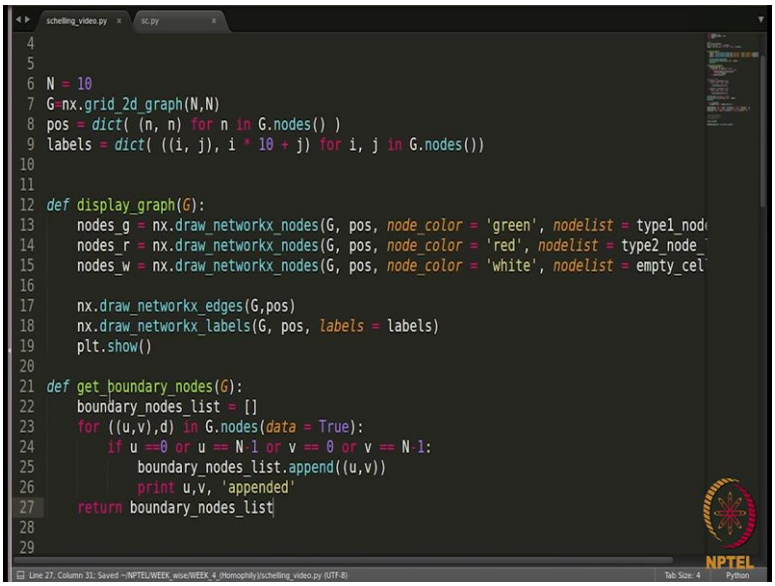


This is the graph that we initially displayed in this is the graph that now we are displaying which looks quite better and we are able to see which nodes belong to which type.

So, it looks pretty good if you want you can change the font size you can change the node size so that all you can play around. So, what is the next step we have the network we have the people. The next step is to identify the people were unsatisfied now how do if how do you find which people are unsatisfied we should be able to know for a given node which are its neighbour right it only then we will be able to you know tell whether this node is satisfied or unsatisfied. Now to be able to know which the neighbours of a are given node we should know whether the node is an internal node or is it a boundary node. So, that we must find out. So, for that matter it will be helpful if we create a list of boundary nodes as well as a list of internal nodes.

So, maybe we can create a function for that to get the list of boundary nodes. So, what we can write is get boundary nodes lets pos the graph here boundary nodes list is equal to this. So, basically we are going to put all the nodes which are aligning in the boundary into this list through this function which is get boundary nodes let us create this functional we will see how we can do that.

(Refer Slide Time: 11:58)

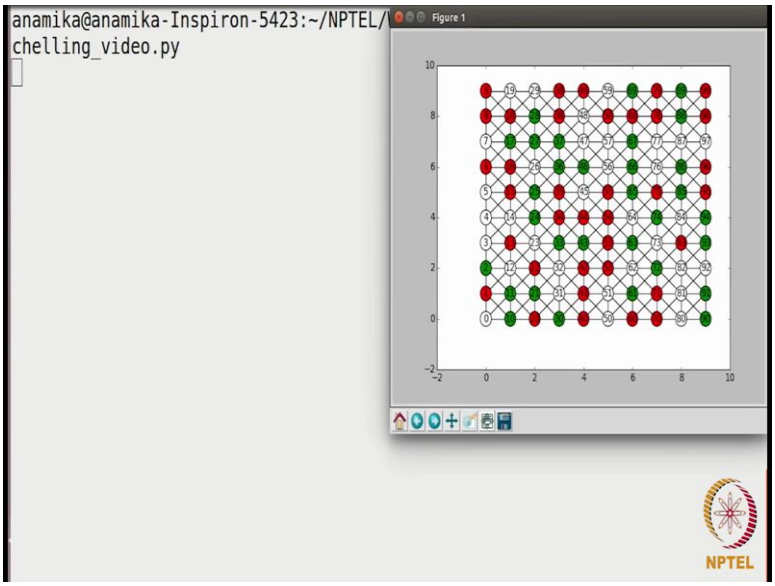


So, I will create this function here now how to do this we should know which nodes are boundary nodes if you remember the kind of grid that we were getting you will know that the nodes which have either u or v if (u, v) is a node the nodes which have either u as 0 or v as 0 is a boundary node also the nodes which have either u as n - 1 or v as n minus one they are also boundary nodes.

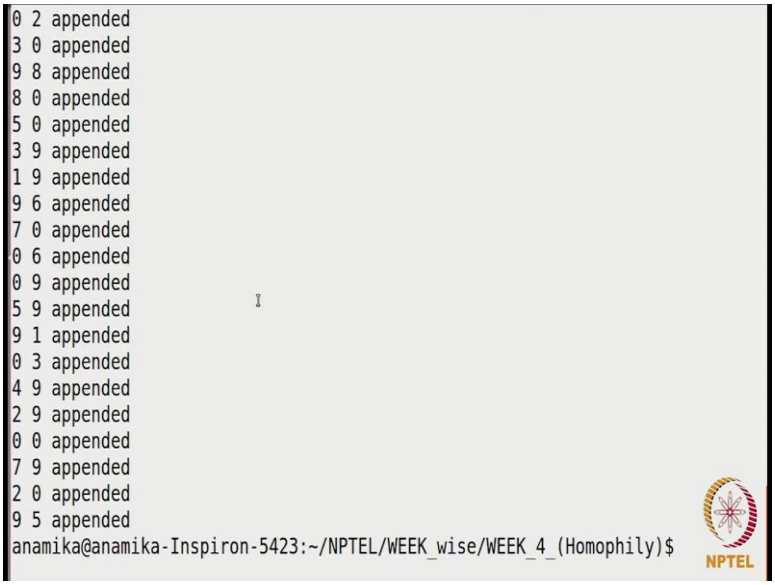
So, you can check the grid and see how it how it comes out to be. So, let us write this function lets create of lists which we will be writing. So, I will write boundary nodes list is equal to empty. So, I will write for u, v what else is there t which is a type we can actually skip the type here because we do not need this lets see in G.nodes(data = true). So, what I was saying was data is not needed for these particular steps. So, we can skip that is one and we can keep that is one. So, and choosing to keep it just like the now as I told you if u = 0 or u = n - 1 or v = 0 or v = n - 1 then the node is a boundary node.

So, what we will do is the boundary nodes lists.append we are going to append u v that yeah we should go let me print also it is to cross check whether things are fine or not. So, let us print this whenever a node is appended to the list there will be is this state. So, once that is we are going to written this list. So, we have already called this function yes let us call let us execute. Before that let me remove this displaying of the graph because now we are because now we have created a function and we are getting a better visualizations.

(Refer Slide Time: 14:35)



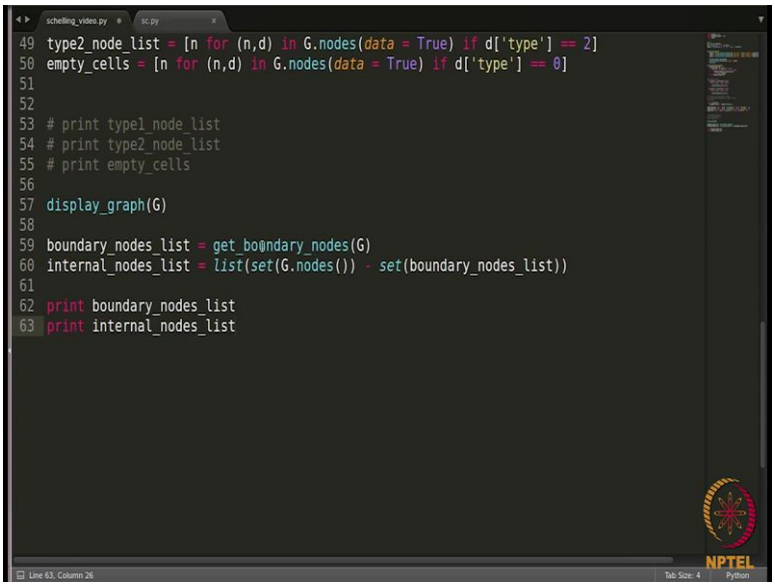
(Refer Slide Time: 14:38)



So, I have commented on that. So, this is the graph I am closing it yeah. So, we are we are getting things working fine these are the nodes which are appended to the list.

Now, the second step is to get the list of the internal nodes right. So, what we can do is if we subtract the boundary nodes from the total nodes; obviously, whatever remains is the list of internal nodes.

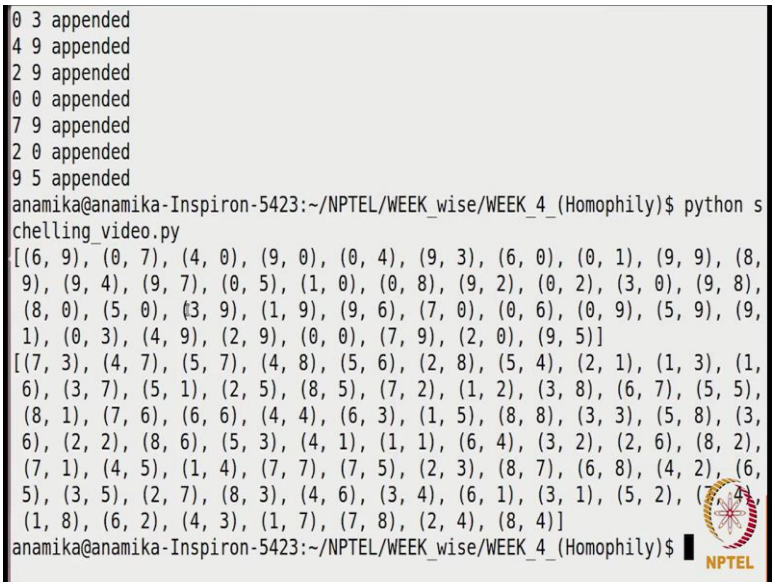
(Refer Slide Time: 15:05)



So, what we can do is I will create a list for internal nodes internal nodes list is equal to; now what is this I told you this technique if you want to subtract one list from the other you can use is set function. So, I am going to convert list of all the nodes into a set. So, I will write G.nodes; yeah minus set or should I write boundary nodes list yeah this work. So, let me try printing these 2 lists just to cross it.

So, I will write print boundary nodes list and print internal nodes list yeah let me command that appending. So, that we get only what we want to print (Refer Time: 16:08) that yeah in this function I am going to commentary this let's check yeah. So, we are getting 2 lists here first lists are boundary node list and second list is internal nodes. So, we are doing well.

(Refer Slide Time: 16:20)

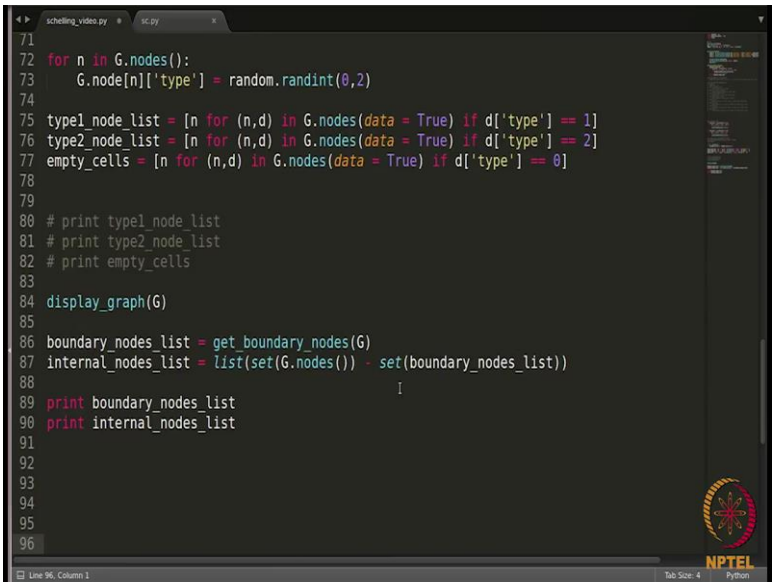


So, now we have a list of boundary nodes internal nodes what is the next step the next step is to identify the unsatisfied nodes and then moving them to a new location we will do that in the next part of the video.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 60
Homophily (Continued) & Positive and Negative Relationships
Schelling Model Implementation – Getting a list of unsatisfied nodes

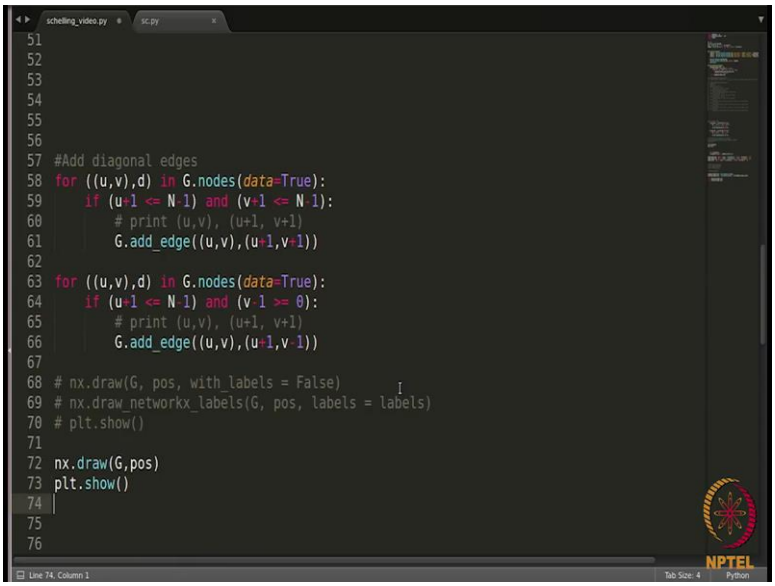
(Refer Slide Time: 00:06)



In the previous video we computed a list of boundary nodes and we computed a list of internal nodes as well. Now what is the next step? We have to find a list of unsatisfied nodes so that we can move to a new location one by one. Now how do we get a list of unsatisfied nodes? So, basically, we will take all the nodes from G.nodes one by one and we will test whether they are unsatisfied or not. Now how do we do that? We need to get a list of neighbours of that node if it is a boundary node it will have a different set of neighbours if it is an internal node it will have a different set of neighbours.

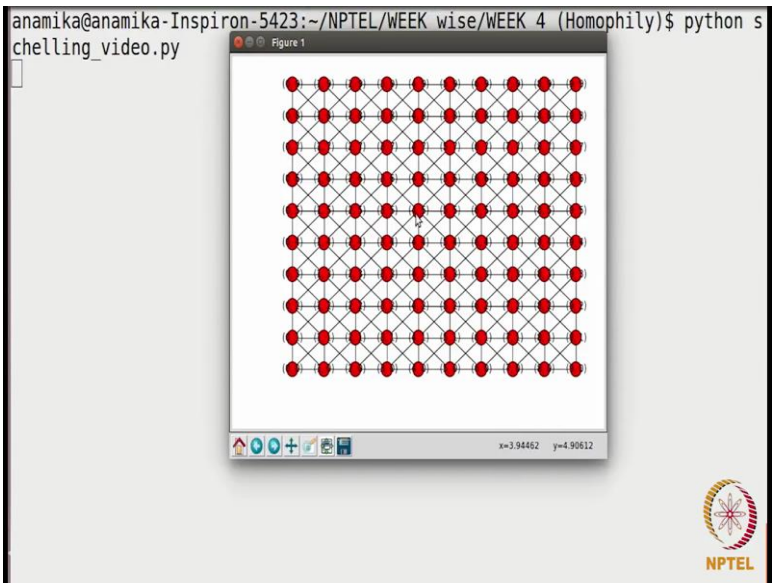
So, the first step is to get a list of neighbours for the given node. So, we might need to create 2 functions for that one function for getting a list of neighbours if the node is a boundary node and another function for getting a list of neighbours if the node is an internal node. Now let me show you how we can do that let me comment this I want the previous version of the graph here.

(Refer Slide Time: 01:20)



So, let me show you that once, I will just draw the graph the normal way here. So, I will write nx.draw(G) and pos position also I want to pos and then plt.show.

(Refer Slide Time: 01:37)



Now, let us see what we get yeah. So, see if it is an internal node for example, let us take the case of 4, 5.

Now, what are its neighbours 4, 5 has 8 neighbours now what are these; the left one that is 3, 5 the right one that is 5, 5 the bottom one the top one now how do we get them. So, given a node (u, v) how do we get the neighbours (u, v) we will have a neighbour which

is (u – 1, v) it will have a neighbour which is (u + 1, v) it will have a neighbour which is (u, v + 1) it will have another neighbour which should be (u, v – 1). Now next come the diagonal nodes. So, they will be (u - 1, v + 1), (u + 1, v – 1), (u + 1, v + 1), (u - 1, v – 1).

So, whatever I told you are going to be the neighbours of an internal node which is (u, v). So, that was about the internal nodes. So, we can write a function where we pos (u, v) and we get the list of all the neighbours now if the node is a boundary node it will have different set of neighbours. So, we need to create a separate function for that now in case again there are 2 kinds of boundary nodes the nodes which are in the corner the 4 nodes and the rest of the nodes now again there will be special cases there if the node is 0 , 0 I do not think they can generalize that. So, I think they going to have to do that case by case.

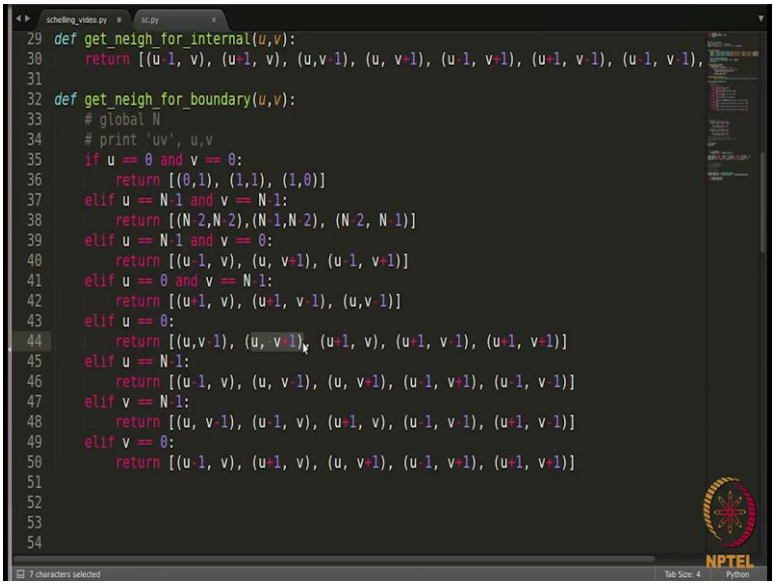
So, if the node is 0 , 0 its neighbours will be 0 1 1 1 1 0 if the node is a nine , 0. So, if you want to generalize you can just write if the node is N - 1 , 0. So, you remember we passed N = 10 here. So, N - 1 , 0 is going to be corner node similarly N - 1 , N - 1 is going to be in other corner node and 0 , N - 1 is going to be another corner node now these for special cases for which the neighbours are going to be only 3. So, we can write these special cases there the rest of the boundary nodes are these once. So, one list of one chunk of boundary nodes are going to be the once where u is equal to 0 right and in those cases the boundary the neighbours are going to be 1 2 3 4 5.

So, if it is 0 , v it will be (0, v + 1), (0, v – 1), (1, v + 1), (1, v), (1, v – 1) you see how I am doing that I think you can do that for the rest of the cases where let me give you one more example. So, if it is amongst the nodes where v is equal to and minus 1 as you can see it will give us this row of boundary nodes right. So, again and those cases you can write the and the neighbours that are going to be there and this is going to be the row where u = N - 1 and this is going to be the row where v = 0. So, you can write all those special cases and you can get a list of the neighbours for the given boundary node.

Let us go back now to save time I have already written all the cases. So, that we do not spend a lot of time in the video making. So, let me show you all though I have already explained to you. So, this is the function get neighbours for an internal node. So, we are posing (u, v) and for N internal node the cases are very straight forward. So, then you completely generalized it. So, you are you are returning a list there which is having all

the 8 neighbours. So, that is straight forward another function that I have created already is forgetting a list of neighbours for a boundary node as I told you.

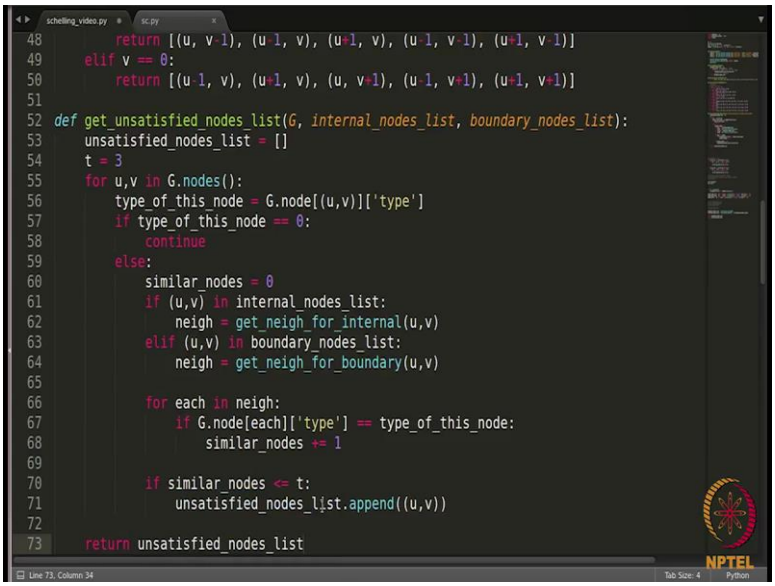
(Refer Slide Time: 06:02)



There are 1 2 3 4 8 cases. So, this you can get. So, there are this also I comment. So, you see there are 8 cases here 1 2 3 4 5 6 8 cases all the cases that their as I told you first 4 cases are the ones for the corner nodes and the rest 4 cases are the ones for the rest of the boundary nodes.

So, I do not think I need to explain much there. In fact, I would like you to try yourself N case you need help we can just check the values here. So, these are all possible values you can just open the graph the way I opened and then you can note down the neighbours are going to be there that that should be simple to do now we have to get a list of unsatisfied nodes.

(Refer Slide Time: 06:56)



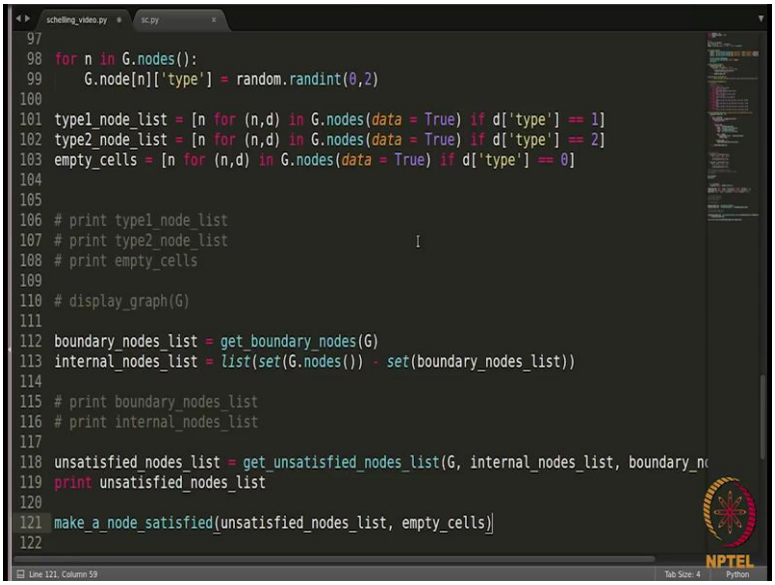
So, we will create a function for that we will write get unsatisfied nodes list lets pos graph here. Now we will check all the nodes one by one and we will check how many neighbours of the given node have the same type as the as the type of the given node right. So, let us create a list; unsatisfied nodes list and (Refer Time: 07:34) initially now we have to check for each node. So, we will write for (u, v) in G.nodes.

We have to keep a node of the type of this node that is (u, v). So, we can write type of this node because we will be comparing write type of this node is equal to G.node, we have to see the type of u v. So, you write u v and then type here there should give us the type of the node. So, we have stop here in case the type of this node is equal to (Refer Time: 08:18) what are we going to do we basically do not have to do anything as in if the type of this node is equal to 0 it can never be unsatisfied right. So, we will just continue, and we will check the next node. So, we will write if type of this node is equal to 0 we just continue and check the next node as for now we are going to do.

We are going to see in the list of neighbours of this node how many nodes are having a similar type as this nodes type. So, we will have to keep drag of that. So, we will write similar nodes is equal to 0 initiate initially now we have to check whether this node u v is a internal node or is it a boundary node because accordingly we will get the list of neighbours. So, we have to check here now do we check whether or node is internal node

or boundary node we have already created a list of internal, internal nodes in boundary nodes here, right.

(Refer Slide Time: 09:22)



So, it will be nice if you pass this these lists into the function boundary_nodes_list. So, we will pass these 2 lists in the function and we will pass these lists and we will check whether the node belongs to this list or not that we will tell us whether the node is boundary node or internal node.

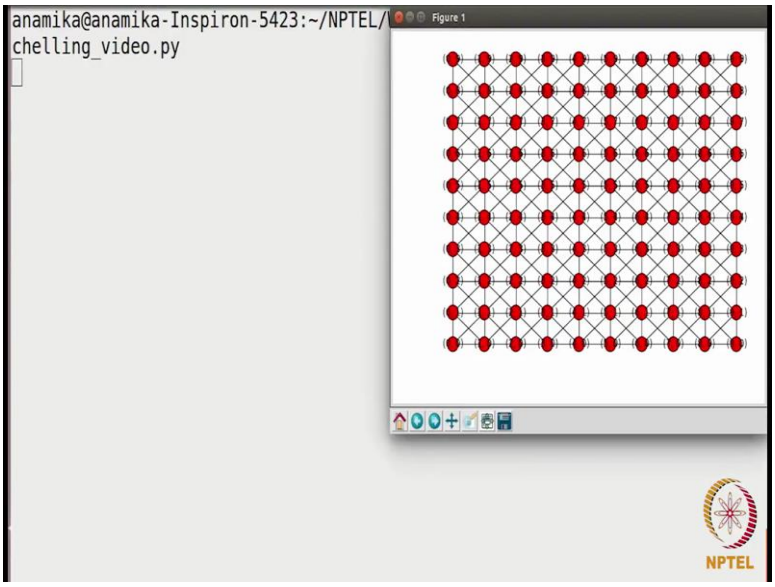
internal_nodes_list: so, what we have into do here we will check if (u, v) sorry in internal_nodes_list which will mean that it will it is an internal node then its neighbours. So, you must get all its neighbours how we get its neighbours we have already created a function for getting the neighbours of an internal node we will write get neighbours for internal, right. So, we are going to pos the node here in case this u is not an internal_nodes_list it will (Refer Time: 10:33) in the other list that is boundary_nodes_lists. So, we will write for (u, v) in boundary_nodes_list what do we do now we get a list of neighbours again we have created a function for that get neighbour of boundary right for boundary yeah get neighbour for boundary and we will pass the node here you will.

So, now we now got a list of neighbours what do we have to do now we have to check amongst these neighbours how many of them are having the same type as the type of this node that is what we have stored already. So, we will start a loop for all the nodes in the

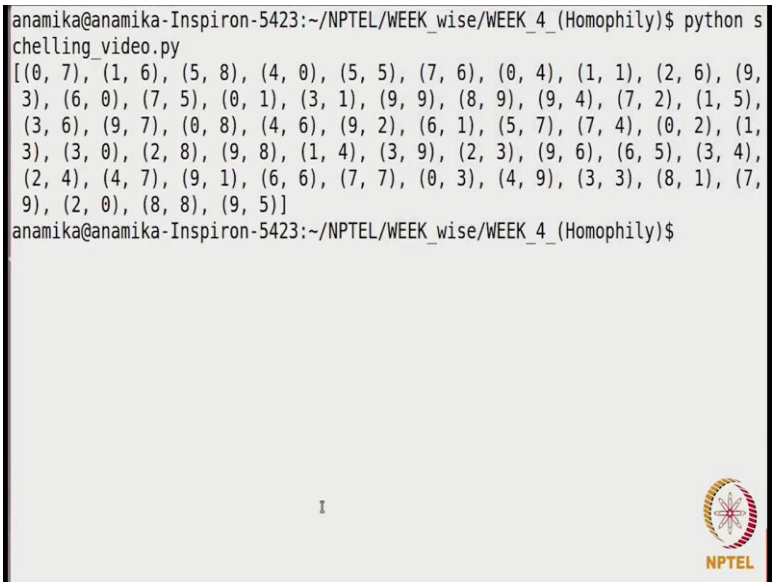
neighbour. So, you write for each in neighbour if G.node. So, we have to check the type of each. So, I will write if I am sorry if this is equal to type of this node, we have we have to increment the counter that is we already initial right similar nodes plus is equal to one. So, number of similar loads nodes well be incremented after this loop is over, we will have a count of the similar nodes. So, if that similar node is less than equal to the threshold. So, maybe you can initialize the threshold here maybe t = 3. I will write here.

So, if the same number similar node is less than equal to t this should mean that the node is unsatisfied. So, we will at this append this node to the list of unsatisfied nodes. So, write unsatisfied node list or append u v. u v and this function is going to write the list of unsatisfied nodes I will write return unsatisfied nodes list. Now let us call this function here. So, we will write unsatisfied nodes list is equal to get unsatisfied no nodes list and the parameters are going to be the graph and internal_nodes_list and boundary_nodes_list yeah in order to verify let us print whether it works it works fine or not. So, I will print unsatisfied nodes list let me comment this I do not want to see them. So, like see. So, this is the graph we are getting the list of unsatisfied nodes here. So, we are doing good. Now what we have to do?

(Refer Slide Time: 13:19)



(Refer Slide Time: 13:22)



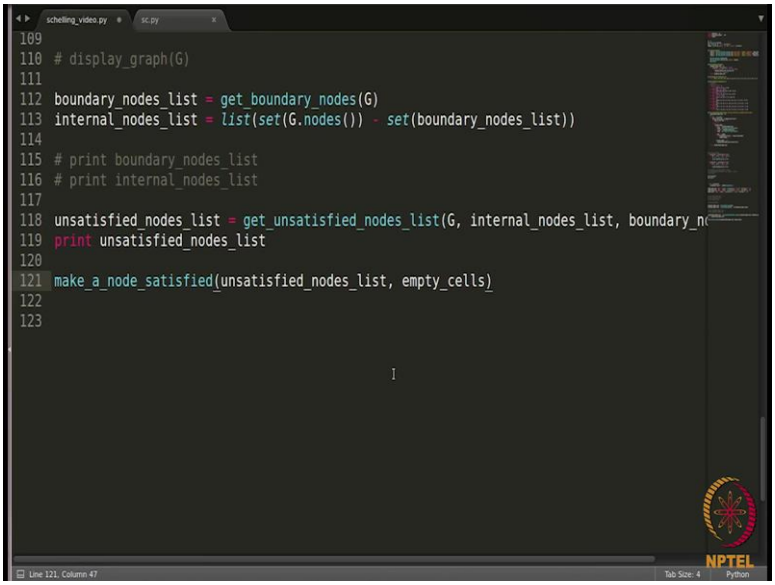
After getting a list of unsatisfied nodes we have to take the nodes take one of the unsatisfied nodes and we have to move it to a new place now that new place should be empty, right. So, we also have to keep a track of the empty places we initially did that we kept a list of empty cells. So, that we have already. So, what we will do is I will go back what we will do is from this list of unsatisfied nodes we will choose one node randomly from the list of empty cells we will choose one place randomly we and then we will put the randomly chosen node from the unsatisfied list into this randomly chosen position from the empty cells and after that; obviously, the number of unsatisfied and satisfied nodes we will change. So, the graph we will actually change after that we will re compute all the values that is unsatisfied empty satisfied everything will be re computed and then we will see how the graph looks like this we will keep on repeating for several iterations and in the end we will compare how we are how the graph is looking like and how the graph is looking like initially.

So, as I said the next step is to make or node satisfied we can create a function for that make a node satisfied what do we need to pass their we; obviously, need to pass the list of unsatisfied nodes because we will randomly choose one out of that and second thing that has to pass just the anti cells list because we will run issues one position out of that I think this is good. So, in the next video, we will implement this function make a nodes satisfied.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

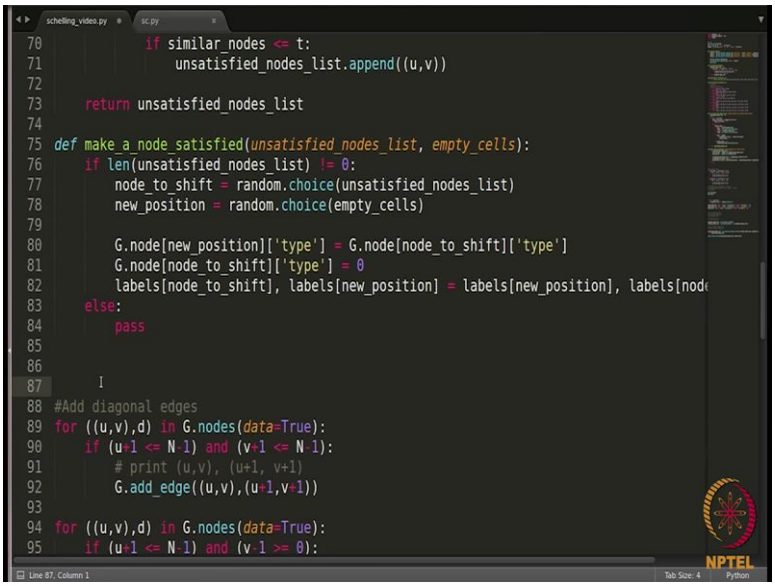
Lecture – 61
Homophily (Continued) & Positive and Negative Relationships
Schelling Model Implementation - Shifting the Unsatisfied Nodes & Visualizing the Final Graph

(Refer Slide Time: 00:05)



We now have list of unsatisfied nodes which we completed in the previous video and we have to now choose one node out of that list and we have to make it satisfied; how do we do that we have to move that node to an empty cell which also we will be randomly choosing. So, let us create this function.

(Refer Slide Time: 00:35)



So, let us create this function, here I will write this. So, in case the list is empty we will I add just one condition here if length of unsatisfied nodes list is not equal to 0 then we will do or this and what is that we have basically have to choose one node randomly out of this list we will make use of the function rand dot choice.

So, let us write a node to shift the one we which we will be shifting in this particular iteration. So, in one iteration we will be shifting 1 node. So, we will choose the node randomly random dot choice unsatisfied nodes list now where do we shift this node again, we have to choose one empty location randomly. So, maybe we can write new position is equal to random dot choice from empty cells now we got the node and we have to move it what kind of changes we will happen when a node is shifted. So, basically the, so, here we have on the left-hand side assume we have a node which is to be shifted on the right-hand side we have another node which is empty.

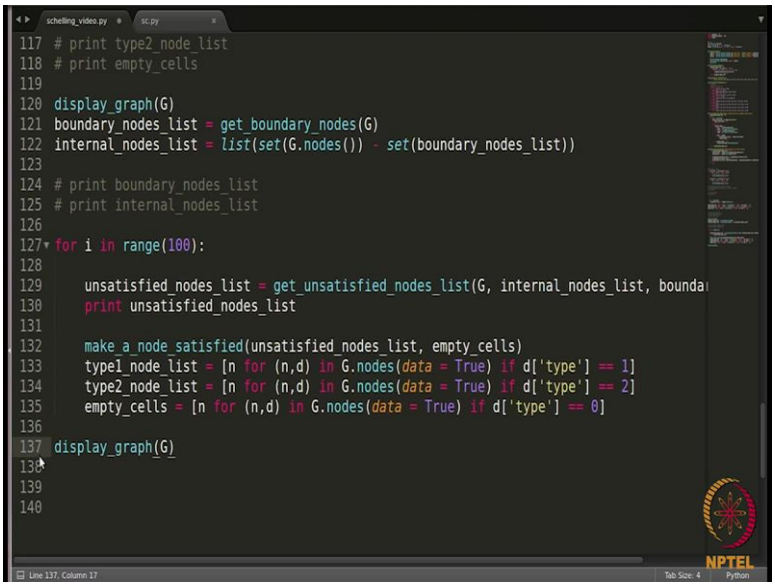
So, the type of the empty node is right now 0. So, the type of this empty node will change to the type of the node which is now coming into this cell that is one thing also when we look at the left hand side when this node is shifted this location will become empty. So, the type of this node will become 0. So, this changing of type will happen as the first thing second thing that has to happen is the changing of the labels. So, the label of the node on the left-hand side and the node on the right-hand side will be exchange will be exchange yeah basically. So, I hope you are able to imagine how I am going to

explain you. So, let us first change the type. So, I am going to change the type of the empty cell to the type of a node which is going to be shifted. So, I will write G dot node new position is basically the location of the news of the empty cell type is going to be equal to the type of the node which is node to shift.

So, now we have change the type of the empty cell to the type of the node which is going to be coming into this cell and the node which got shifted will now be 0 as type. So, I will write G.node to shift it is type will now be 0 done second thing that has to be done is to exchange the labels. So, in python exchanging is very straight forward 1 1 simple command we can just to; now where are we storing the labels if you remember initially, we created this dictionary that is labels, right. So, this dictionary is having all the labels where this is where we have to make changes. So, in this dictionary labels there will be a label for node to shift there will be and in this dictionary labels there will be a label for the node new position already we have to exchange that.

So, let me I am sorry how do we do it x comma y is equal to y comma x as implies that this is how you exchange contents of two variables this no need for any temporary variable. So, that is this while do is well labels node to shift comma labels new position is equal to labels new position comma labels node to shift I think where done in the function let me write the else part of this if. So, in case the there is nothing in the unsatisfied nodes list basically all the nodes are satisfied we are not going to do anything. So, just we will write a pass nothing to be done. So, we are done with this function which is make a node satisfied now let us get back, here we call this function make a node satisfied after which the list of type a nodes and the list of type two nodes and the list of empty cells we will change and we will need the those lists again. So, they need to update them. So, what is going to write is type one I think we did that in this copy paste let me yeah.

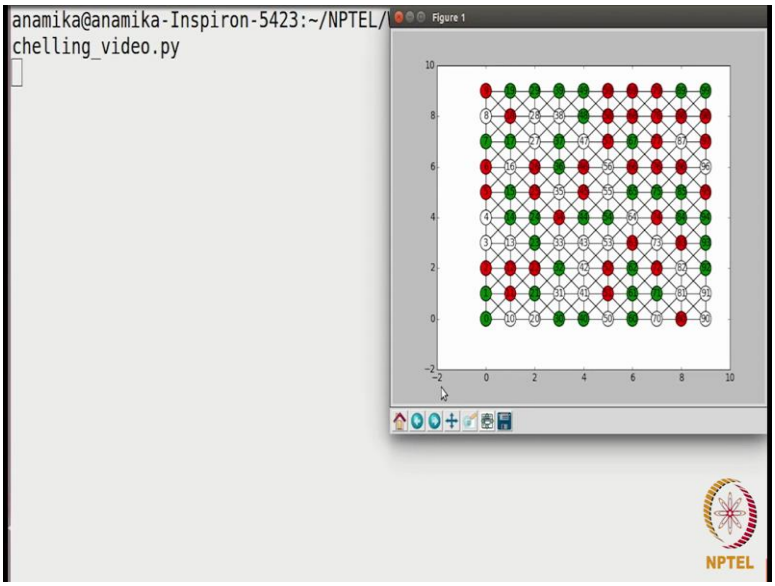
(Refer Slide Time: 06:02)



So, this is where we computed these 3-list type one node list type two node list empty cells now after node is shifted to another location these lists are going to undergo changes is well. So, we need to re compute them. So, I am just copying pasting here over here now how many times we do and then do we have to do it as you see make a node satisfied only makes one nodes satisfied. So, we have to keep doing this for all the nodes which are unsatisfied. So, maybe we can start a loop here. So, I will right for I in range you can change recording or you can keep a number over here which is dependent on the total number of nodes in the grid I am for now I am keeping it just hundred. So, basically, we have to repeat all these steps until all the nodes gets satisfied. So, this all will go under this loop once this is done, we are going to display the graph again. So, we will call this function display graph G.

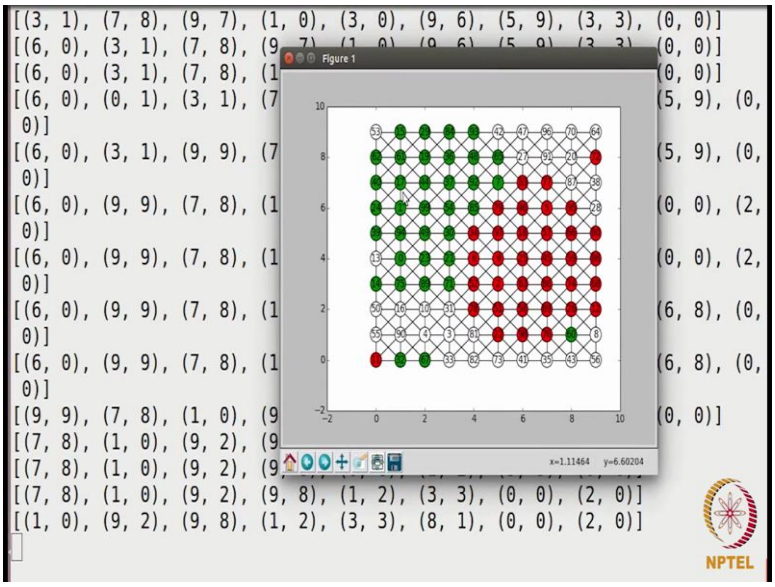
Now, before I will execute let me it removes these least commons common based commons which where displaying the graph in the very first form. So, let me trend the let me display the initial graph is well display graphs. So, we are going to we are just displaying the initial version of the graph and after that we have this loop in which all the shifted will happen and in the end this is how the graph this is the command that will display the final graph. So, this should be displaying the initial graph they should be displaying the final graph and in the middle, they are these hundred iterations let me increase the number of iterations here. So, what should basically happen the nodes should move towards nodes of their own time?

(Refer Slide Time: 08:31)



So, that is what is expected let us see how it works lets executed and see how it works. So, this is the initial graph as you can see the nodes of different colours are scattered. I am closing it and now.

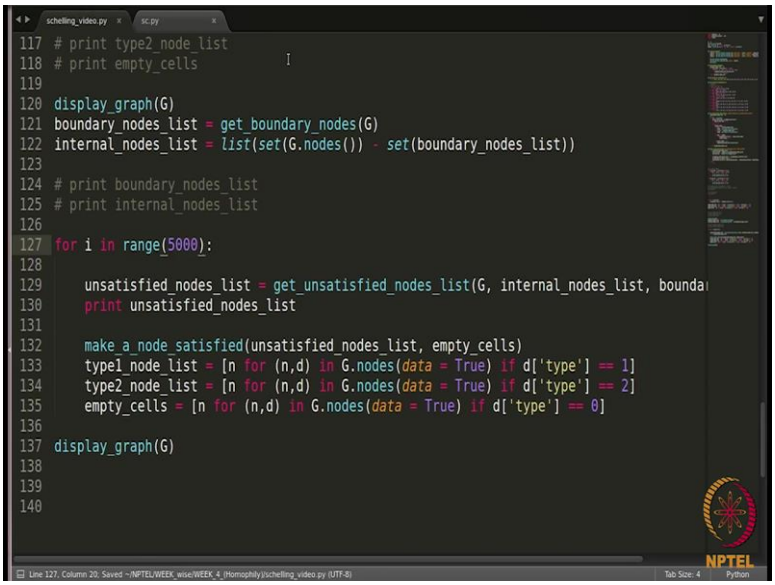
(Refer Slide Time: 08:42)



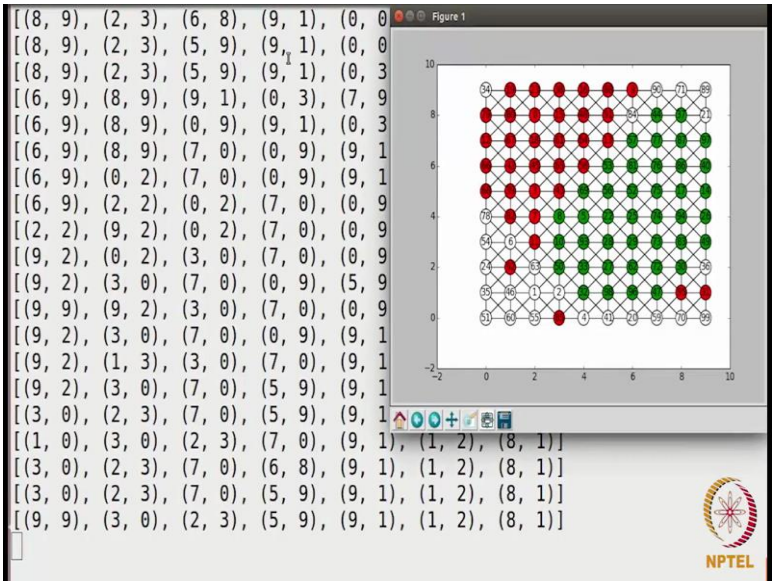
So, this is the final graph as you can see there are all the green nodes are on the one side all the red nodes are on the other side and there are few other nodes as well with scene unsatisfied is well maybe we can increase in the on the counter there. So, that counter will basically depend on many things it could depend on the size of the network that if

we have taken. Secondly, we have assigned the nodes randomly. So, for different random settings you might require different number of iterations you can also keep a track of the number of hydration that that were required to convert this graph into a graph where the people are same type on together.

(Refer Slide Time: 09:25)



(Refer Slide Time: 09:29)



10,000 seems too much; may be 5000; 5000 let us try for these many iterations. So, it is going its going good still there are a few unsatisfied nodes, but in more iterations they will converge they will converge basically it. So, happens that all the nodes become

satisfied another thing to note here is that you can also change the value of t. So, here we have taken t is equal to 3. So, even for an internal node since it has 8 neighbours even out of 8 neighbours only 3 are of its own type the node will not move to any other place. So, so you can change t to 4 5 or any other value and you can see the number of iterations that were required to get to the convergence. So, you can I think you have gotten enough tools now you can play around with all these values you can keep changing and you can observe and if needed you can plot things as well as in the number of iterations required for converging the these many number of nodes you can change the value of capital N which we initially to a capital N was 10 which means aware 100 cells.

So, that was about this Schelling model I would suggest you code yourself and you know watch the video only when it is needed see in the initial video I think I gave the structure as to what has to be done. So, if for every video you do that in the sense you just listen to the structure and the aim that has to be implemented and then code is code it yourself without just following the video that will be actual learning for you. So, I would suggest you to code yourself and check the video whenever needed only.

Thank you.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

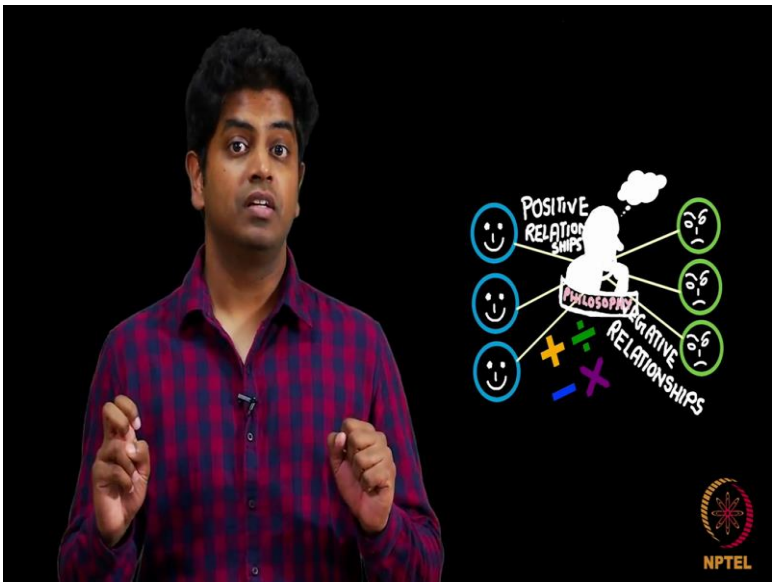
Lecture – 62
Homophily (Continued) & Positive and Negative Relationships
Positive and Negative Relationships
Introduction

(Refer Slide Time: 00:15)



So, let us start with a new chapter, here is an interesting chapter for more than many reasons one being it is both philosophical as well as mathematically very elegant. We will sort of talk less of the data sets here and we will speak more of logic probably the only chapter where this kind of an approach we are going to take.

(Refer Slide Time: 00:37)



Let me motivate you with a nice question; in our life we make a lot of friends, some of them are good, some of them are not so good. Now in fact, we have I can classify them as positive and negative relationships with some of them we get along very well we are very close to them it feels very good being with them on the contrary we sometimes do not like being with some people this is how life is that is the philosophy part of it.

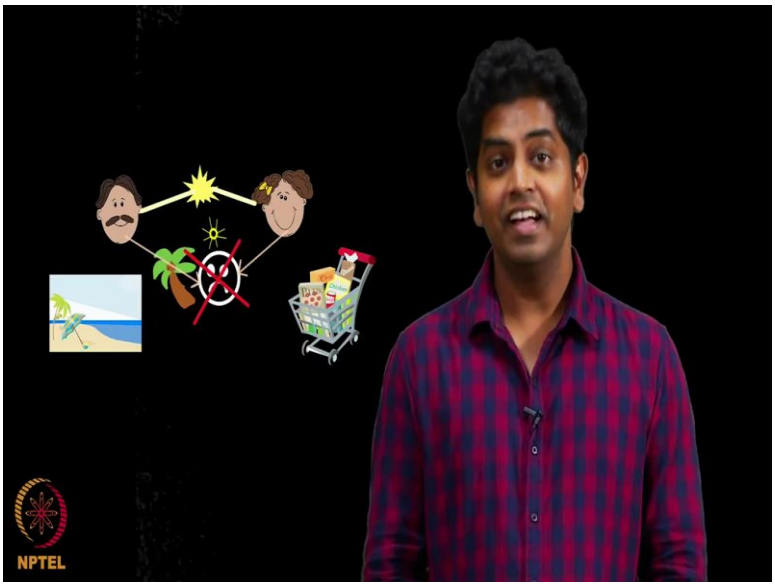
But what should entry give is the mathematics of this can be very well studied and the mathematics of it is both eyebrows rising as well as eye opening. So, let us see more of this. In fact, this is one chapter that is my personal favourite given the counter intuitive results that we all will see let us get started.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 63
Homophily (Continued) & Positive and Negative Relationships
Structural Balance

So, let me motivate you with a very personal example and then let me talk about an abstract example a personal example.

(Refer Slide Time: 00:15)



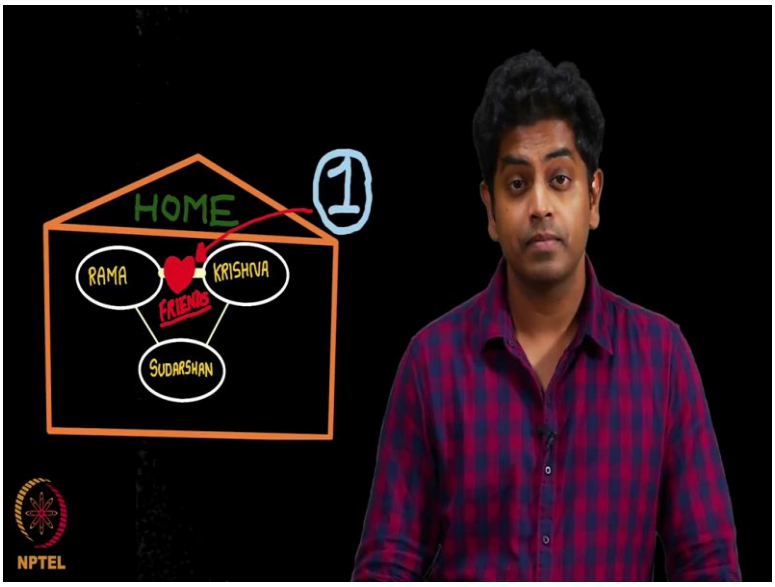
I used to spend my vacations with my uncle and aunt a very loving couple very nice to hang around with them, but occasionally they used to sort of have some disagreement and they would they would fight for a minute or 2 and that is when I would be in a fix for a simple reason that uncle and aunt would start controlling me. Uncle would say come Sudarshan, let us go to the beach; aunt would say no, no, no, no, come let us go shopping, I get you something to eat uncle would say no, no, no, if you going with your aunt and I am not going talk to you; aunt would say the same; if you are going with your uncle I am not going talk to you.

So, I am in a fix here, look I cannot go with only one person here, there will be hatred from the other person; what do I do I cannot go with both of them, I cannot go with one of them. The best thing here is to say call it quits I am not going with anyone, I am

neither going with uncle nor with aunt this is to be on the safer side right. We all of us would have encountered such a situation in our life you love 2 people equally and they ask you to take a call love me or love the other person you cannot love both. So, it is so happened that I would take the excuse of I have homework and I would say start scribbling something and things would come to a standstill and then they would become normal.

This is my personal example let me try using abstract example to explain the same thing and I just use personal example. So, it people can relate to it and abstract example.

(Refer Slide Time: 01:57)



Let me consider this friend Ram and Krishna, Rama and Krishna; these 2 people are very close to me let us say very good friends best friends, but unfortunately, they do not like each other. Here is the similar problem I told you of the uncle and aunt example I cannot invite Rama and Krishna home for a party for a simple reason that in case they come for the party they would not talk to each other.

Party is done to distress yourself if you have 2 such people in our party it will add to your stress. So, what will happen? Think about it for a second Rama and Krishna come home they sit together they do not talk to each other; they do not want me to talk to one-person other person gets offended what will I do. There are 2 consequences of such a thing, in my life if I want to continue being friends with these 2 people I must take a decision one by hook or crook make them become friends that is a nice decision if it can

work. But generally does not work 2 people who hate each other now we would want to continue hating each other what do I do? I will try my best to make Rama and Krishna become friends with each other I will convince them take them out with them I will play football with them party with them no, it may not work if it works well and good if it does not I am in a fix.

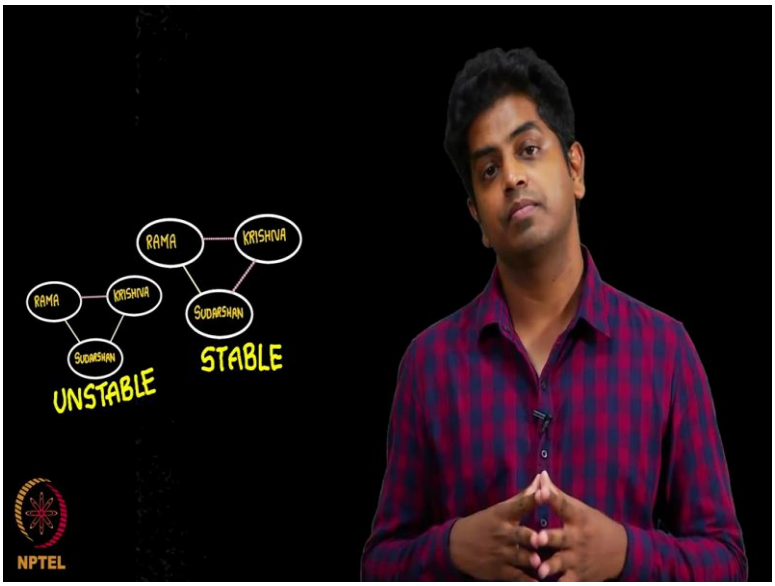
Why I am in a fix it is because Rama will say Sudarshan enough is enough stop being friends with Krishna otherwise I will stop being friends with you Krishna will tell me the same thing. So, what is happening here think for a minute let me draw a picture and then observe what is happening here?

(Refer Slide Time: 03:53)



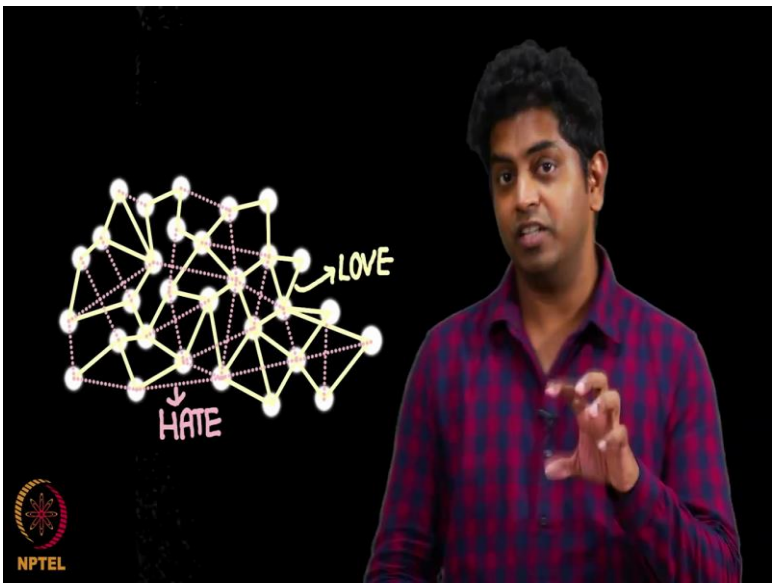
Here is me and I have I am friends with Rama I am friends with Krishna Rama and Krishna hate each other now this kind of a triangle with 2 lines and one dotted line dotted line representing hatred and straight line representing friendship is a dangerous combination. This will not continue to be stay this way for a long time eventually either this dotted line should become a straight line you say what I am saying either Rama and Krishna should become friends because of Sudarshan's involvement or whatever or Sudarshan should start hating one of them.

(Refer Slide Time: 04:38)



In case Sudarshan starts hating Krishna the triangle will look like this Sudarshan likes Rama, Rama likes Sudarshan of course, and Rama of course, hates Krishna as and always Sudarshan has started hating Krishna this is a stable situation. The previous one was actually an unstable situation unstable simply because there is pressure for the structure friendship structure to change now this was with 3 people imagine there are thirty people in a organization is a lot of hatred lot of friendship what will happen.

(Refer Slide Time: 05:10)



This entire chapter is about studying this and studying this and inferring some really cool results lets gets started with more results right now I will now make an attempt to enumerate all possible cases what do I mean by all possible cases.

(Refer Slide Time: 05:35)



You see there are 3 people A B and C between A B and C; what all are possible 3 friendships are possible as you can see case 1.

(Refer Slide Time: 05:46)



(Refer Slide Time: 05:55)



Case 2; 2 friendships and 1 hatred, this is case 2 what is the next case as you can guess 2 hatred one friendship.

(Refer Slide Time: 05:59)



The third the fourth case is all 3 hatreds, right, let us take this one at a time and then observe what are the consequences of these cases what are the cases that are stable what are the cases that are unstable. I hope you are following what I mean by stable and unstable by stable I mean this kind of a friendship does not have any dangers.

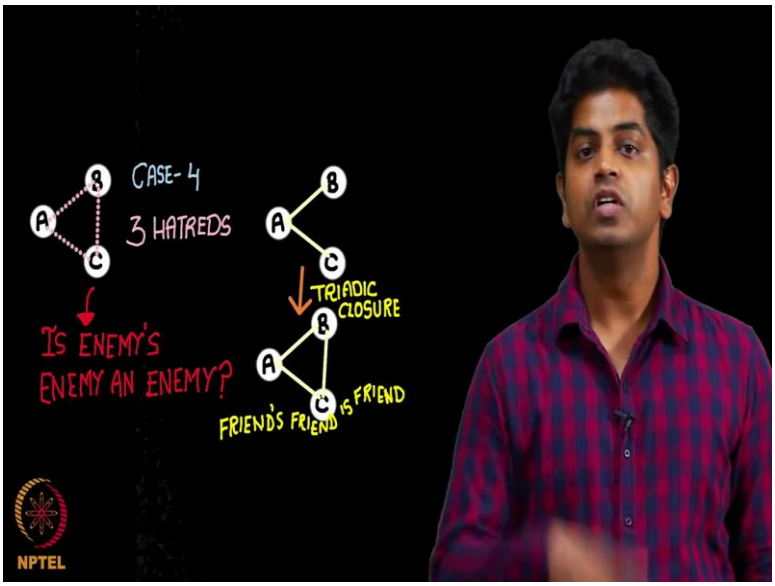
(Refer Slide Time: 06:29)



For example 3 friendships I am there is no problem there at all versus the case that I told you awhile before Rama Krishna case what you can see here 2 friendships one hatred. Now that is dangerous it is unstable eventually something we will happen to this kind of a triangle right either it should become a complete 3 friendships or 2 hatred one friendship as discussed earlier what I will do right now is I will go slowly with all these cases and see what case has what consequence. So, first case 3 friendships a happy with need I same over 3 people who are friends with each other I mean there you have a wonderful life.

There is absolutely no hatred between anyone period there is just nothing to study here at least in this chapter 3 friendships is stable there is no pressure it similar to my Rama Krishna that is a tough life here is a happy life.

(Refer Slide Time: 07:38)



Now, what if there is 3 hatreds A B and C, there 3 people hate each other by the way let me ask you a question remember triadic closure friends friend is a friend do you think and enemies enemy is an enemy think about it a friends friend is a friend the same rule applies here it is. In fact, true that an enemies enemy is actually an enemy think about it I am right is I just because of professor says something then he will be right pause for a second analyze the situation and tell me is it really true that enemies enemy is an enemy if you are not clear just watch this video clip and come back and I am sure you will understand what is wrong with this statement enemies enemy is an enemy.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 64
Homophily (Continued) & Positive and Negative Relationships
Enemy's Enemy is a Friend

Hey, are you Anamika, I have been teamed up with to do the artificial intelligence assignment given by Professor Sam.?

Yeah, I am Anamika, but I have to go home this weekend, can we start on Monday?

Oh, no, this requires a lot of hard work and you going home will only delay the submission.

I am sorry, but I have to go home, I can start from Monday only.

Fine, but you do not simply understand that sometimes we have to be adjusting.

You do not tell me what to do I know who I am and what I have to do.

Well, anyway I will start with the assignment I have no other option other then working with you, the only other girl left is Deepika and I do not like her at all, it is better to give a late submission than working with her.

Are you talking about that short, thin Deepika having long hair (Refer Time: 01:02) also?

Yes exactly, do you know her?

Yeah, I have been an dance class with her, she is a great show off and moreover she has ruined my rehearsal many a times.

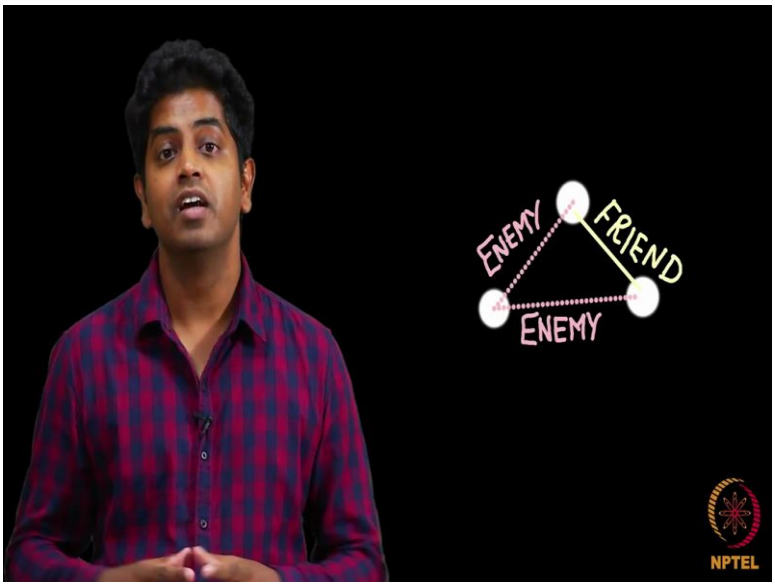
Oh, I am sorry for you, yeah, she is a great show off she thinks she is some great person.

Yes, actually I have to show her what I am, wait, I will come, I will cancel my plan of going to home and work with you in on this weekend, we will show great show.

Come let us go for a coffee.

Yeah, we are friends up here.

(Refer Slide Time: 01:37)



So, we watch the video what is your inference enemy's enemy is a friend comes as a counter intuition in the beginning, but the video made it very clear this 2 people initially they were they were not liking each other, but they got it topic to discuss and the topic is the third person who is not there and the third person is an enemy to both of them over we have seen many such situations in our life where 2 people only thing that they need is an excuse to talk about a third person who is not there and there is extreme bonding between these 2 people. We have seen this in many occasions we have done this too, right yeah. So, this is prevalent to an extent that is a lot more than what you can imagine this always happens.

(Refer Slide Time: 02:37)



I am going to motivate you by giving you very nice example as to why this happens lot of studies which says the following. When 2 people struggle together some kind of a bonding that happens between them I do not know whether it happens everywhere, but it is to happen in the place where I studied we used to have what is called the freedom run every august 15th, august 15th of every year we wish to have what is called the freedom run. We would run until this sweat and take a big ground and then come back. I was always wondering why do we run and why is it called a freedom run the point is very simple when you sweat together sweat together struggled together when you run together there is some kind of a bonding that happens with each other.

In fact, try this out if you want to increase your friendship with someone try working out with that person, try taking a long jog with that person you will see a difference in bonding rather the bonding will increase. So, maybe I should try that with my Rama and Krishna friends, one way to make them become friends is to run with them if we sweat together we become friends it seems say is research. Anyway I am going to give you some personal a tip here the course is not all about abstraction and exams and assignments I am going to give a life tip right now, I hope NPTEL does not edit this piece of video.

(Refer Slide Time: 04:17)



Here is a very nice tip especially for single people it is believed that if you work out sweat it out with a person of the opposite gender people can even fall in love what do I mean by this if you are seeing someone if your relationship status a sort of complicated. So, try taking a long jog with them or try playing badminton or workout go to a gym and try some aerobic; so, whatever.

In case you sweat together there is a high possibility that you people will fall in love with each other I am not saying this research says this try Googling for this information falling in love because of sweating together or something like that there is psychology to the article which has description of all the research in this direction anyway. So, let me know in case there is any success with this step alright.

(Refer Slide Time: 05:16)



Getting back to my original discussion there are 2 people struggling there is extreme bonding between them and 2 people beat friends or enemies just in case there talking to each other about their struggle imagine there is a bully in the office someone who troubles these 2 people and these 2 people actually do not like each other very much. But there is a common thing for them to talk which is this office person who bullies this people let us say the boss is always the bully.

So, boss bully is this 2 people and this 2 people they start talking about the boss and they get very close now do you do you do you see what is happening here, do you see the reasoning behind why an enemies enemy is a friend this is the reason. So, what is the inference?

(Refer Slide Time: 06:12)

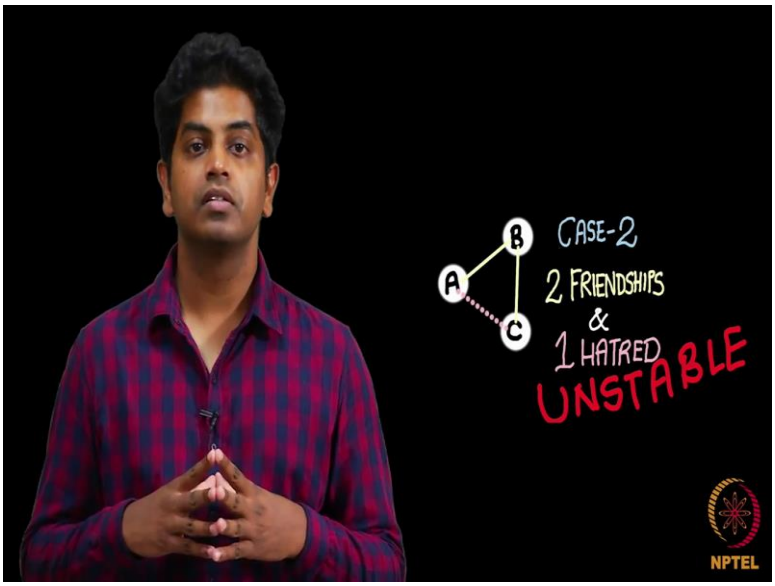


The inference is the following and enemies enemy is a friend take one message hatred, hatred, hatred 3 hatreds in a triangle is not stable when 2 people come together for some odd reasons and may they get to know that there is a third person who is a common enemy that is enough for these 2 people to become friends say psychology research. So, now, we are going to put this particular case of 3 hatreds under the unstable umbrella we are going to say 3 friendships means stable happy world as I told you 3 enemies means it is not stable there will be a friendship that will originate very soon. So, let us paraphrase what is happen?

(Refer Slide Time: 07:08)

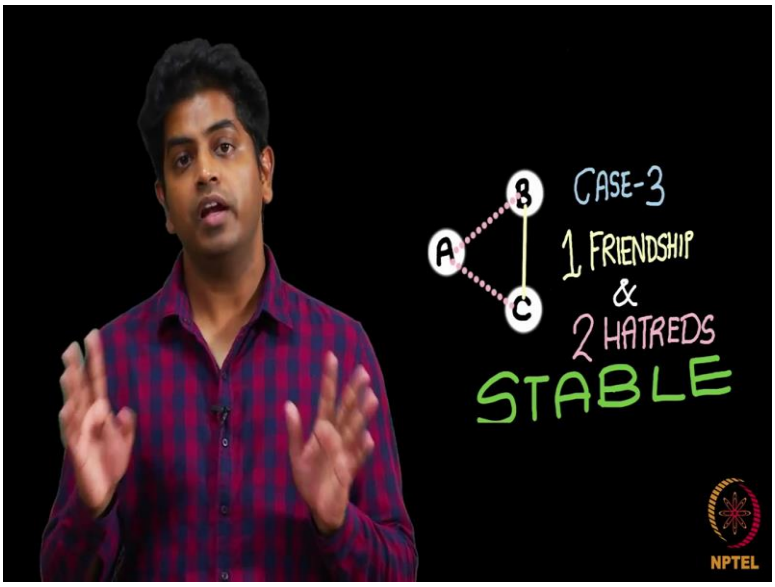


(Refer Slide Time: 07:14)



We had four cases just quickly go through this four cases case one 3 friendships stable no problem case 2-2 friendships one hatred Rama Krishna example unstable this will result in at least one friendship breaking this is an unstable state case.

(Refer Slide Time: 07:28)



3 2 hatred 1 friends stable nothing goes wrong here it will continue to be like this. In fact, the same hold theory that I told you just now holds good here you have a common enemy and that will increase your bonding when you talk about this common enemy with each other what is called Crebbing; Crebbing complaining whatever you call it.

(Refer Slide Time: 07:54)



Last case is 3 hatreds we discussed that us again that brings in some sort of an imbalance by imbalance I mean a it is not stable where in 2 people get together out of this 3 enemies mutual enemies when 2 people get together tells start talking about the third person and the bonding increases. So, 3 enemies; 3 hatreds unstable, so, you see these four cases 2 of them are stable 2 of them are unstable.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

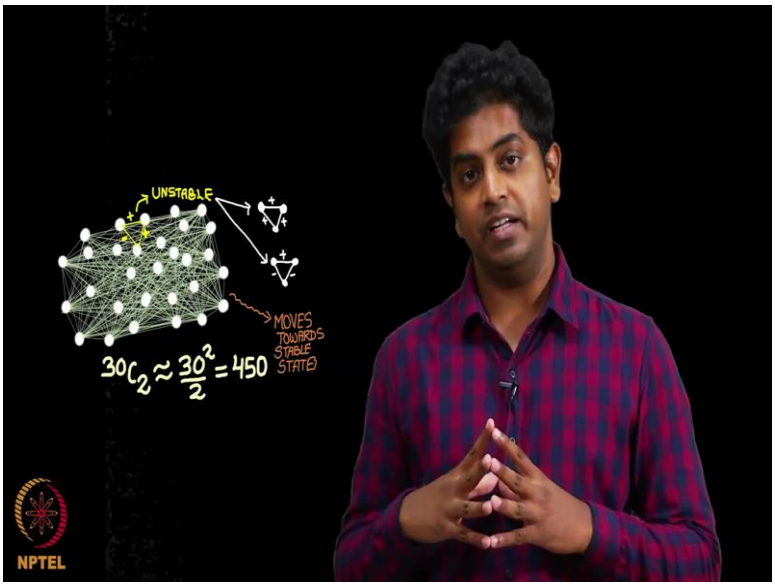
Lecture – 65
Homophily (Continued) & Positive and Negative Relationships
Characterizing the structure of balanced networks

(Refer Slide Time: 00:16)



Let us now shift gears and look at something slightly deeper you are now going to observe how in an organisation such positive and negative relationships can actually affect the organisation. Let me now ask the big question the main question in this chapter. So, he has the puzzles that impose the big question as a puzzle.

(Refer Slide Time: 00:34)



There are 30 people in an organisation 30 out of these 30 people since its only 30 people they know each other. So, is what is called a complete graph 30 people who know each other perfect? So, how many possible friendships are their here we have discussed this before it is roughly $30^2/2$ to be precise it is (30 * 29)/2, but throughout the discussion in this course a graph on n number of vertices we assume has roughly $n^2/ 2$ number of edges.

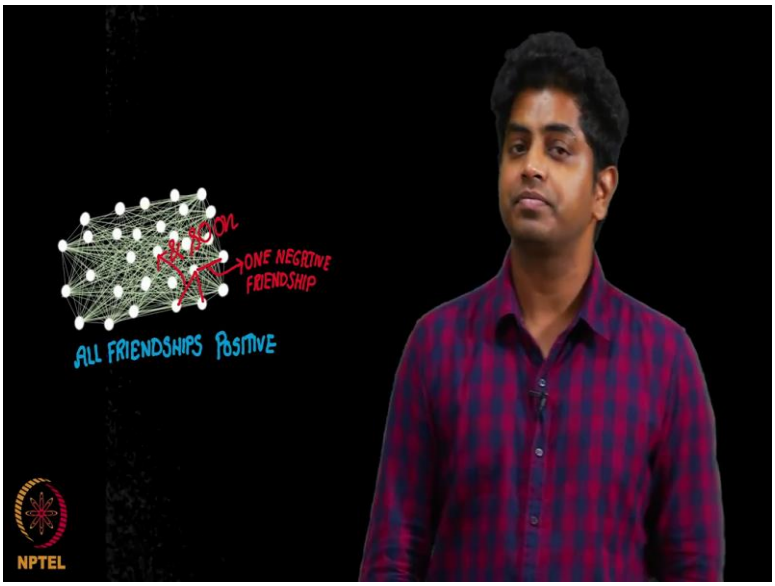
So, with 30 people you have $30^2/2$ number of friendships which is 450, right. So, far so good out of these four fifty friendships that you are seeing some of them can be positive some of them can be negative. So, observe carefully here is the graph with 30 nodes 30 people and roughly 450 friendships which can be positive and negative. We also know the four cases if there is a triangle here with a positive, positive, negative the it is unstable it will move towards positive, positive, positive or positive, negative, negative we have discuss this enough or you should remember by looking at this figure is that given a complete graph with 30 nodes and some random plus minus symbols on these edges in an organisation it moves towards stable state because every triangle here if you see may not always be stable.

(Refer Slide Time: 02:48)



So, what is the big question the big question is as it goes towards the stable state in an organisation how will the graph look like when there are no unstable triangles here I repeat out of 30 people given 450 friendships with some random plus minus symbols on the edges which is friendship and hatred what is what does one mean by this particular graph becoming stable, alright. So, in (Refer Time: 03:18) heard to analyze that that see one first of all one should get the question right once we get the question right we should make an attempt to answer to do you see the motivation behind this question the motivation is if a bunch of people get together there could be there could be friendships or hatred between them if there are unstable triangles it moves towards stable state what exactly happens eventually.

(Refer Slide Time: 03:48)



Let us consider a happy about assume 30 people how 450 friendships and all this friendships are positive never happens, but let us assume this happens all 30 people are friends with each other and their all positive now remember the code that I started this chapter with what is the code a bunch of apples you have and you just put one rotten apple there one rotten apple spoils the barrel right. So, this is just like this you have milk you put some if you drops of curd into it and the entire milk eventually turns into crud right given this kind of a happy world with all relationships being positive just try introducing one negative friendship what will that result in observe.

One negative friendship will result in a triode around it at least one triode right one triangle one negative friendship and that results in Rama Krishna kind of a phenomena and this one negative friendship ensures that one of their friendships becomes negative and this cascades whatever mean by this cascades one negative friendship results in another negative friendships so on and so forth. But wait a minute we will this result in all of them becoming negative my curd milk example is not actually true here why introducing one negative friendship we will result in negative friendships, but then if there is a triangle with three negative friendships that will result in positive friendship. So, it is not really true that introducing one negative friendship results in all negative friendships it may not really be true.

Now what else us what else will happened here I repeat the question given 30 people I am using 30 for example, reasons you can it an 100 200 whatever let us stick to 30. Given 30 people 450 friendships positive or negative there are some unstable triangles here it will reach stability slowly how does a organization with all stable triangles look like is the big question that we are going to answer now.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 66
Homophily (Continued) & Positive and Negative Relationships
Balance Theorem

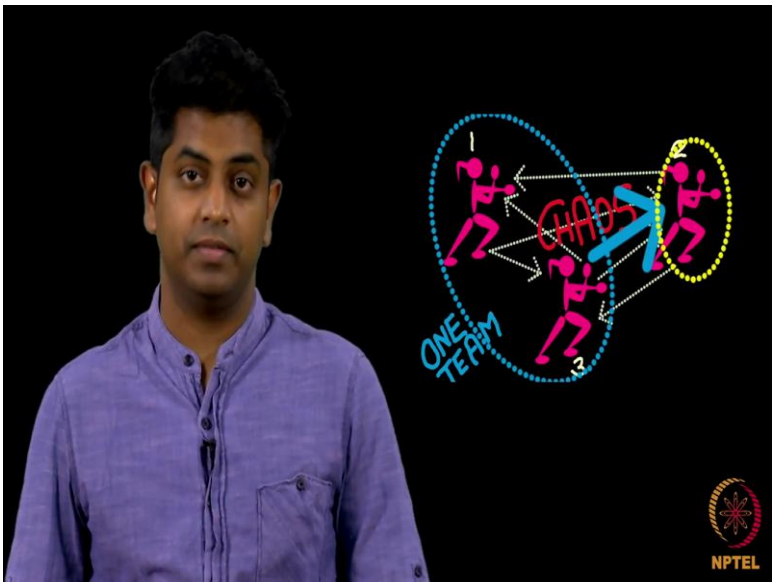
So, we are now going to show a very real world observed phenomena. In fact, I will tell you about it and it will be surprisingly true you will even say why do we have to talk more on this isn't it obvious, but then this is very elegant mathematical proof for it what is the question imagine this is a boxing ring.

(Refer Slide Time: 00:34)



And there are 2 people in the boxing ring and what you do is you introduce a third person when you introduce a third person what happens 2 people are trying to fight.This is a third person who is trying to fight and the third person does not join the first 2 people he is trying to fight the 2 people every person here is trying to fight the other 2 people we will this work what will happen eventually.

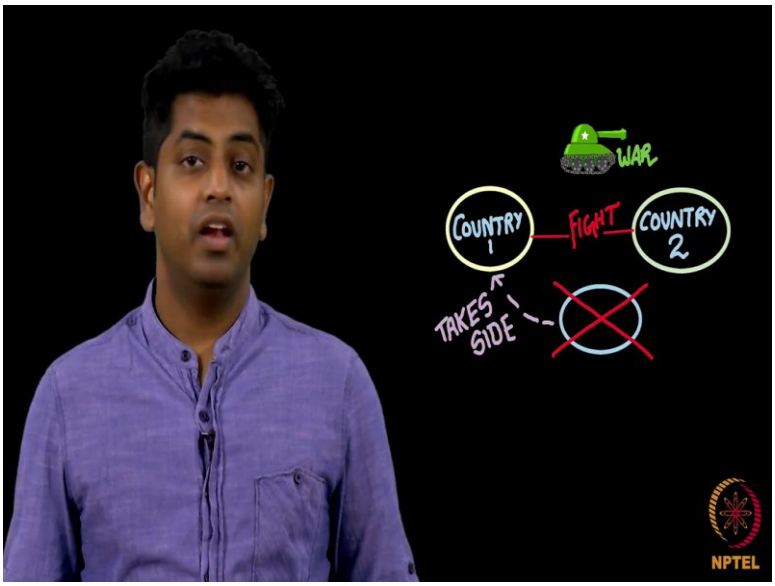
(Refer Slide Time: 01:09)



The first person is trying to hit the third person and second person second person is trying to hit the first person and the third person the third person is trying to hit the second person and the first person.

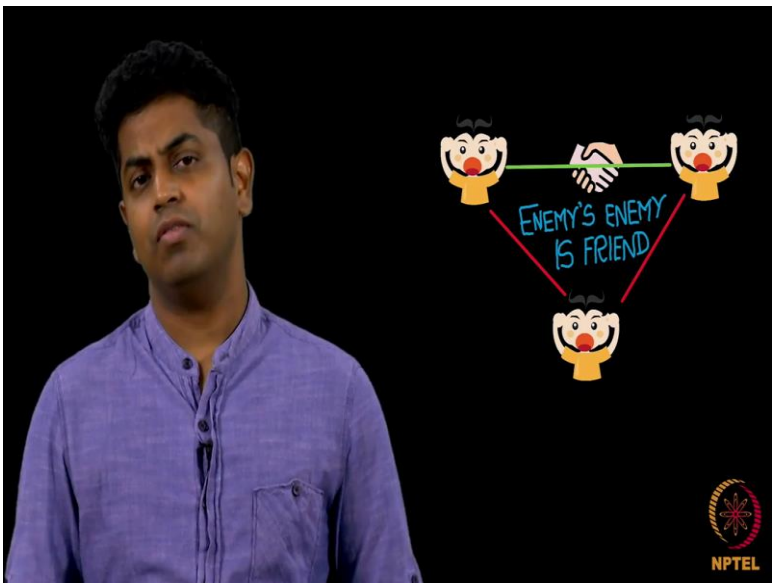
It will be a chaos it cannot be a proper fight unless 2 people get together into one team in that case these 2 people we will fight with this one person and they will be a when or a loss, but then when you have these 1 1 and 1 the boxing ring gets very chaotic does it sound very familiarity have not you seen in a group the fight is generally between 2 teams.

(Refer Slide Time: 01:52)



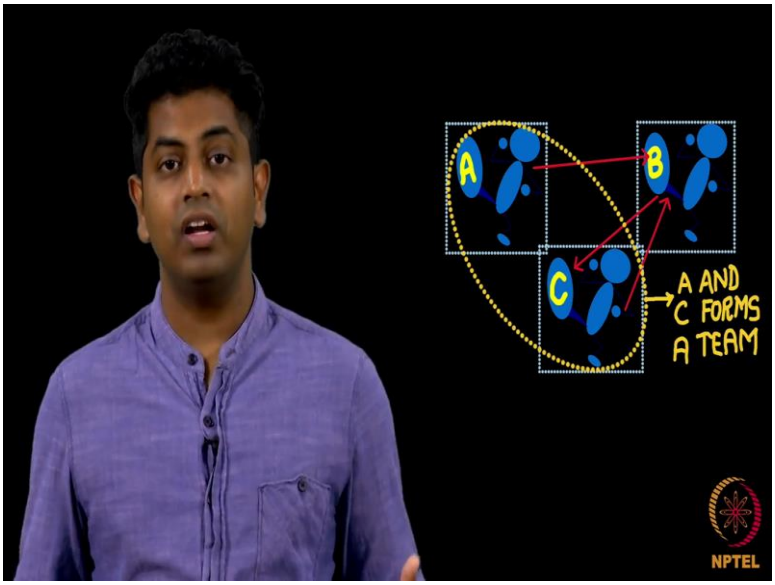
It is never between 3 teams why is that this sounds all the more familiar to you war when its waged it is never between 3 countries it is always between 2 countries may be is the third country, but the third country we will take some one side. In fact, it is observed in world war one and 2 that countries did take the side of some other country and the war was generally based between 2 groups in a bunch of people the war is always between 2 groups not between 3 groups.

(Refer Slide Time: 02:42)



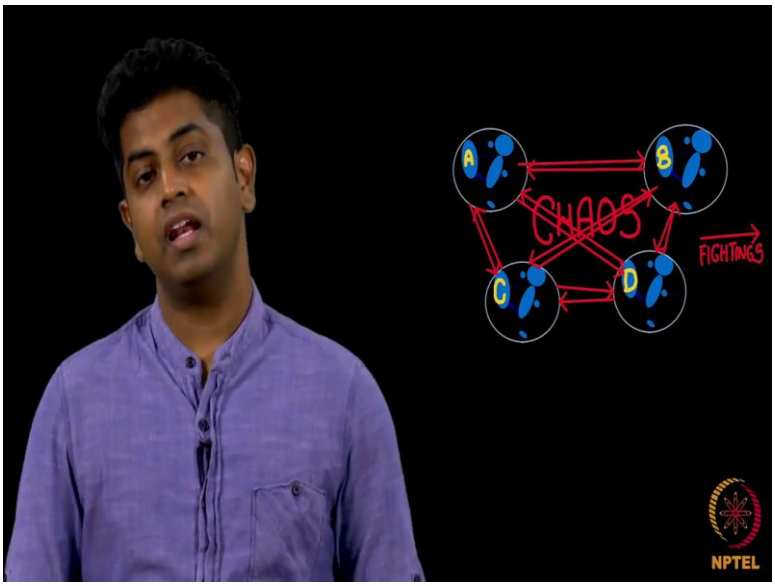
Now, can you connect the dots and tell me why exactly this happens that is because enemies enemy is a friend correct as simple is that how.

(Refer Slide Time: 02:50)



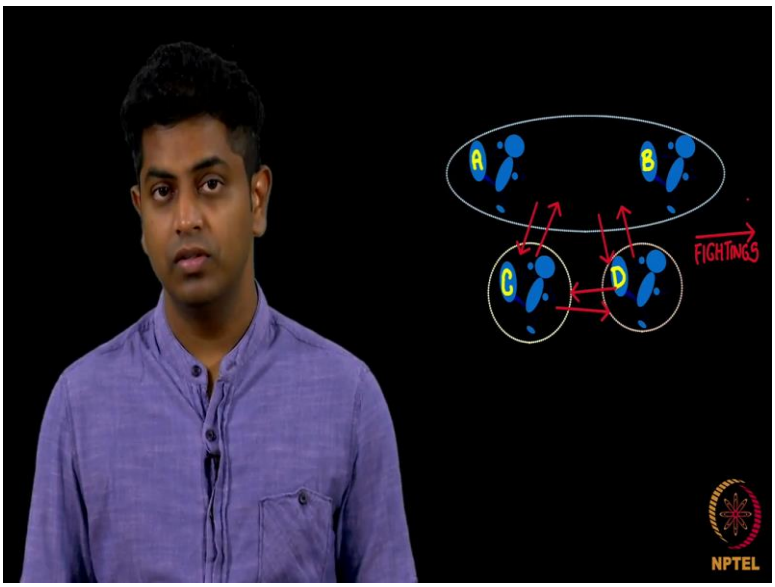
Assume you how a boxing ring and there is A B C fighting with each other when A realise us that he needs to hit B and B realise that he is should hit C right a should hit B A and C also should hit C there A C can get together and hit B, right so on and so forth. This is the some kind of a chaos here and then it can only be resolved by 2 people taking one side. So, when these people take a start getting taking in sides that enemies enemy becoming friends sort of a thing.

(Refer Slide Time: 03:29)



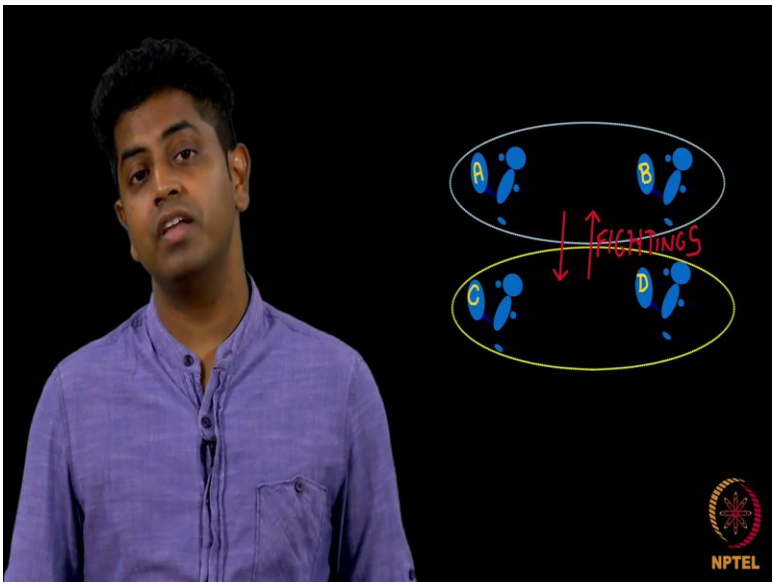
Then there becomes 2 teams imagine there are 4 people who are trying to box in a boxing ring now what happens again the same thing there are there is 1 2 A B C and D trying to box with each other and then A hits B, A hits C, A hits D and every single person is trying to hit the other 3 people chaos.

(Refer Slide Time: 03:53)



So, some prefer order emerges there and then this split into few teams maybe 3 teams from 4 teams A B C D is 4 teams and they realise this cannot happen this cannot continue and this aggregate themselves into 3 teams still there is chaos.

(Refer Slide Time: 04:07)

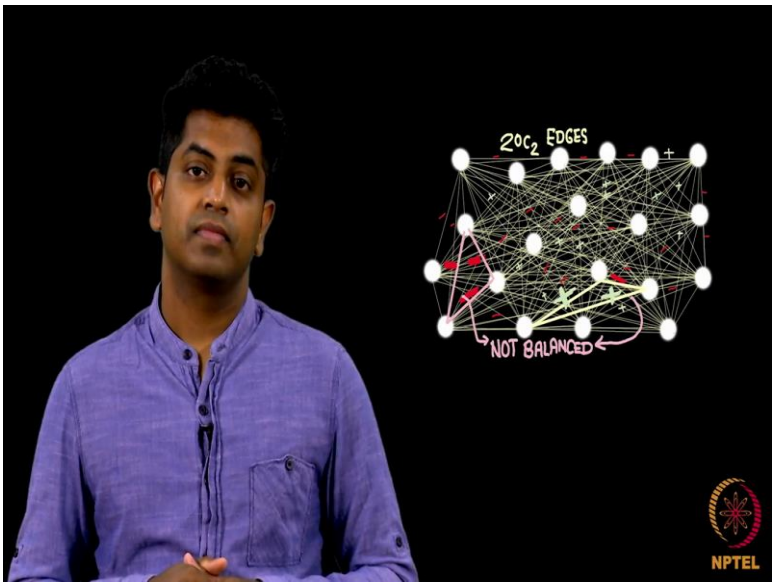


As I told you before these 3 teams again gets converted to 2 teams no matter how many people you put it is always 2 teams, why is this true let us try to look at it mathematically right now.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 67
Homophily (Continued) & Positive and Negative Relationships
Proof of Balance Theorem

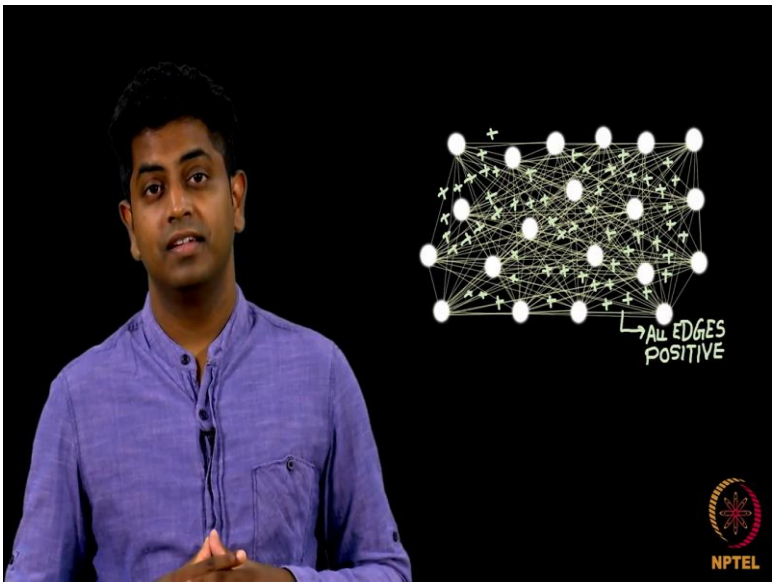
(Refer Slide Time: 00:21)



So, let us try to mathematically rigorously try to understand what is happening here. What I will do is firstly, I will develop the necessary machineries we will assume there is a complete graph, the graph on n nodes let us say some 20 nodes and you put edges between any 2 people, $^{20}C_2$ edges. Now there are some plus and there are some minus relationships, I put that 2 is this structurally balanced maybe not, you see there is a minus, minus, minus triangle there is also a plus, plus, plus minus triangle this is not structurally balanced. But what if it was structurally balanced let us see.

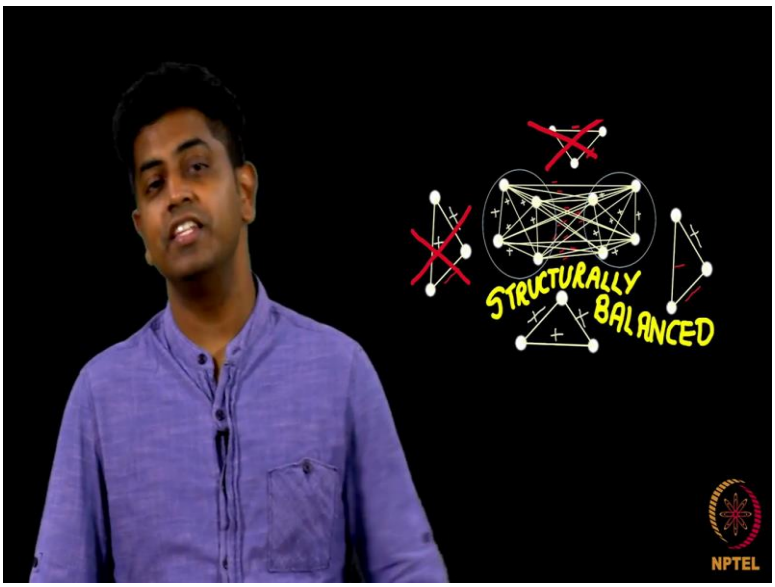
Let me now think of an example where is where a given graph is structurally balanced how we will look like this is an example of our graph that is structurally balanced see how it looks like.

(Refer Slide Time: 01:20)



Obviously, all of them are plus I know this is structurally balanced what is so great about this. Is there any graph with negative edges as well that is structurally balanced?

(Refer Slide Time: 01:44)

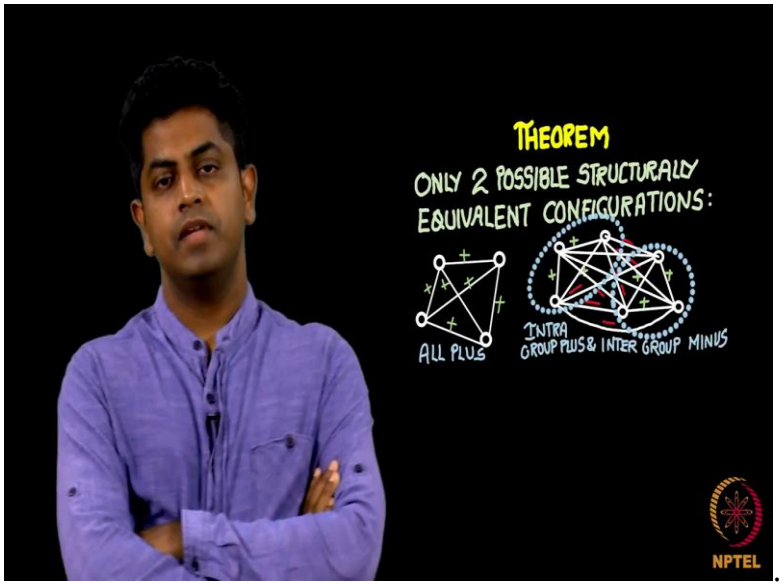


Let me think of finding one as I keep drawing. I converged to only structure like this do you see there are 2 clusters and edges inside these clusters are positive and edges across are negative. Basically there are 2 teams here 2 teams means what within the team the relationships is positive right you will see that in the figure relationships between 2 people inside the team is positive relationships between 2 people across the team is

negative. This is structurally balanced why, let me take a triangle here a triangle here is either positive, positive, positive as you observe or positive, negative, negative correct can you find a triangle here which has a positive, positive, negative plain impossible just observe can you give me a triangle here which is negative, negative, negative no impossible which means looks like this is structurally balanced.

So, when you have one team full of positivity structurally balanced when you have 2 teams where positivity is within negativity is across structurally balanced some basic observation that you did this structurally balanced. Now if I give you a graph g and college structurally balanced how will it look like we will it look like all positives or only 2 teams with positives inside negatives across is there are third type. So, we prove that there is no third type there is there only these 2 types the type one where there is only one team the type 2 where there are 2 teams.

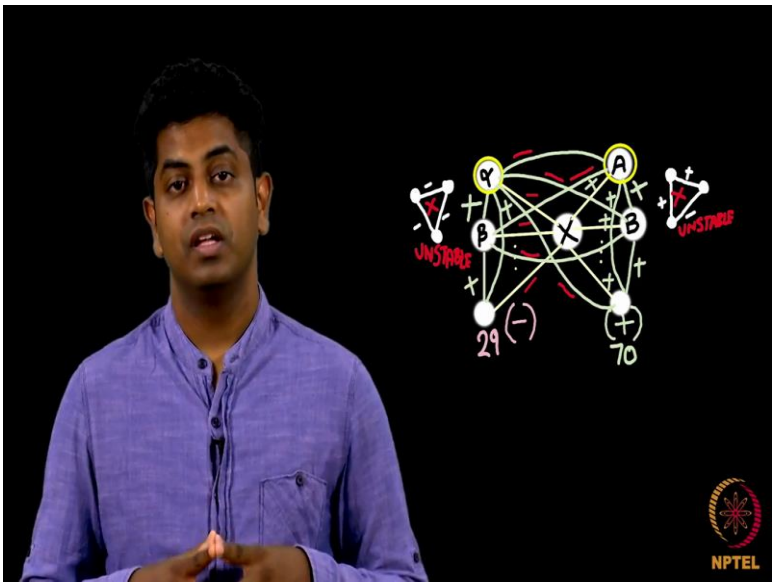
(Refer Slide Time: 03:53)



And period nothing else can happen. So, here is a theorem which states these are the only 2 possible structures when you say a given network is structurally balanced.

Let we let us try proving this now with good amount of rigger how do you go about it.

(Refer Slide Time: 04:17)



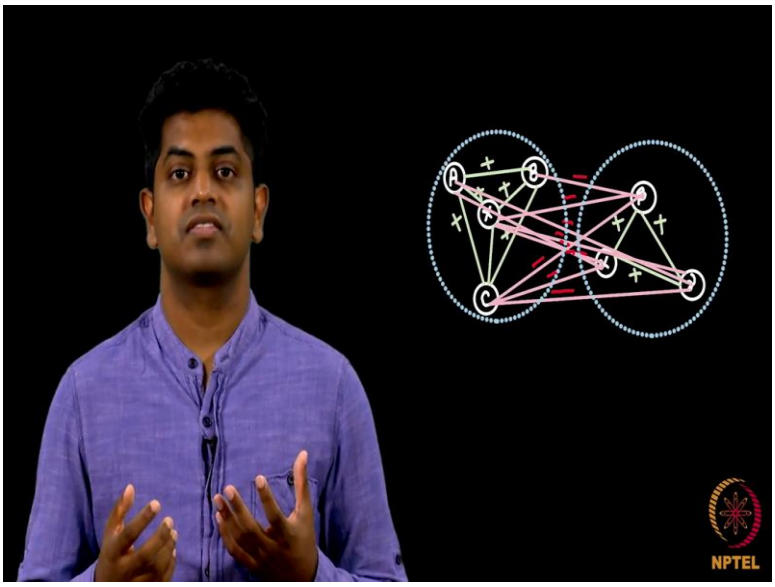
Let me take a given graph g with many nodes let us say 100 nodes and I pick one node from this network as you know in these graph with 100 nodes there are edges across any 2 nodes and there is a sign for every single edge plus or minus. As you can observe let me pick some node here call it let say x node x. x is a just sent to ninety-nine other nodes some of them positive some of them negative.

Let me say sorry roughly seventy of them is positive and twenty nine of them are negative seventy are positive twenty nine of them are negative now is you need obvious that this vertex x is friends with positive edges and the encompassing triangle on these edges as you can see x A B this triangle A B cannot be negative because I told you it is structurally balanced correct. Now look at this side x α, β x to α negative x to β negative now what can be the edge between alpha and beta it has to be positive because its structurally balanced correct think about it. This vertex x if it is a adjacent to 2 people positively the relationship between these 2 people as I told you in the example A and B should be positive with the vertex x is adjacent to α and β and x α is negative x beta is negative and what should be between alpha and beta enemy is enemy right it should be positive.

So, I observe that when you pick an element x and look at all its friends which are positive and all its friends which are which are negative you observe that amongst the positive friends of x relationships are all positive amongst the negative friends of x

relationships are all positive, but if you pick a friend A of x and a friend α of x. x A is positive, x α is negative then what is the relationship between A and α 1 positive one negative which means A and α should be negative that is straight forward. So, what do I gather all the friends that x is adjacent with they are friends with each other with positive friendship all the people that x is adjacent to with negative friendship negative relationship?

(Refer Slide Time: 07:22)



They are all friends with each other which means there are 2 clusters here what are those 2 clusters the clusters x and all his friends there all positive within and all the negative friends of x there all positive within, but negative across as simple as that that closes the theorem. So, the theorem just states that in case you have a structurally balanced network it better we type one all positive friendships within there is only one team type 2 there are precisely 2 teams where friendships within them is positive friendships occurs is negative.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 68
Homophily (Continued) & Positive and Negative Relationships
Introduction to Positive and Negative Edges

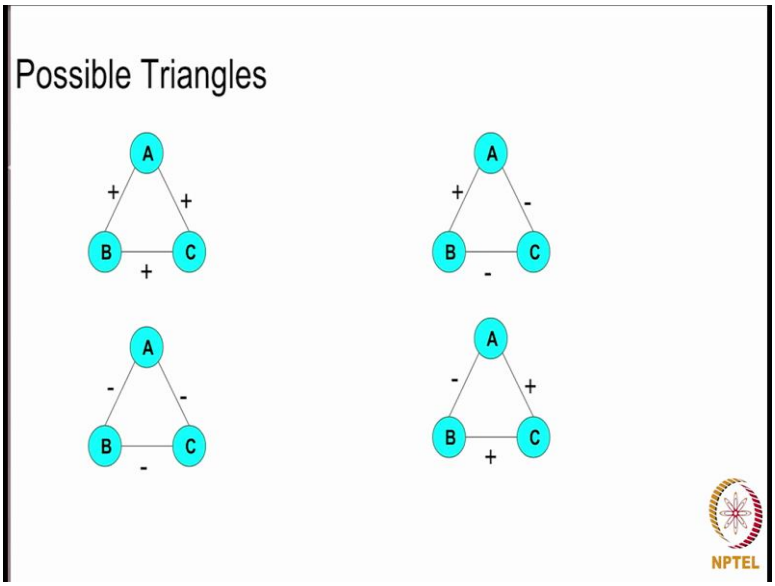
(Refer Slide Time: 00:05)



Hi everyone, in this video sequence we are going to implement signed networks where the edges have either positive or negative weights attached to them. And if the nodes represent individuals then the positive way the positive edges indicate friendships amongst these individuals and the negative edges indicate animosity amongst these individuals. In such networks there exist some sort of triangular structures which when analysed lead to interesting observations, in this video sequence we are going to analyse those structures which are basically different combinations of positive and negative weights across the edges of the triangles present in the network.

So, I assume you have been already introduced to these networks in the previous videos, I will just be briefly discussing the possible structures that are prevalent in the networks and then we will go ahead to implementation.

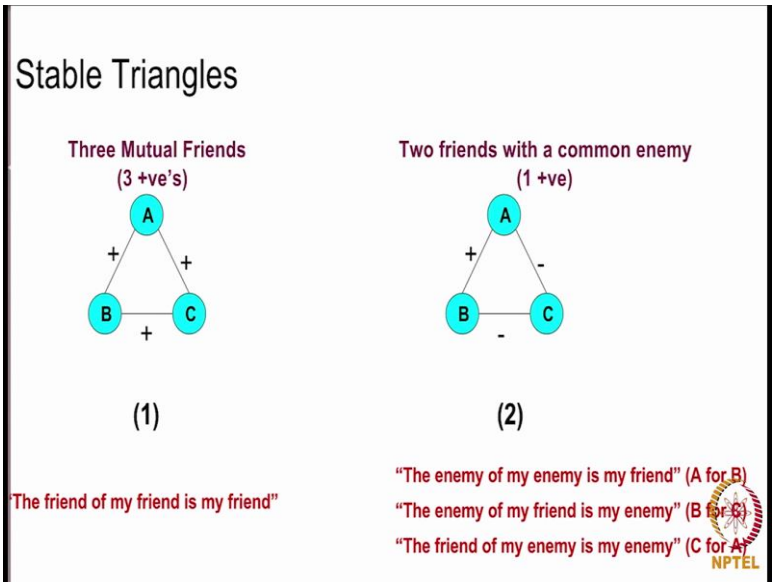
(Refer Slide Time: 01:08)



These are the possible combinations of positive and a negative way across the triangles present in the networks. Now some of these triangles are called stable and some of these are called unstable. Now what do we mean by unstable triangles or unstable combinations? These are the combinations that do not stay in the network for a long time simply because they are inconsistent with respect to the relationships among amongst the nodes and they tend to move towards a stable state.

Now, we are going to see which of these are stable combinations or stable triangles and which are unstable triangles. Please note that they could be different permutations of these signs across the triangles you anything to note here is that there are three positive signs here and there is one positive sign here there are 2 positive signs here and there is no positive sign here.

(Refer Slide Time: 02:18)

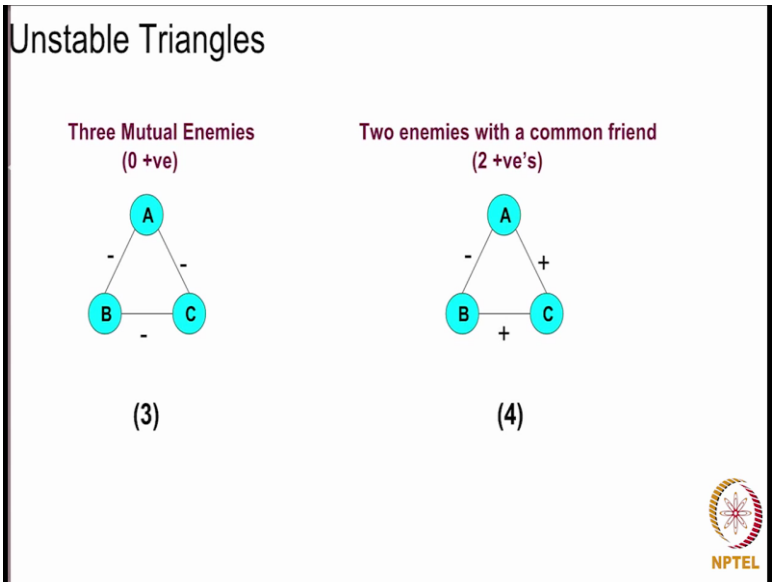


So, this is these are the different combinations of course, the different permutations can exist. So, let us look at these stable triangles first. So, a triangle having all positive edges is one of the stable triangles. So, there are three mutual friends they cannot be a better situation done this, so, every bodies every body's friend.

This kind of structures seems to follow this popular social belief that the friend of my friend is my friend the next social sorry next stable triangle is this one where you have one positive edge and 2 negative edges. So, this basically shows that there are 2 friends who have who have one common enemy, right. So, the triangle will have one positive edge, but it indicates that 2 friends have one common enemy this is also one of this stable triangles this triangle seems to follow a number of interesting social believes. For example, if you look at node A with respect to be the node a seems to follow the enemy of my enemy is my friend when it comes to node B with respect to C its seems to follow this belief that the enemy of my friend is my enemy as you can see here and if when it comes to node C with respect to A it seems to follow this belief that the friend of my enemy is enemy.

So, it is pretty interesting to see that all the nodes seem to be satisfied with the kinds of relationships they have they have with their neighbours in the triangle. So, that is how it another stable triangle. So, you can remember that triangle which has either one positive edge or three positive edges are stable triangles let us now check the unstable triangles.

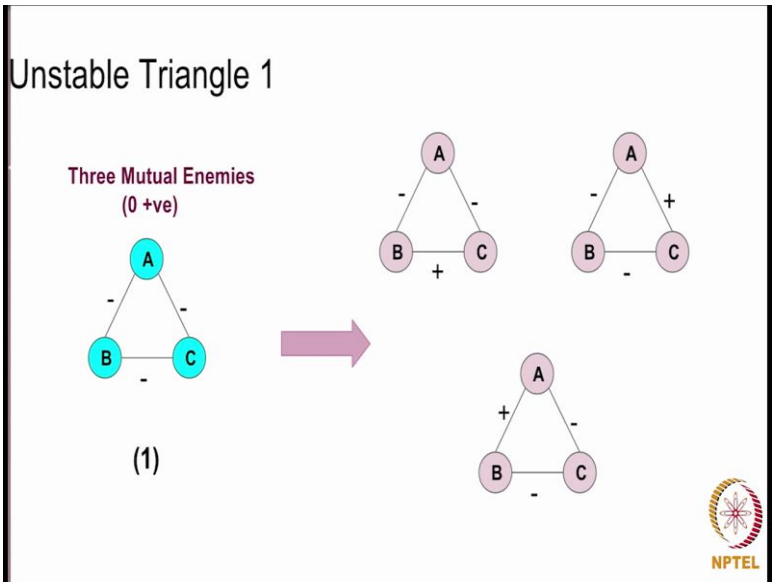
(Refer Slide Time: 04:12)



So, the first one as you can see is the triangle which has all negative edges. So, this is one of the unstable triangles because all the nodes are enemies of each other and this; this kind of situation cannot stay for a long time given the belief that enemy of my enemy is my friend. So, this seems to actually play a role here which leads to some of the nodes befriending other nodes and hence changing the configuration and hence moving towards stable state.

So, this is the one where there is 0 positive edge let us look at the second unstable triangle now here as you can see there are 2 positive edges and there is one negative edge. So, this basically means that there are 2 enemies which is A and B here there are 2 enemies and they have one common friend. So, this is again another unstable situation which does not tend to stay for a long time because 2 enemies would not like to have one common friend. So, this again leads to another state which is table hence changing changes the configuration of the network. So, you can remember that the; and that a triangle which has either 0 positive edge or 2 positive edges is an unstable triangle right. So, you can remember it that way now what we are going to do is we are going to look at each of these unstable triangles and we are going to see which stable states can these triangles move towards.

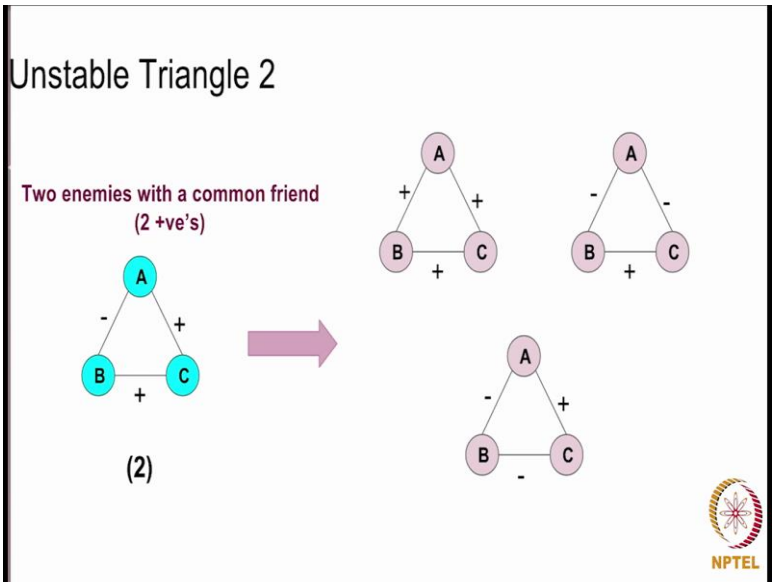
(Refer Slide Time: 06:01)



So, we are taking the first unstable triangle here where there are three mutual enemies as I said the beliefs that enemy of my enemy's enemy is my friend seems to play a role here in the network. So, when there are three negative edges which means that some of these nodes ten to make a make friends make friendship friendships with each other. So, it may happen that B and C become friends of each other because there enemy there because they will then have A common enemy which is A or it may happened that these three negative signs leads to a and C becoming positive because they will have B as a common enemy or it may happen that A to B becomes positive because they will have a common enemy C.

So, whenever we have a triangle which has all negative edges that are 0 positive edges it can move to any one of these their stable states where there is one positive edge each. So, let us talk about the next unstable triangle which has one negative edge and 2 positive edges.

(Refer Slide Time: 07:18)



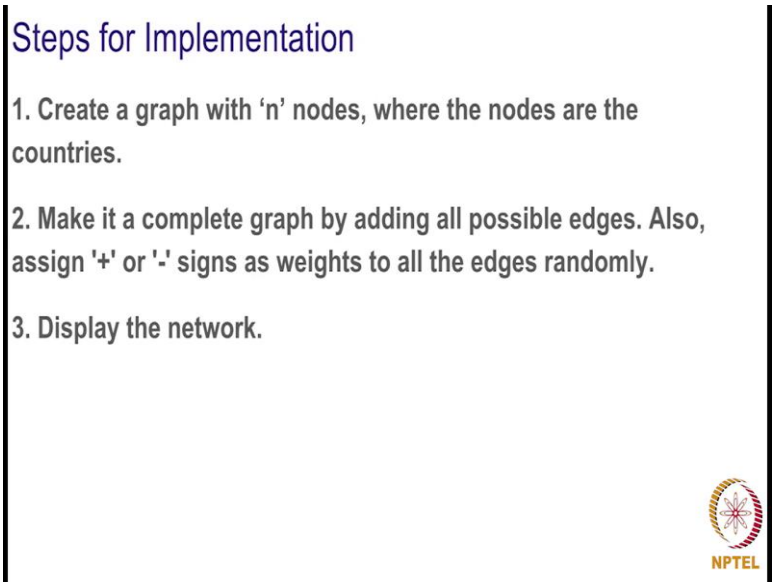
So, here there are 2 enemies who have one common friend which is see now there are they can we they can we three things which can happen in this scenario first thing is since C is a common friend to A and B C might help A and B also to become a friend right. So, A and B, A to B, h can be come positive like this. So, this is one thing that can happen second thing that can happened here is A has a friend C who is friends with an enemy of A. So, that may lead to animosity between A and C. So, this is what can happen in this case third thing that can happen is since B is friends with C who is friends with an enemy of B which is A.

So, that can lead to animosity between B and C as well. So, it can lead to this kind of situation. So, as you can see on the right hand side the three configurations one of them has three positive edges which is a; which is a stable one that that you have already know and 2 of the configurations have one positive edge which is again a stable relation. So, all these three are stable relationships. So, the relationship on the on the left hand side leads to these three configurations in the implementation we are going to start with a few configurations and we will see how they move towards stable states in the next video we are going to see the steps that you are going to follow for the implementation.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Homophily (Continued) & Positive and Negative Relationships
Lecture – 69
Outline of implementation

(Refer Slide Time: 00:05)



Before we start the implementation, we are going to look at the steps that we are going to execute during with the implementation. So, that its easier for you to follow and understand the subsequent videos.

So, let us look at the bigger picture first. Our aim is to have a network of countries where the friendship or animosity details amongst these countries is stored in the network. Given this scenario a number of unstable relationships amongst the countries emerged which tend to move towards a stable state.

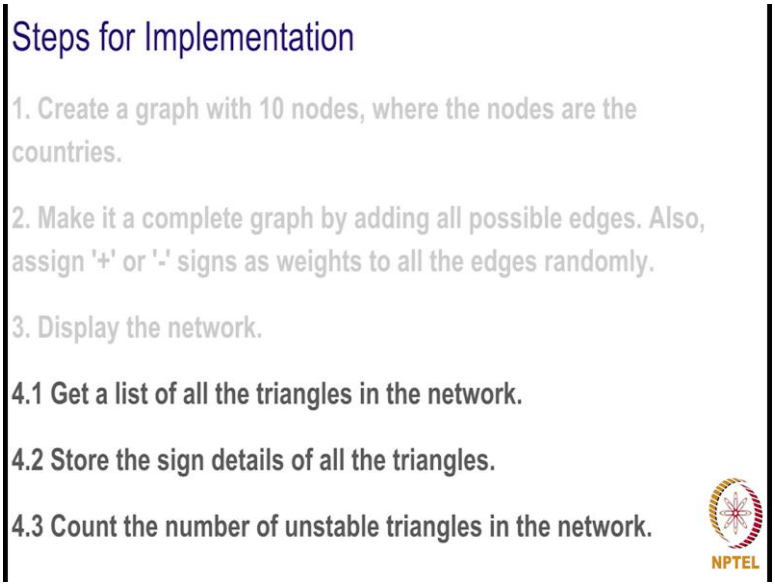
So, over time what happens the network starts from an unstable state and it gradually moves towards a stable state. Now, what do we mean by a stable network? Its a network where there is no unstable relationship. So although, all the triangular relationships will be stable and that network does not change after that. So, our first aim is to observe that how the network evolves from unstable state to a stable state. Now second thing that can be observed in this scenario is that. Once the network becomes stable the nodes in the

network can be divided into two groups such that: the nodes in the first group are all friends to each other, the nodes in the second group are all friends to each other. However, the nodes in the first group are enemies to the nodes in the second group

So, that is an interesting pattern that emerges. And the second aim of our implementation will be to observe this division of nodes into two groups. And we will see how the relationships existed amongst the countries initially, and how the relationships exist amongst the countries later after this evolution.

So, to start with we will create a graph with n nodes where the nodes are countries. We are going to make it a complete graph by adding n choose two edges and we are also going to randomly assign a weight either positive or negative to all these edges. And after that we will display the network. So, this is the starting step. Once the network is creative our aim is to analyze the unstable triangles in the network.

(Refer Slide Time: 02:25)

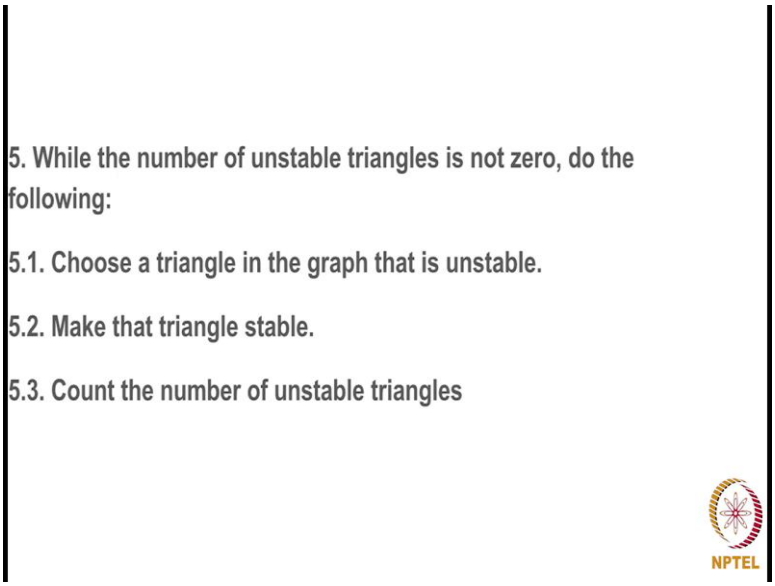


So, what we are going to do is: we will create a list of all the triangles in the network. So, if there are n nodes, they are going to be n choose three possible triangles in the network. Since we have to check whether these triangles are stable or unstable, we will store the sign details of all these triangles. Maybe, we will create a list where we will store the sign details of each and every triangle.

Now, you might remember from the previous video that if the triangle has 3 or 1 positive edges then it is stable. And if the triangle has 0 or 2 positive edges then that triangle is unstable right. So, we are going to use this detail to check whether a given triangle is unstable or not. And we will finally get the total number of unstable triangles in the network

Our aim now is to move the network from an unstable state to a stable state. Basically, we have to see what happens when there are unstable triangles in the network. Since we know that they tend to move towards a stable state so that is what we are going to implement in our video.

(Refer Slide Time: 03:34)



So, what we are going to do is we are going to choose a triangle which is unstable and will move this triangle to a stable state. Now I will show it during the implementation.

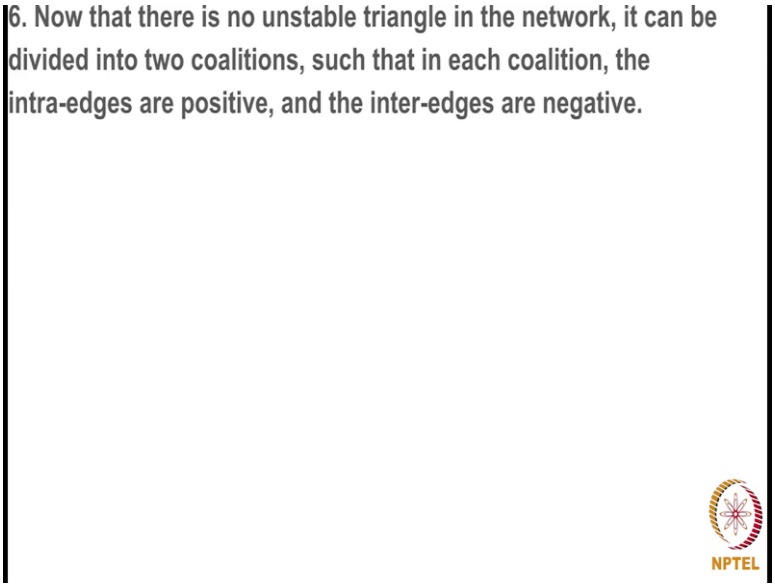
So, we will make that triangle stable and after the triangle becomes stable a number of relationships change and due to due to that the adjacent relationships also get affected. So, the adjacent triangles may become stable or they can become unstable as well. So, this changes the total number of unstable triangles it can increase it can decrease. So, we will actually see during the implementation how it fluctuates

After this moving of one triangle from unstable state to stable state the total number of unstable triangles will change, so we are going to keep a count of that. So, we are going

to do this until the total number of unstable triangles becomes 0. So, we are going to repeat the steps 1 2 and 3 here until the whole network becomes stable.

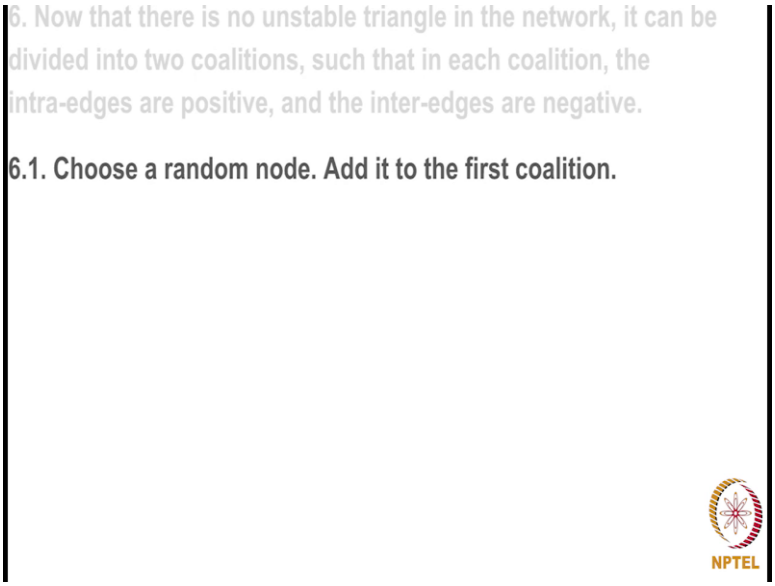
So, that is the whole idea of making the network move from unstable to stable state.

(Refer Slide Time: 04:49)



Now, as I told you once the network becomes stable it can be divided into two groups right, where the nodes in first group are all friends and the nodes in second group are all friends, and they all and the nodes in first group are enemies to the nodes in second group. So, how can we create that network? Now we have a stable network and we have to divide it into two groups, how do we do that? There can be various ways that we can use to divide the network; what we are going to follow here is that.

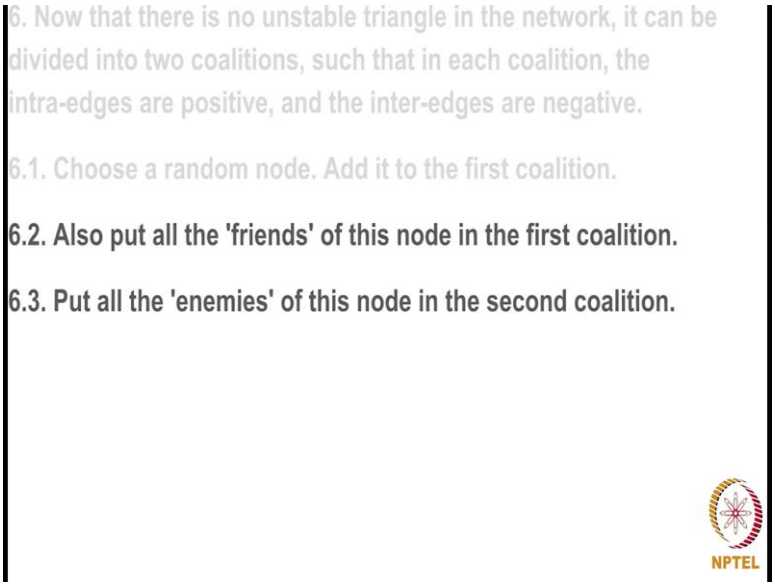
(Refer Slide Time: 05:24)



We are going to choose a random node and we will add it to first group, after that we will look at the neighbors of this node some of these neighbors will be positive some of these neighbors will be negative. As in the some of these neighbors will be friends, some of these neighbors will be enemies.

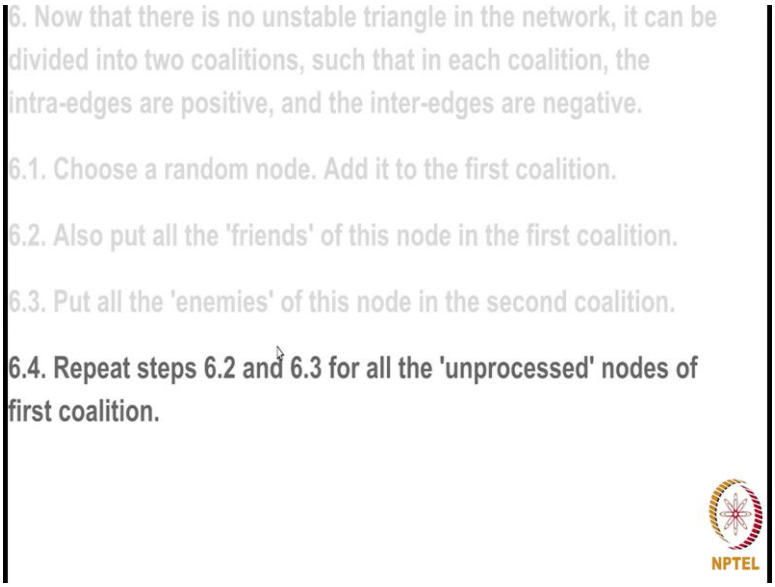
So, we are going to take the friends of this node and we will add the all of them to the first group. And the enemies of this node we are going to add to the second group. After that we will take the friends of this node which we added to the first group, we will take them one by one and we will repeat the same process for them. That is we will we will look at their neighbors, we will add the friends to the first group, we will add its enemies to the second group.

(Refer Slide Time: 06:17)



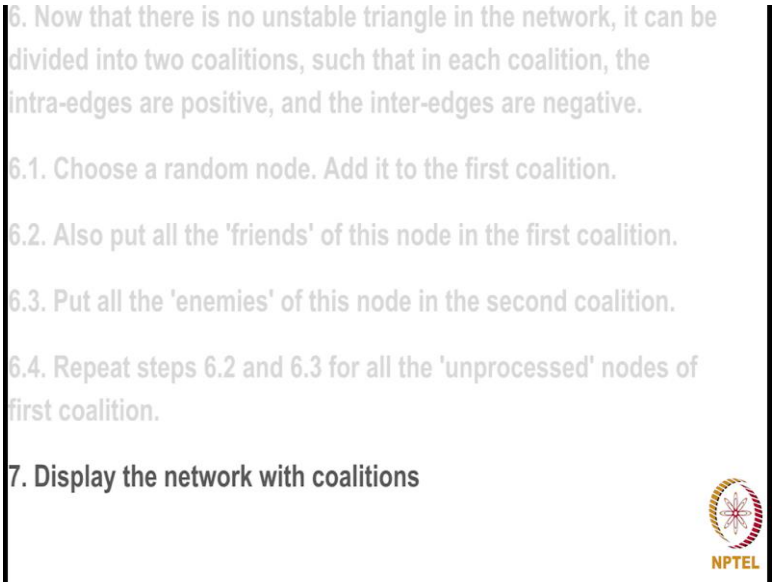
So, we will keep taking all the nodes from the first group which we have added, if there if they have not been processed they are going to repeat this process. So, we will put all the friends in the first group, we will put all the enemies in the second group.

(Refer Slide Time: 06:25)



And we will keep repeating this process until we have processed all the nodes in the first group. So, at the end of this process we will be having two groups of nodes as stated.

(Refer Slide Time: 06:37)



And then we will display the network with the two groups, and we will observe the kinds of relationships that existed amongst the countries initially, and the kinds of relationships that existed amongst the countries after all this evolution.

So, these were the steps that we are going to follow let us now start the implementation.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Homophily (Continued) & Positive and Negative Relationships
Lecture - 70
Creating graph, displaying it and counting unstable triangles

(Refer Slide Time: 00:05)

```
import networkx as nx
import matplotlib.pyplot as plt
import random
import itertools

def get_signs_of_tris(tris_list, G):
    #tris_list = [[1,2,3],[4,5,3]]
    #all_signs = [['+', '-', '+'], []]
    all_signs = []
    for i in range(len(tris_list)):
        temp = []
        temp.append(G[tris_list[i][0]][tris_list[i][1]]['sign'])
        temp.append(G[tris_list[i][1]][tris_list[i][2]]['sign'])
        temp.append(G[tris_list[i][2]][tris_list[i][0]]['sign'])
        all_signs.append(temp)
    return all_signs

def count_unstable(all_signs):
    stable = 0
    unstable = 0
    for i in range(len(all_signs))
```

Let us get started with the implementation. So, for your convenience I have adding all the steps so here so that we will pick these steps one by one and we will implement them ok. So, initially let me import the packages that we are going to need, since we are going to work with networkx.

Let me import networkx. We are also going to display the network. So, we might need matplotlib so I will import that. We will be randomly choosing the triangles out of all possible triangles. So, we will need the random package. And to choose all possible triangles we might need to use combinations function from itertools package we used it in previous video as well. So, I will import itertools. So, these were the packages.

Now let us look at the first step we have to create a graph with n nodes, where the nodes are the countries. So, let me create graph this is an empty graph as of now. So, let me add the nodes to this graph let us initialize this n to 5 and we can change it later. So, we are

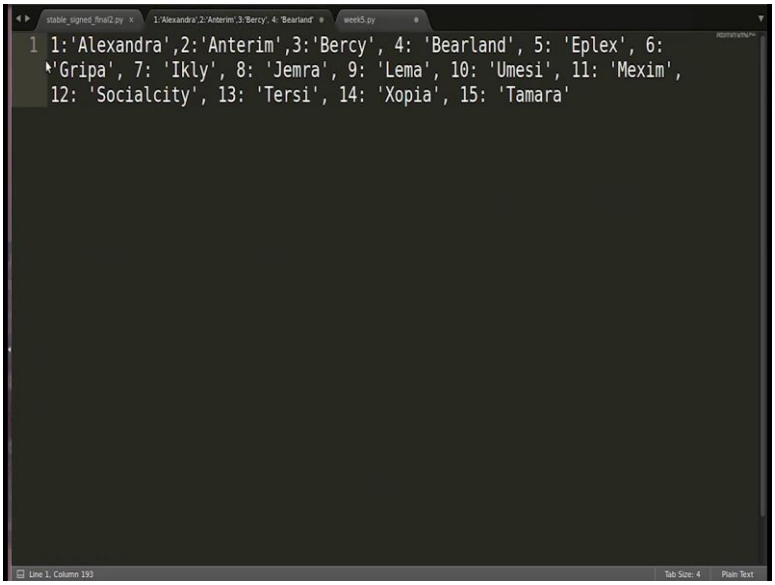
just going to start with a simple network, where there are 5 countries here increase that later.

So, n is equal to 5, we are going to add 5 nodes to it. So, I will write we have to add 5 nodes. So, let me pass a list of 5 nodes over here as a parameter. So, I will write i for i in range; the nodes should be numbered from 1 to n. So, I can write i to n + 1 because range if it starts from 1 to n + 1 it goes from 1 to n. So, this is what we are going to pass here.

Now we have added the nodes to this graph we have to assign the names to these nodes, the country names to these nodes. So, let me use dictionary; we are we are going to use dictionary for this because we have this function a networkx. That is nx.relabel_nodes; what this function does is? It labels the nodes, because as of now the nodes are labeled from 1 to n right that is 1 to 5 in this case. We have to assign the country name as a label to all these nodes. So, they are going to make use of this function relabel nodes from networkx.

The parameters are two: the first one is a dictionary let me name this dictionary is mapping that is what will pass. This dictionary will contain the labels the new labels of the nodes. So, the key will be the old label and the value will be the new label right. The second parameter is the graph itself. So, we are going to make use of this function, for this we first need a dictionary which has the new labels.

(Refer Slide Time: 03:33)



I already have this here; I have some random names of the countries. So, I am going to use these fictitious names for the countries. So, this is our mapping dictionary which we will pass to this graph and the nodes will be relabeled by this function.

So, the first step is over, we have created a graph with n nodes where the nodes are the countries. Let us go to the second step. The second step is we have to add all possible edges to this graph and we also have to assign wait either positive or negative to these edges. So, let us create a list with two weights: positive and negative. So, what we are going to do is: we will randomly choose one of these weights and we will assign to the edges.

Since we have to create a complete graph, we have to add all possible edges. So, how can we do that? I can start the loop here. So, I can write i in G.nodes for j in G.nodes, if i is not equal to j then we will add this edge. So, we will write G.add_edge we have to add the edge between i and j, we also have to assign the weight.

Let us call the attribute sign; sign is equal to. So, we have to randomly choose the sign out of these. So, the function that we can use is random.choice. What this function does is: it chooses, so the parameter is a list that is the signs here out of this the values of this list it randomly chooses one value. So, out of plus and minus it will randomly choose either plus or minus and we will assign to this edge.

(Refer Slide Time: 05:22)

```
 8 G = nx.Graph()
 9 n = 5
10 G.add_nodes_from([i for i in range(1, n+1)])
11 mapping = {1:'Alexandra',2:'Anterim',3:'Bercy', 4: 'Bearland', 5: 'Eple
12 G = nx.relabel_nodes(G, mapping)
13
14 # 2. Make it a complete graph by adding all possible edges. Also, assig
15 signs = ['+', '-']
16 for i in G.nodes():
17     for j in G.nodes():
18         if i != j:
19             G.add_edge(i, j, sign = random.choice(signs))
20
21 # 3. Display the network.
22 edge_labels = nx.get_edge_attributes(G, 'sign')
23 pos = nx.circular_layout(G)
24 nx.draw(G, pos, node_size = 5000)
25 nx.draw_networkx_edge_labels(G, pos, edge_labels = edge_labels, font_si
26 plt.show()
27
28
```

So, now we are done with the edges. Next step is to display the network; the command to display the network is nx.draw(G). However, we want to display number of other things on the network. So, we are going to use some extra features of drawing functions from networkx. For example, we can use a layout let me use circular layout so that the nodes are nicely visible. So, I will write nx.circular_layout.

Now, by default the edge labels are not displayed. So, we will have to use an extra function nx.draw networkx edge labels. So, we are going to use this function and I will tell you the parameters to this. Another thing is that when we use this function the labels will be displayed. However, the labels will be like sign is equal to plus sign is equal to minus. We do not want that to be displayed like this, you just want plus we just want minus, we just want the signs not sign is equal to plus. So, for that we will have to use one more extra function. We will write edge labels is equal to nx.get_edge_attributes from G we must take the attributes and the attribute whose values we have to take is sign. Now these edge labels we are going to pass here the first parameter is G itself, the second parameter is the layout that we are using and then we will pass the edge labels.

Edge labels is equal to edge labels which means the values that we are getting out of this function those values only should be displayed here. If you want, you can change the font size you can change the font color.

(Refer Slide Time: 07:25)

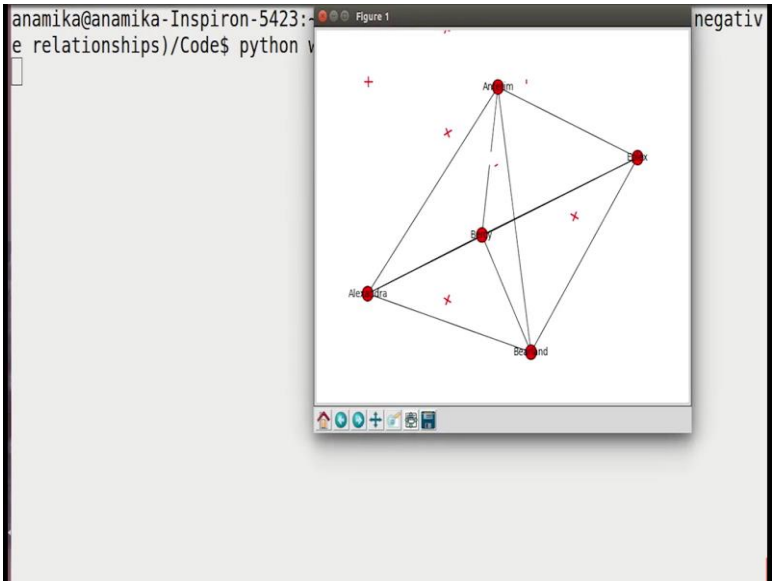


So, let us use this font size is equal to say 20, you can also change the font color let us do that is well let us make it red. So, these are just some notification parameters you can use them if you want. So, just to show you how they work and going to show then in this video.

I think this should be sufficient. So, to display the network we have to use plt.show. So, to sum up we first added the nodes here, and we really build; oh, I think first we pass the graph and then we pass the dictionary ok. So, first we added the nodes, we relabeled the nodes, then we added the edges, and we assigned weight to these edges, and now we are just going to display this.

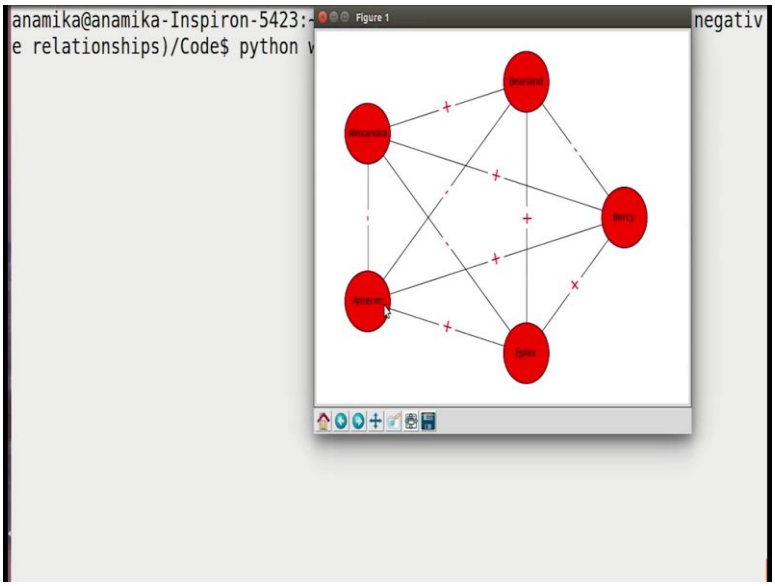
So, let us try executing this, ok.

(Refer Slide Time: 08:18)



So, we go back here and let us execute this ok. So, this is not coming in a circular layout, let us go back here ok. We have not passed this pos here while we are drawing, so we can pass this. Another thing that we can do is we can change the node; the node size is well since you saw that the country names are not visible nicely in the nodes. So, we can change the size. Let me do that node size you can specify any size I am just giving this, ok.

(Refer Slide Time: 08:55)



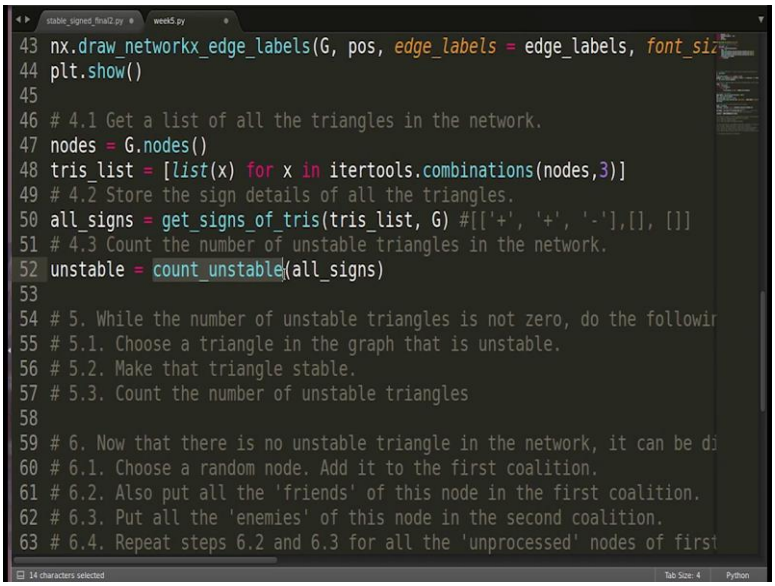
So, let us go back here and try to run it: oh, now this looks better. So, as you can see here. So, these are the 5 countries just for simplicity we have kept a smaller number of countries and will increase them later. So, the edge labels indicate whether they are friends or enemies to each other.

Now, you can see there are some unstable triangles here. For example, Alexandra, Bear land and Bercy you can see that they are all enemies to each other. So, we see that this kind of these kind of relationships do not tend to stay for a long time in the network, they tend to go towards the stable state. That is what we will show in our implementation. So, that is one unstable triangle. Another unstable triangle let us see Alexandra, Anterim, and Bearland you see they are all again enemies to each other that is another unstable triangle.

If you look at Anterim, Eplex and Bercy they are all friends to each other. So, this kind of relationship should stay as it is. So, this is the first three steps completed: we have the network with us, now we have to make it evolve from one state to the other.

Let us close it and go back to our program.

(Refer Slide Time: 10:15)



So, let us go to the fourth step now.

Now, we have the network with us and we have to observe the triangles. So, let us execute this 4.1 that is get a list of all the triangles in the network. How do we do that? To get all the triangles all the possible triangles we will have to choose three nodes from the list of nodes right, all possible combinations of three nodes. So, for that we first need list of all the nodes.

So, I am going to get that nodes is equal to G.nodes. So, we got a list of all the nodes. Now we have to out of this list we have to choose three nodes at a time. So, let us create a list: tris list which will keep which will keep all the triangles in the network. So, how do we do that? We will create a list and we will make use of combinations function here.

So, let us call it x for x in itertools.combinations out of wish list we have to choose we have to choose out of nodes list. How many choose nodes we have to choose we have to choose three nodes. So, basically what we are doing we are taking out all the combinations of three nodes from this list nodes. And since that is in the form of a tuple, we are going to store that in the form of a list and that list we will store in this list that is tris list.

So, basically this tris list will be a list of list where every item of this list will be a list having the three nodes from the network. These three nodes will indicate a triangle. Now

that we have the triangles, we have to see whether these triangles are stable or not, and which ones of these are unstable so that we can you can we can implement moving them towards a stable state.

So, for that we have to see what the weights on the edges on these triangles are. We are going to create a function for that. Before that we will create a list that we will keep track of all the signs of these edges. So, I will create a list and we will call a function we will just create that. We will pass the triangles list in that and we will pass the graph. So, this function we will just create: what this function we will do is it will take the triangles list and the graph as the parameter and it will return a list where the list where this list is a list of list. And every element of this list will have the signing details of the triangles let me write it as a comment here.

For example, this would be a list and this list will have number of other lists like this, and these lists will have the signing details of every edge. For example, this will have plus minus and so on. So, this will be list of lists this all signs which will be return by this functional will just create this function.

Now, go to the next step. Once we have gotten all the sign details of the triangles. We have to check how many unstable triangles are there. So, let me create variable here and let me create a function also: count unstable. We will pass this all signs list here so that this function can tell us how many unstable triangles in all these in all this list are, ok.

So, once these two functions are created, we are will be done with the four steps. Now, let us go back and implement this function first, get signs of triangles. So, we will go up and we will create a function. The parameters as we passed here. We basically need a triangle list and we need the graph as well, so we will pass them here. So, what do we have to do in this function as an input we have tris list? Let me show you the format of tris list. So, it's going to be list with element lists in it and this list will have the nodes; for example, 4, 5, 3. So, this is an example.

And the output has to be the sign list: like this call this one. So, basically, we have to the triangle is the one which is comprising of the nodes 1, node 2, node 3. So, we have to see the sign between 1 and 2 and we have to store it here, we have to see the sign between 2 and 3 we have to store it here, and we have to see the sign between 2 and 3 and we have to store it here right. So, the output will be something like this for example.

So, this plus indicates that the sign between first and second node of the triangle is plus. This minus indicates that the sign between second and third node of the triangle is this, and this indicates that the sign between first and third node of the triangle is this. So, this is how we are going to create the all signs list and that is what we will return from this function. Similarly, the format of the signal list and all the subsequent lists will be like this only.

So, we will create this entry list that we are going to return. Now for every list inside this what we have to do? We have to do something, so we are going to start a loop for every element list inside this list; so for i in range length of tris list. Now how do we get the sign details of these edges? The sign details are stored in the graph.

So, we are going to take the signs from the graph only. So, all the signs is an empty list it should be returned as this list. So, we have to add a list as an element here. So, I will create our temporary list and I will keep adding values to this list and ultimately we will add this list to all signs. So, how can we get the sign? The sign we will get from G and tris list i-th elements 0th entry and the first entry. This will be the first node.

And the second node will be tris list i-th element first element; I am sorry one will be there ok. So, what we are doing is; we are passing first parameter here which is tris list i-th elements 0th value. So, if this is the i-th element the 0th value will be 1. And second parameter is tris list i-th elements first entry; so i-th elements first entry which is 2 here. So, we want to get the sign of the edge between 1 and 2. So, the third parameter is the sign that we will pass here.

Now whatever sign we get out of here we have to store that in temp. So, I will write temp.append this whole thing. Similarly, we have to get the sign between 2 and 3 and we have to get the sign between 1 and 3. So, I am just going to copy paste this. So, initially this is the sign between 0 and 1, the second will be the sign between 1 and first and second entry and the third will be sign between second and 0th entry right or 0th were second entry this is undirected. So, you can use it either way.

So, by this time we have gotten this temp list which we have to store in all signs. So, I am going to write all signs.append this temp right. So, we have done it for all the all the lists in the tris list, in the end of this list we can in the end of this far loop we can return all signs. This was the implementation of the first function that is get signs of triangle.

So, we have now stored the signing details of all the triangles. And now let us get back here the second function that we have been using and I am we are yet to implement is count unstable.

So, now, we have all the sign details in this list, all the signs which has been returned by this function. This list we will pass to the next function which will count the number of plus in every list and we will tell us whether it is stable or not and it also we will keep a track of the total number of unstable triangles.

So, let us now implement this function count unstable. Let us go up here, the parameter was all signs right and it is returning variable which is the unstable. So, we will write all signs here and it is returning a variable unstable. Let us also keep a track of stable triangles. So, initialize to 0, Now we have all signs as a parameter, and you know that the format of all signs is like this where every element list has the number of causes plus and minus given in it.

So, we can simply count the number of plus and minus, and we can get to know whether the triangle is stable or not. So, we can start loop here length off all signs I am sorry, length off all signs.

(Refer Slide Time: 21:14)

```
10      for i in range(len(tris_list)):
11          temp = []
12          temp.append(G[tris_list[i][0]][tris_list[i][1]]['sign'])
13          temp.append(G[tris_list[i][1]][tris_list[i][2]]['sign'])
14          temp.append(G[tris_list[i][2]][tris_list[i][0]]['sign'])
15          all_signs.append(temp)
16      return all_signs
17
18  def count_unstable(all_signs):
19      stable = 0
20      unstable = 0
21      for i in range(len(all_signs)):
22          if all_signs[i].count('+') == 3 or all_signs[i].count('+') == 1:
23              stable += 1
24          elif all_signs[i].count('+') == 2 or all_signs[i].count('+') ==
25              unstable += 1
26
27      print 'Number of stable triangles out of ', stable+unstable, 'are',
28      print 'Number of unstable triangles out of ', stable+unstable, 'are'
29
30
```

So, for every element we have to pick one element list from all the signs, and we have to count the number of plus minus in that. Or we can simply count the number of plus as we discussed in the previous video.

So, if all the signs i-th list.count; what do we have to count we have to count plus if it is equal to 3 or if this is equal to 1; then what we have discussed in that case the list will be stable. So, we can increase the stable count in this case. On the other hand, let us use elif. On the other hand, if the total number of plus is equal to 0 or 2 then to be unstable; so, I am just copying pasting here. So, if the total number of plus is 2 or it is 0, then it will be unstable. So, we will increase the unstable count here. So, I think we are done.

So, by the end of this for loop we would have the total number of stable and unstable triangles in the network, which we want to keep track of to know whether the complete network has become stable or not. So, we can print this information so that we can keep track of it. So, let us print. Number of stable triangles; I let us also within the total number of triangles ok. Similarly, I am just doing it so that while executing the code we can keep track of what is going on; number of unstable triangles out of stable plus unstable are unstable.

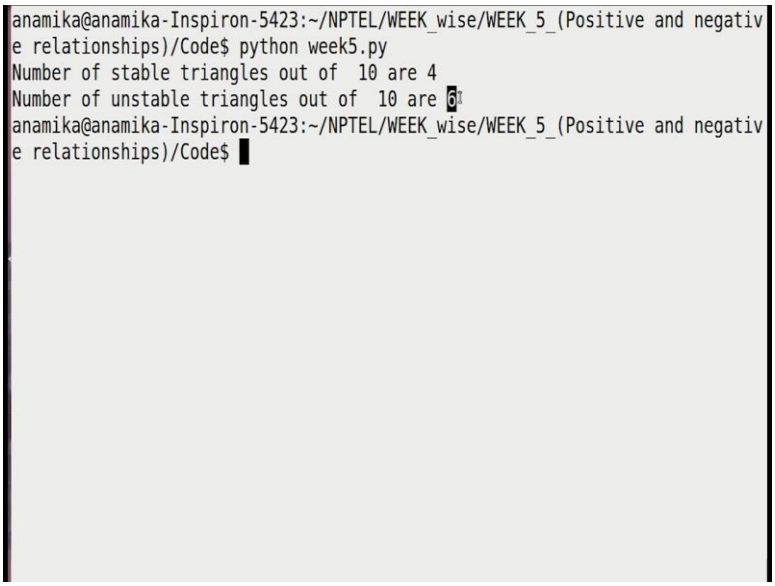
(Refer Slide Time: 23:27)

```
stable_signed_final2.py    week5.py
10  range(len(tris_list)):
11  = []
12  .append(G[tris_list[i][0]][tris_list[i][1]]['sign'])
13  .append(G[tris_list[i][1]][tris_list[i][2]]['sign'])
14  .append(G[tris_list[i][2]][tris_list[i][0]]['sign'])
15  _signs.append(temp)
16  ll_signs
17
18  nstable(all_signs):
19  = 0
20  = 0
21  range(len(all_signs)):
22  ll_signs[i].count('+') == 3 or all_signs[i].count('+') == 1:
23   stable += 1
24  all_signs[i].count('+') == 2 or all_signs[i].count('+') == 0:
25   unstable += 1
26
27  umber of stable triangles out of ', stable+unstable, 'are', stable
28  umber of unstable triangles out of ', stable+unstable, 'are', unstable
29
30
Line 28, Column 77    Tab Size: 4    Python
```

So, I think we are done with these two functions implementation let us go back here and ok.

So, we have implemented this, and we have implemented this. So, it should work fine let us try executing it and see whether it is telling us the stable and unstable triangle count or not. Let us execute this and let us check. This is the graph that we just visualized; I am going to close it ok.

(Refer Slide Time: 23:57)



We are getting this number of stable triangles out of 10 or 4. So, total number of triangles are 10, and the stable are 4 and unstable are 6 ok. So, this is working fine, let us go back.

In the next part of the video we will see how we can choose an unstable triangle and how we can move it to a stable state. So, we will do it for all the triangles and the network will eventually become stable.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Homophily (Continued) & Positive and Negative Relationships
Lecture – 71
Moving a network from an unstable to stable state

(Refer Slide Time: 00:05)

```
52 nx.draw(G, pos, node_size = 5000)
53 nx.draw_networkx_edge_labels(G, pos, edge_labels = edge_labels, font_siz
54 plt.show()
55
56 # 4.1 Get a list of all the triangles in the network.
57 nodes = G.nodes()
58 tris_list = [list(x) for x in itertools.combinations(nodes,3)]
59 # 4.2 Store the sign details of all the triangles.
60 all_signs = get_signs_of_tris(tris_list, G) #[['+', '+', '-'],[], []]
61 # 4.3 Count the number of unstable triangles in the network.
62 unstable = count_unstable(all_signs)
63
64 # 5. While the number of unstable triangles is not zero, do the followir
65 # 5.1. Choose a triangle in the graph that is unstable.
66 # 5.2. Make that triangle stable.
67 # 5.3. Count the number of unstable triangles
68 while(unstable != 0):
69     G = move_a_tri_to_stable(G, tris_list, all_signs)
70     all_signs = get_signs_of_tris(tris_list, G)
71     unstable = count_unstable(all_signs)
72
```

In the previous video we created a network on countries; we also stored the positive and negative signs across the edges. We also computed the total number of unstable triangles in the network. Our next aim is to move this network to a stable state for that what we will do is we will choose one triangle which is unstable, and we will move it to a stable state. We will keep doing this until the complete network become unstable.

So, we have to see how the number of unstable triangles fluctuates and how it changes with respect to each iteration. So, let us get started on that. We have this variable unstable which keeps track of the number of unstable triangles. So, we can start the loop here while this unstable become 0. So, we can write while unstable is not equal to 0 we have to keep repeating this. Now, what do we have to keep repeating?

Since we have to repeat this it will be better if you we create a function for this particular task. So, let me create a function here maybe move a triangle to stable state, we need to pass the graph here because that is what we will get changed during this task. And we

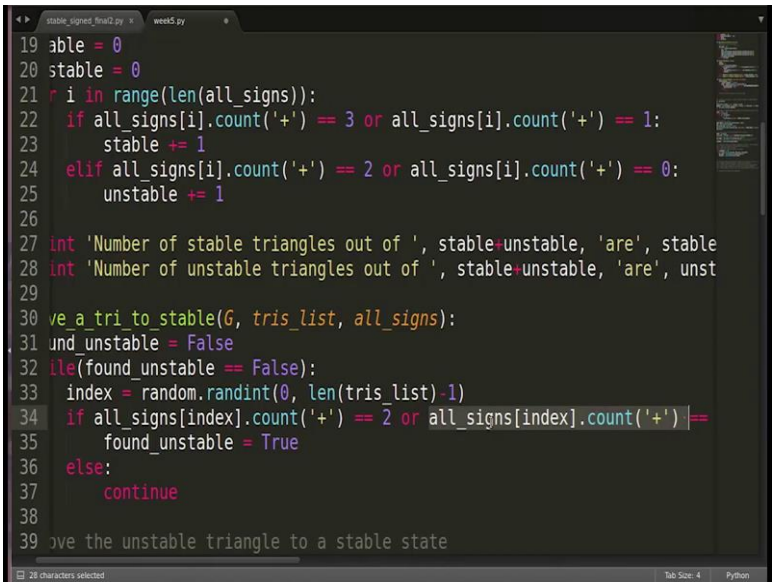
also need to pass all the triangles list we might also need all the signs there. So, let us pass this and in this function, we will make some changes in the graph with respect to the signs of one of the triangles.

So, we too want the updated graph back. So, this is the function we will just implement. Once we get the updated graph that is once one of the triangles has moved to a stable state; the signs of the adjacent triangles might also get affected. So, we need to recompute the signs of the adjacent triangles. So, we can do one thing we can do one of the two things. We can either recompute the signs of the adjacent triangles only or we can recompute the signs of all the triangles.

Here for simplicity I am recomputing the signs of all the triangles. So, basically, I am going to call this function once again; get signs of triangles which we have already created. So, I am going to call this once again on the updated graph G here; also once we get all the signs we need to check how many unstable triangles are there now right. So, we need to get the updated count here. So, we will call this function count unstable once again.

So, now up this variable unstable has the updated count of the number of unstable triangles. So, after that the control will go back here if this is not equal to 0 it will go back again inside; where it will go inside this function; where it will choose one of the one of the triangles which is unstable and we will move it to the stable state and so on. So, this loop will keep running until the entire network become unstable. So, we are done with this part we are just we just have to implement this function or move a triangle to stable.

(Refer Slide Time: 03:31)



So, let us see how we can do that; let me copy the syntax let go up and let us create this function. Now we have the graph G, we have the list of all the triangles, we have the list of all the signs on these triangles. Our aim is to choose one triangle which is unstable, now how do we do that? So, what we can do here is; we can choose one triangle out of this list randomly and then we can check whether this triangle is unstable or not. If this triangle is unstable, we have founded a triangle which we have to work on, if this triangle is not stable. I am sorry if this triangle is stable we will check another triangle.

So, we have to keep checking the triangles until we get an unstable triangle. So, how do we do that? Let us create a flag maybe found unstable = false. So, initially we have not found any unstable triangle we will keep running this loop until we find one. So, write while found unstable; while found unstable = false we have to keep running this. We have to keep running what basically we have to choose one triangle randomly out of it.

So, what we can do is; we can make use of a function run in which is there in random package. So, maybe index is the variable that will store the index of the triangle. So, so we can write random.randint. So, we can start from 0 length of triangles list minus 1. So, basically what we are doing is this triangle this tris list has some length say 0 to x. We have to choose one index out of 0 to x randomly, right.

So, what we are doing here random.randint this randint function will provide a random integer out of 0 to len(tris_list) - 1. We are doing minus 1 because the next is starting

from 0. So, that index will be stored here; now we have to check whether the triangle which is stored at this index is unstable or not. Now how do we how can we check that? We have this list all signs, which is storing all the signs for every triangle. So, we can check whether the triangle which is existing at this index has 0 pluses 1 pluses 2 pluses 3 pluses. You know what I what I mean. We will count the number of plus their positive sign is there.

So, accordingly we will judge whether the triangle is unstable or not let us see how we do that. So, what we will do if all signs the triangle which is available at this index all signs index.count; we have to count the number of plus there.

(Refer Slide Time: 06:51)

```
 = 0
le = 0
in range(len(all_signs)):
 all_signs[i].count('+') == 3 or all_signs[i].count('+') == 1:
  stable += 1
if all_signs[i].count('+') == 2 or all_signs[i].count('+') == 0:
  unstable += 1

'Number of stable triangles out of ', stable+unstable, 'are', stable
'Number of unstable triangles out of ', stable+unstable, 'are', unstable

_tri_to_stable(G, tris_list, all_signs):
unstable = False
found_unstable == False):
dex = random.randint(0, len(tris_list)-1)
 all_signs[index].count('+') == 2 or all_signs[index].count('+') == 0:
  found_unstable = True
se:
  continue

the unstable triangle to a stable state
```

If this = 2 or if this = 0; that means the triangle is unstable. So, we can write flag we have this found_unstable = true; which means we have found an unstable triangle.

In case the number of plus is not equal to 2 or 0, which means the triangle that we found was a stable one so we will continue with our loop. Now that we have found an unstable triangle, now that we have found an unstable triangle, we have to move it to a stable state. Now how do we do that? Firstly, let me write it here move the unstable triangle to a stable state ok.

Now, how do we do this? Firstly, the point to be noted is that there are two kinds of unstable triangles; one kind of unstable triangles are the ones where the number of

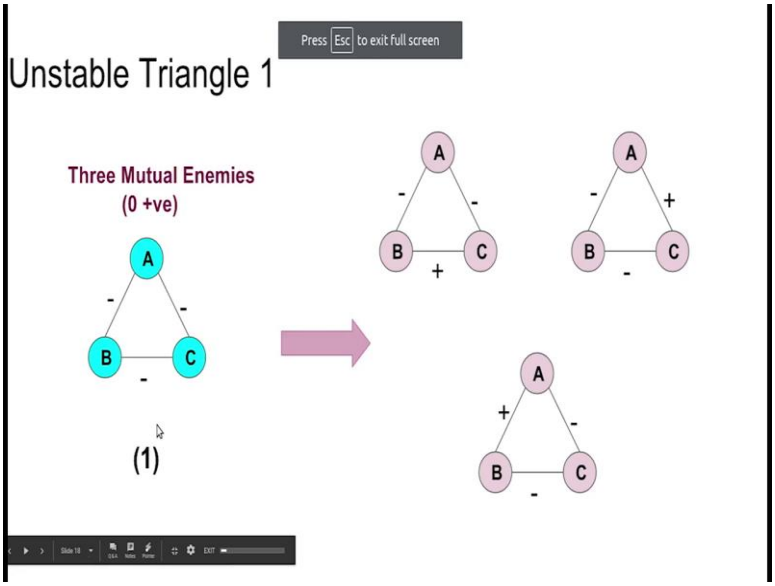
positive edges is two. And the other kinds of unstable triangles are the ones where the number of positive edges is 0. So, we are going to take different action in both the cases.

(Refer Slide Time: 08:08)

```
38
39      #move the unstable triangle to a stable state
40      #[1,2,3]
41      r = random.randint(1,3)
42      if all_signs[index].count('+') == 2:
43          if r == 1:
44              if G[tris_list[index][0]][tris_list[index][1]]['sign'] == '+':
45                  G[tris_list[index][0]][tris_list[index][1]]['sign'] = '-'
46              elif G[tris_list[index][0]][tris_list[index][1]]['sign'] == '-'
47                  G[tris_list[index][0]][tris_list[index][1]]['sign'] = '+'
48          elif r == 2:
49              if G[tris_list[index][1]][tris_list[index][2]]['sign'] == '+':
50                  G[tris_list[index][1]][tris_list[index][2]]['sign'] = '-'
51              elif G[tris_list[index][1]][tris_list[index][2]]['sign'] == '-'
52                  G[tris_list[index][1]][tris_list[index][2]]['sign'] = '+'
53          elif r ==!3:
54              if G[tris_list[index][2]][tris_list[index][0]]['sign'] == '+':
55                  G[tris_list[index][2]][tris_list[index][0]]['sign'] = '-'
56              elif G[tris_list[index][2]][tris_list[index][0]]['sign'] == '-'
57                  G[tris_list[index][2]][tris_list[index][0]]['sign'] = '+'
58
59
```

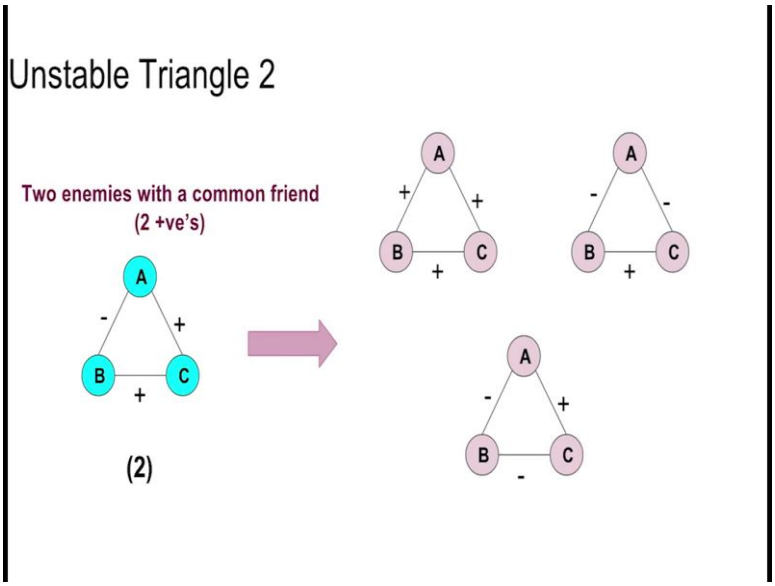
So, we are going to check in the given triangle the number of positive edges; based on that only we will move it to the appropriate state.

(Refer Slide Time: 08:22)



Now, if you remember from one of the previous videos, I had explained the two cases of unstable triangles let me show you once again. So, here you see there are two kinds of unstable triangles unstable triangle 1.

(Refer Slide Time: 08:33)



Unstable triangle 2 let us consider this case first. Now this is an unstable triangle where the number of plus is 2; in this case what happens now this state can be moved to any one of these three states. Now, what are these states? In the first case what happens this negative sign between A to B this negative sign has become positive and the rest signs remain the same.

In the second case this minus and plus remain the same, but this plus sign becomes negative this is the second case. In the third case this minus and this plus remain the same here and this plus sign becomes negative.

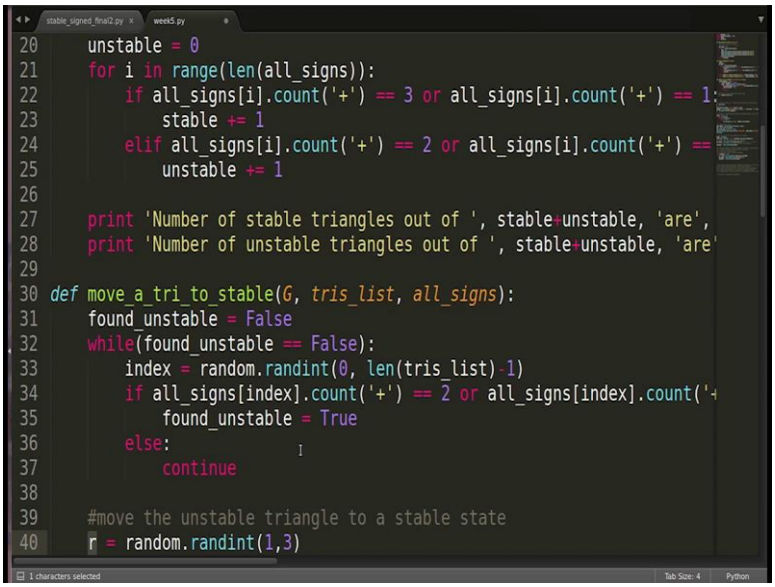
So, basically you see what is happening here in the first case this one this sign is getting inverted. In the second case the sign of the second edge is getting inverted. In the third case the sign of the third edge is getting inverted. So, basically if you take this triangle and choose any one of the edge and invert its sign; whatever you will get that will be a stable state right. So, this is the case of a triangle which has 2 positive edges. If you go back and see the case of the triangle which has 0 positive edges the scenario is a little different here.

So, accordingly we are going to implement as well so let us try to understand this. In this case this triangle which is unstable has 0 positive edges that is all edges are negative. Now these three states are the ones which are stable states which can emerge out of this state. Now what is happening in all these states? In the first state this minus these two

states are these two edges are same and the third edge becomes positive from negative. In the second state this edge becomes positive in the third state A to B edge becomes positive.

So, basically what is happening here is you choose any one of the edges and make it positive; whatever state you get will be a stable state. So, this is how we are going to implement as well. So, you see the moving of unstable state to a stable state is different in both the cases when we have 0 positives or when you have 2 positives. So, we have to keep this thing in mind while we are implementing it. So, let us go back here.

(Refer Slide Time: 11:11)



Another thing to note here is that there are three stable states corresponding to one unstable state. So, we will randomly assign one of the three stable states here. So, we will choose a random number here. So, you again use random package random.randint. So, we want a random integer from 1 to 3. So, any one of these three integers can come in r. So, if r = 1 we will move it to the first stable state if r = 2 we will move it to the second stable state and so on.

So, let us handle the first case where the number of a positive signs = 2; let us handle this case first. I am copying pasting here. So, let us see what we do if all_signs[index].count(‘plus’) = 2. So, as I explained you the 3 stable states that can correspond that can come out of this state. So, the strategy that we are going to follow here is that if r = 1; we will invert the sign of the first edge.

If r = 2 we will invert the sign of the second edge if r = 3 we will invert the sign of the third edge. So, that is the approach that we are going to follow here. Now how do we invert the sign of a given edge for that we should know what the sign of that edge is. How do we get to know what is the sign of a given edge that detail is stored in G basically G has the detail of the sign of a given edge? And how do we access the edges? These edges we will access from tris list.

So, let us see how we can do this. So, write if G so we will pass two parameters here and this the third parameter will be sign as usual simple syntax. So, what is going to the first parameter? First parameter is going to be the node the node of the first edge. How do we access the node of the first edge? All the nodes of the triangle are available in tris list. So, we are going to write tris list which index the index the index that we computed here over here. So, tris list index first node and in the second parenthesis what will come I am pasting here.

Tris list index first a value that is the first index basically means the second value ok. Let me explain it here if you find it is difficult. So, tris list is a list of lists and every list is like this ok. So, 1 will be accessed by the 0th index, 2 will be accessed by the first index and 3 will be accessed by the second by the index two which we are going to use in the next sentence. So, this will basically give us 1 and this will basically give us 2.

So, we are going to pass 1 and 2 to this G to get the sign of that G. So, so we will check whether that is positive or not; if that is positive will make it negative then that is negative and make it positive. So, we will just copy paste here. So, this was positive so we will make it negative ok. Now the same has to be done with the rest of the edges as well. So, I will copy paste I remove this; now this is for the second edge.

The second edge will be accessed with index 1 and 2 here again index 1 and 2 ok; so again, if that is positive this one catch here. So, see what we are doing here is; if the sign of the first edge was positive, we are making it negative, but if the sign of the first edge is negative, we have to make it positive as well. So, this thing we are going to do in the next statement.

So, here we are just copy pasting this. So, let us check this if the sign of the first edge was positively made it negative and if it was negative sorry negative. We will make it positive. So, we are done with the first edge and we will handle the first edge if the value

of r was equal to 1. What is the value of r = 2? Then we have to choose the second edge. So, I will write el else if r = 2 we will now handle the second edge.

Now, again we will check the sign of the second edge we just copy paste things here. If the sign of the first edge was positive, we will make it negative and otherwise I will just copy paste again. So, I will write elif if it was negative, we will make it positive. Since we are repeating things here you can also create a function over here. So, that you pass the values and it makes the changes I am just copy pasting here. And the third thing is if I if the value of r = 3 we will choose the third edge and we will invert its sign.

So, again I will copy paste the entire thing. So, I will write if r = 3 now we have to access the third edge. So, we will write 2 comma 0 or you can write 0 comma 2; write 2 0 like here like here.

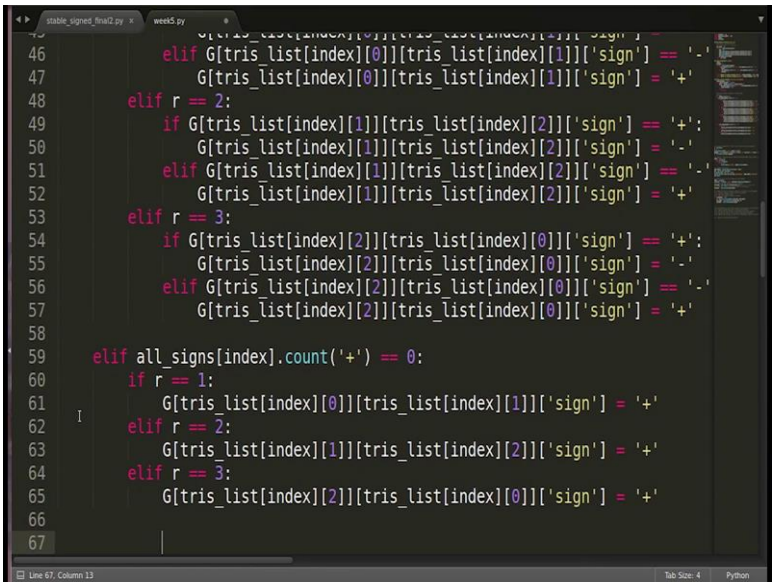
(Refer Slide Time: 17:49)

```
38
39 #move the unstable triangle to a stable state
40 #[1,2,3]
41 r = random.randint(1,3)
42 if all_signs[index].count('+') == 2:
43     if r == 1:
44         if G[tris_list[index][0]][tris_list[index][1]]['sign'] == '+':
45             G[tris_list[index][0]][tris_list[index][1]]['sign'] = '-'
46         elif G[tris_list[index][0]][tris_list[index][1]]['sign'] == '-':
47             G[tris_list[index][0]][tris_list[index][1]]['sign'] = '+'
48     elif r == 2:
49         if G[tris_list[index][1]][tris_list[index][2]]['sign'] == '+':
50             G[tris_list[index][1]][tris_list[index][2]]['sign'] = '-'
51         elif G[tris_list[index][1]][tris_list[index][2]]['sign'] == '-':
52             G[tris_list[index][1]][tris_list[index][2]]['sign'] = '+'
53     elif r == 3:
54         if G[tris_list[index][2]][tris_list[index][0]]['sign'] == '+':
55             G[tris_list[index][2]][tris_list[index][0]]['sign'] = '-'
56         elif G[tris_list[index][2]][tris_list[index][0]]['sign'] == '-':
57             G[tris_list[index][2]][tris_list[index][0]]['sign'] = '+'
58
59
```

So, if it was positive, we made it negative if it is negative, we made it positive. So, all set; now this was the case of the triangles where the number of positive edges were 2.

Next, we have to handle the case where the number of positive edges = 0. In that case as I told you what we will do we will choose any of the edge since all the edges are negative already, we will choose any one of the edges and we will make it positive. So, the resultant state is a positive is a stable state. So, what I will do is; we will write in the else part, if I know this if ok.

(Refer Slide Time: 18:23)

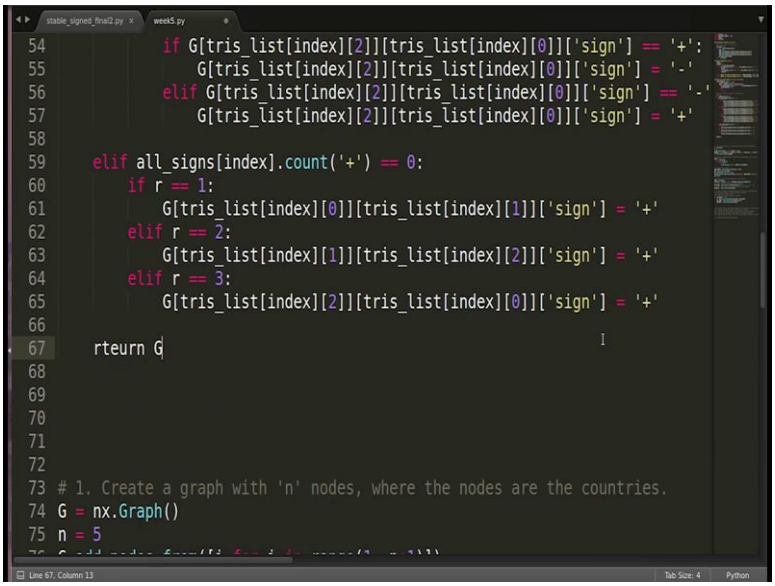


If the number of a positives edge = 0; what we are going to do ok. Again, there can be three cases here based on the value of r we will choose the edge. So, we will write if r = 1 we will choose the first edge and I we will do that else if r = 2 we will choose the second edge else if r = 3 we will choose a third edge. Now how do we choose the edges the same way that we have done. So, like you can choose like this.

Now, there is no checking here you just have to make the sign positive; because we know that already the sign is negative. So, I will write it here the sign of the first edge we are making to be positive ok. If r = 2 we will choose the second edge and second edge will be accessed by a 1 and 2; here we will make it positive. And if r = 3 we will choose the third edge and that will be accessed by 2 and 0 here ok.

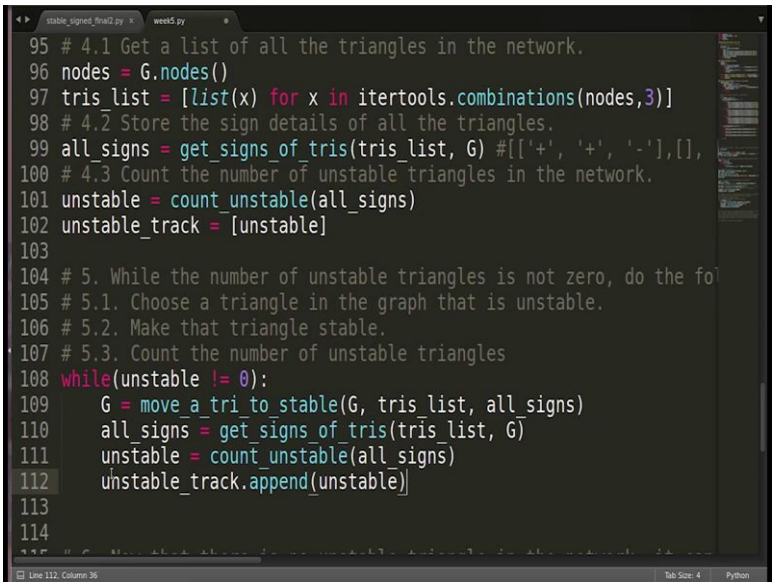
So, by doing this we have handled both the cases of unstable triangles and we have. So, basically what we have done in this function we have chosen 1 unstable triangle over here ok. We have found the index of a triangle which is unstable then we are checking whether; what is the kind of that unstable triangle; based on that we are taking the action. Let us go back sorry based on that we are taking action if the signs are 2 we are executing this if the signs are 0 positive signs is 0 we are executing this.

(Refer Slide Time: 20:24)



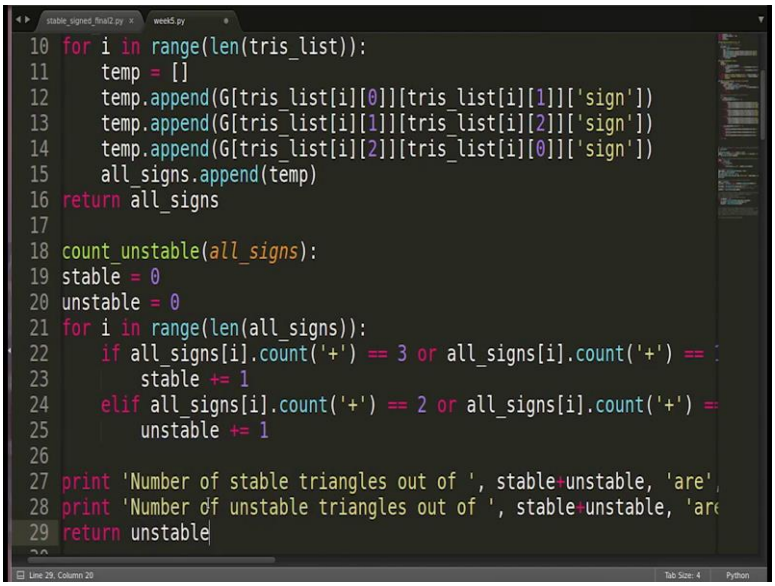
So, we are done here and then we will return the updated graph here.

(Refer Slide Time: 20:32)



Let us go back here and see if everything works fine. So, here we are calling this function which we have just now implemented and after calling this function, we are again computing all the signs, and we are counting the number of unstable triangles. This function is returning this variable unstable we can also keep track of this variable.

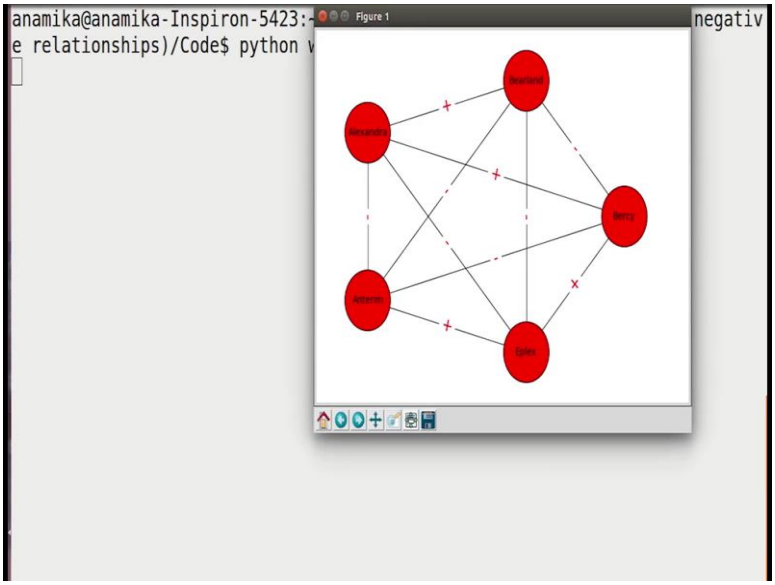
(Refer Slide Time: 20:57)



This one statement missing here I am sorry; we did not return this variable.

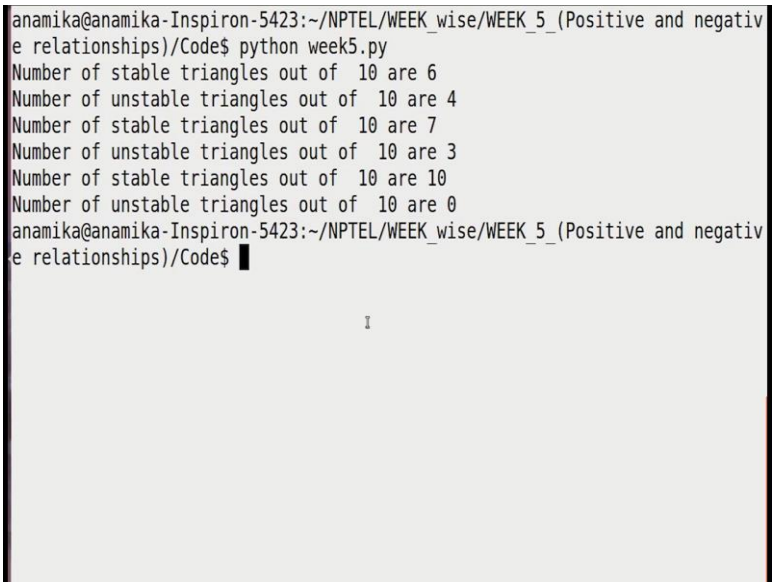
We have to return unstable here.

(Refer Slide Time: 21:19)



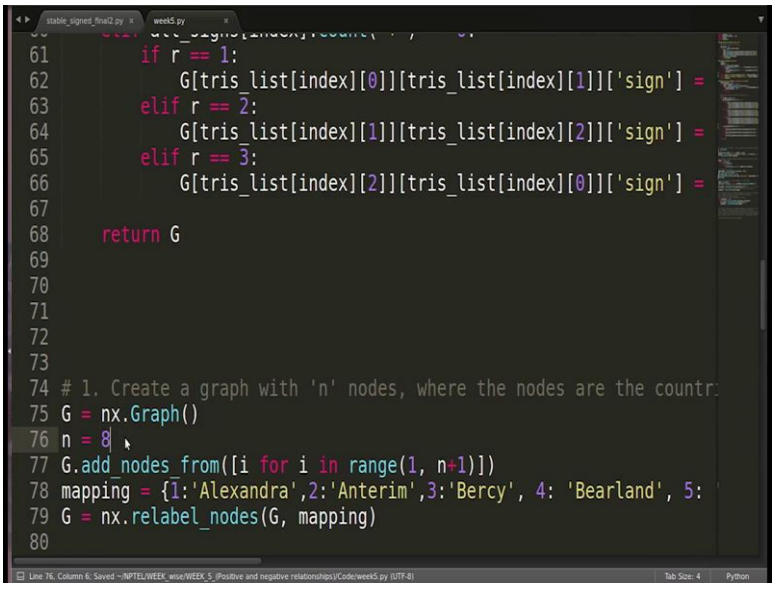
So let us check, let us verify our code we can check this. So, so this is the network that we had plotted that we had drawn earlier.

(Refer Slide Time: 21:28)



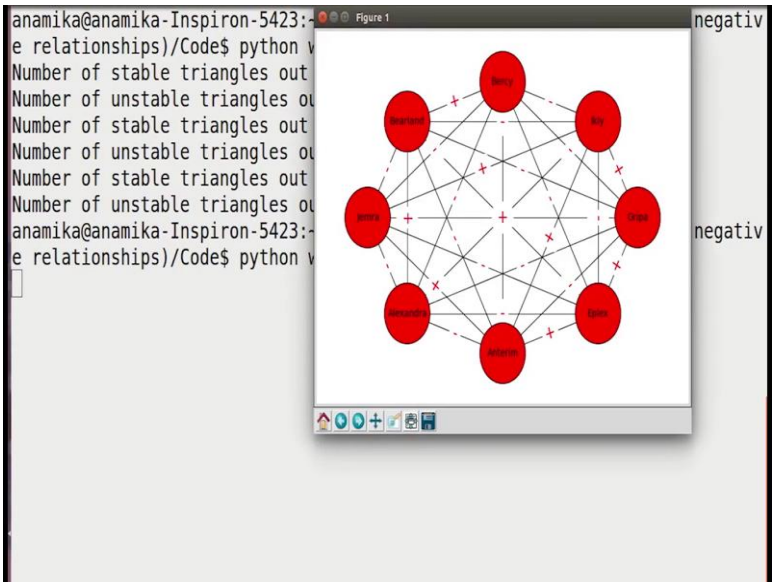
This is the output you see initially the number of unstable triangles was 4 and then it became 3 and then it became 0. So, this was very small network. So it happened it converged quite quickly.

(Refer Slide Time: 21:47)



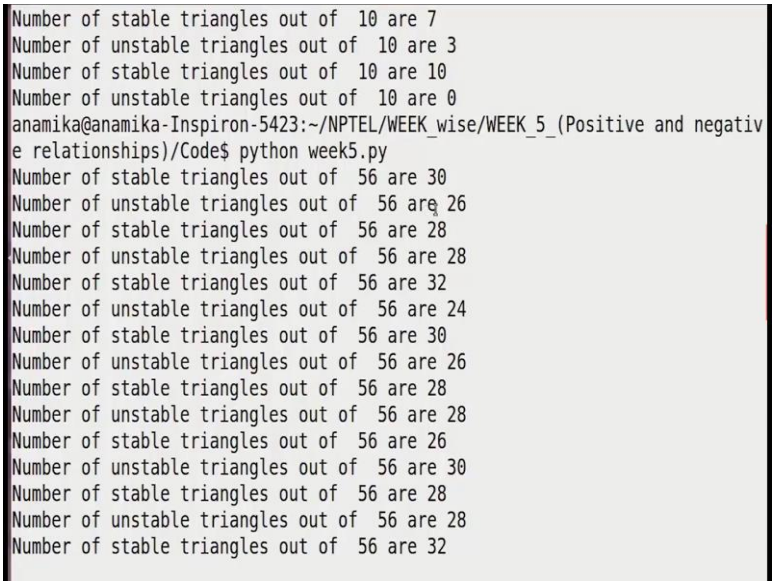
Let us for example, increase this number of countries to say 8 and see how it works.

(Refer Slide Time: 21:53)



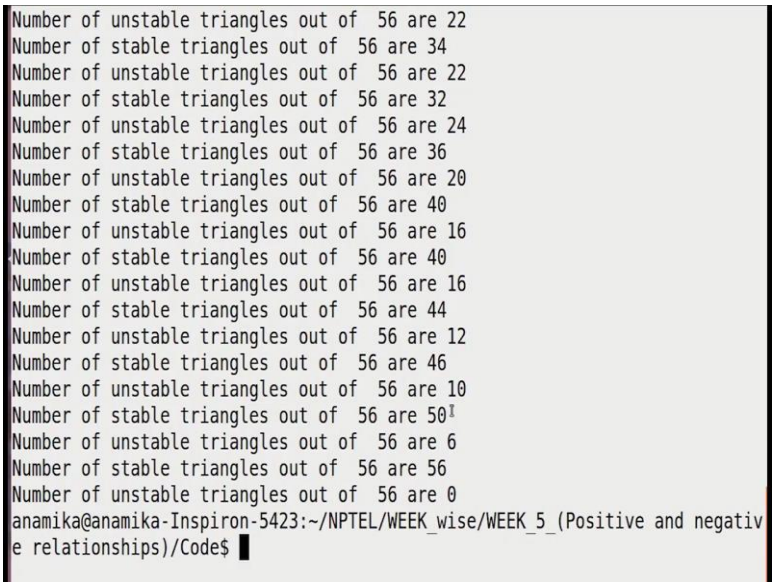
So, this is the network that gets created out of 8 countries and these are the edges I am closing it.

(Refer Slide Time: 22:05)



Here you see; you go up the number of unstable triangles was the 26 then it became 28 you see. After converting 1 unstable triangle to a stable one; the number of unstable triangles increased to 28 then it reduced then it increased, increased, increased and then it decreased. So, as you can see it kept fluctuating.

(Refer Slide Time: 22:29)

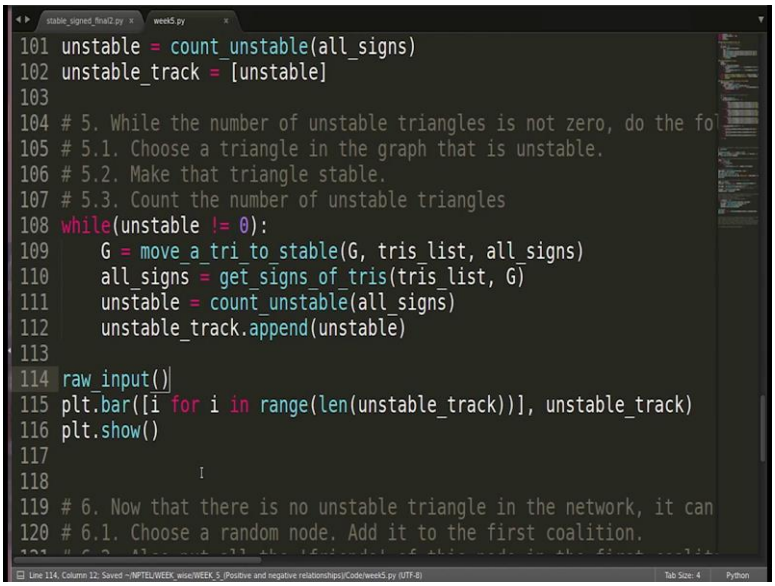


So, it keeps fluctuating, and, in the end, it became 10 and then quickly 0 and then quickly it became 0.

So, if you want you can also plot this let me show you quickly how we can do that. So, basically, we want to see how the number of unstable triangles is changing when we keep moving 1 unstable triangle to a stable state in each iteration. So, this is the count of the unstable triangles; let us create a list and keep storing the number of unstable triangles there. So, let me create a list here unstable track and initially let it contain the number of the initial number of unstable triangles.

And as and when in every iteration we are getting the number of unstable triangles we will keep appending this number unstable to this list alright. So, we will write unstable track sorry unstable track.append unstable. So, this unstable track is a list which has all the number of unstable triangles with respect to each iteration and if you want to plot you can plot that as well.

(Refer Slide Time: 23:46)

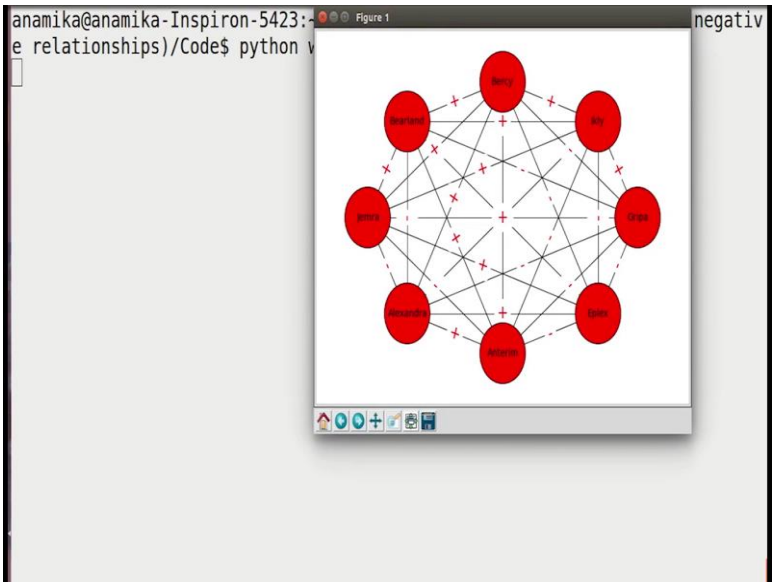


So, we can do plt.plot if you want to bar chart you can do that also.

So, the function is plt.bar and it requires 2 inputs the first will be the values on it on the x axis. So, we can have the values to start from 0 on the x axis. So, maybe we can pass the list here I can write I for I in range we can just pass the length of this list here so that the number of parameters match. So, this was our x axis and the y axis will of course be unstable track and then we can show it plt.show.

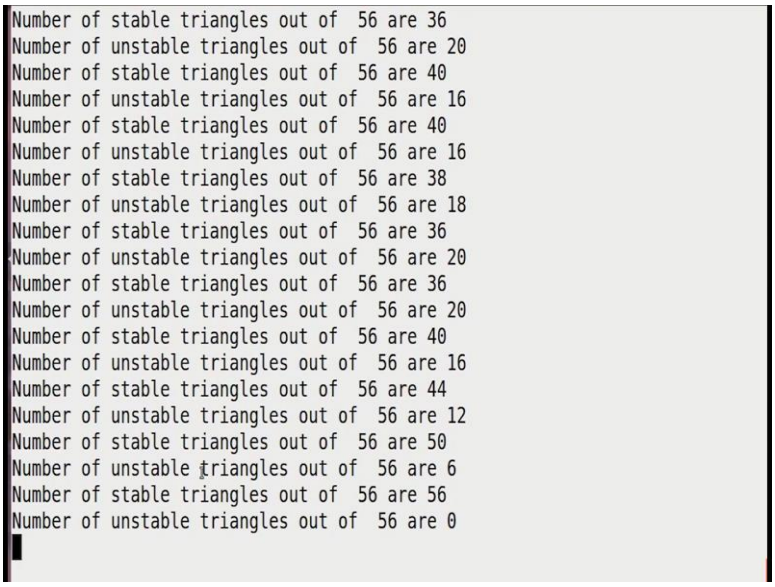
So, we can check whether it works before that we can use this function. So, that it us for an enter before it precedes. So, let us check the function we can.

(Refer Slide Time: 24:42)



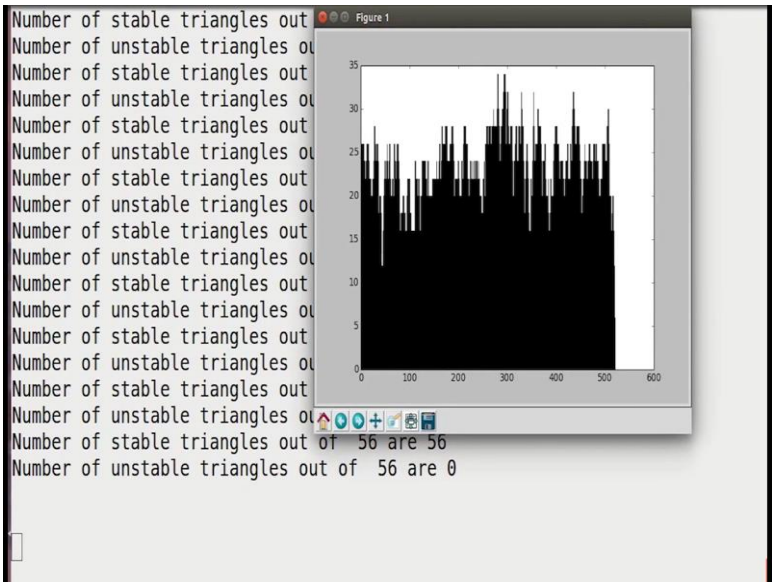
So, this is the network.

(Refer Slide Time: 24:47)



Let us close it and this is the how the number of unstable triangles changed.

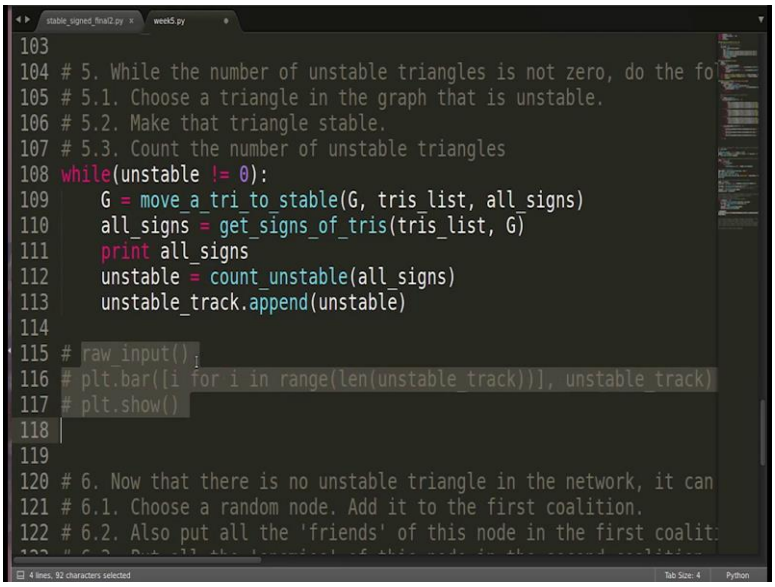
(Refer Slide Time: 24:53)



And I am just pressing enter here and this is the plot that we got. So, as you can see the fluctuations that happened in the number of unstable triangles; I have not written anything on the x and y axis, but you can understand the y axis means the number of unstable triangles.

So, it reach reached here somewhere here and then quickly came to 0. So, that is how it happens let me close it now. Let us go back.

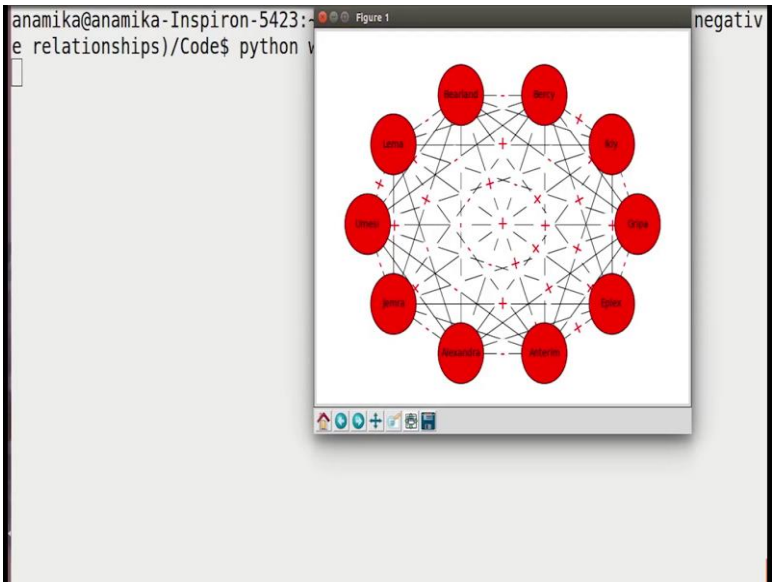
(Refer Slide Time: 25:21)



Now, if you want you can also keep printing this list which is all signed so that you can see how the signs of the triangles are changing. So, let me quickly show you that I am printing all signs. Since we have already seen the plot. I am just commenting this part and let me increase the number of cities to say 10, alright.

So, basically, we want to see the signs how they are changing from the initial configuration through the last configuration where all the triangles are stable.

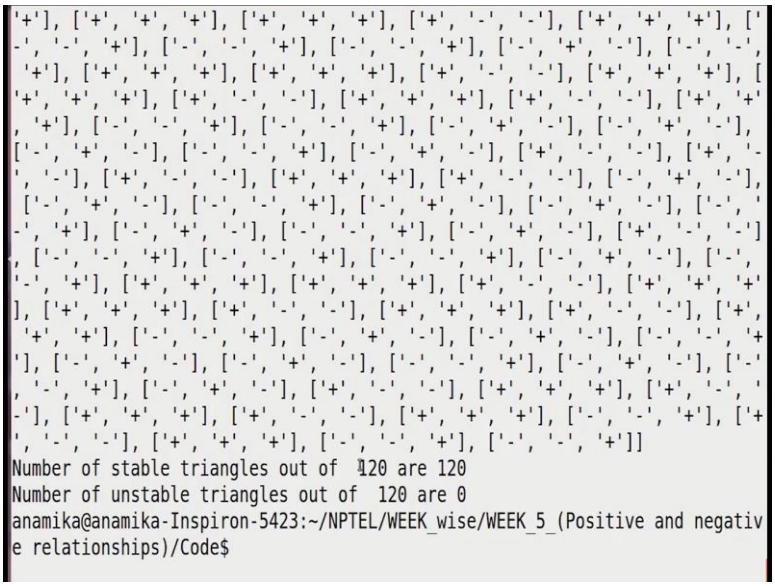
(Refer Slide Time: 25:56)



So, let us check this is the network with ten countries. So, it is taking some time as you keep increasing the countries it takes a whole lot of time from 8 to 10; it is taking a quite a lot of time; it is taking some time.

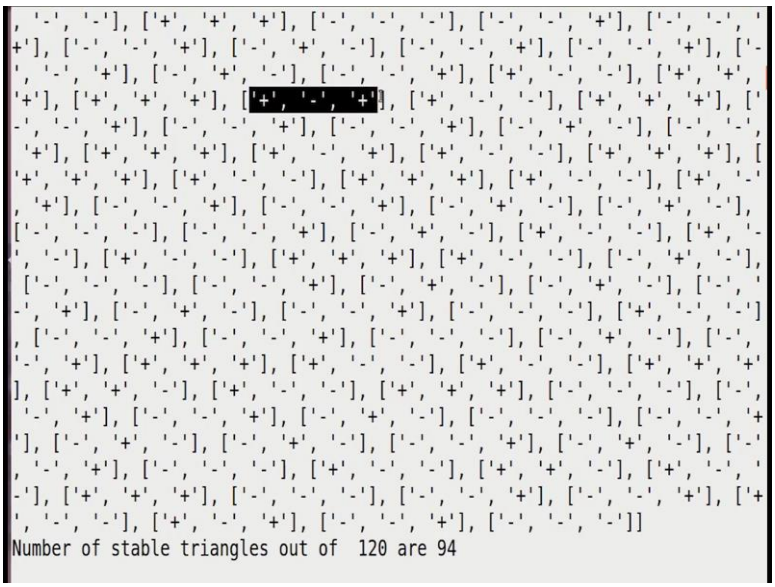
So, as you can see this is the list, which is all signs and once it stops, I will show you some examples here; it is actually taking a lot of time ok. I think I can pause the video and show you the final result ok; I will show you the final result now.

(Refer Slide Time: 26:46)



You see this is the final result: in the end out of one twenty triangles we in the end we have all the stable triangles, and if you go back I am just going a little up.

(Refer Slide Time: 26:59)



Here, you can see a number of unstable triangles for example, here there are 2 positives. So, this is one of the unstable triangles and there are 0 positives. Here this is this is another unstable triangle. However, over time it keeps evolving and in the end whatever you get. So, this all list is completely stable there, there is no triangle which has 0 or 2 positive signs. So, this is how it looks like we can go back here.

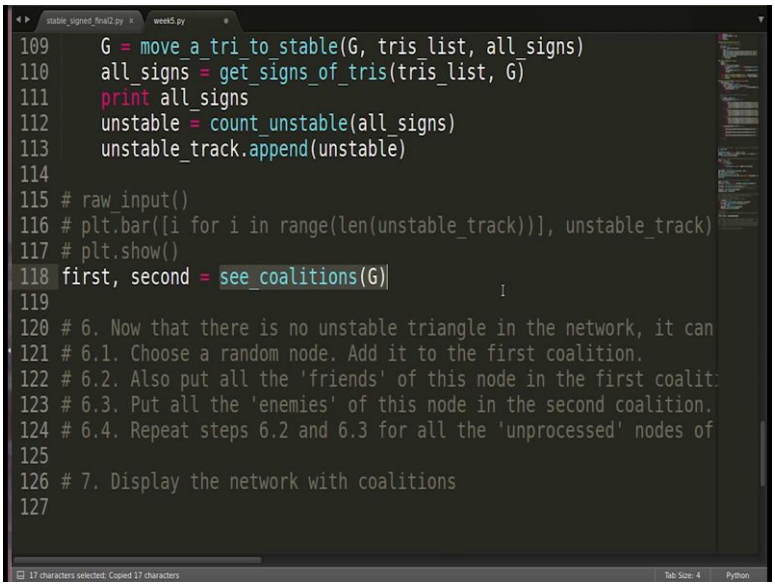
So, we done with evolving the network from us from an unstable state to a stable state As I told you once a network becomes stable it can be divided into two groups two coalitions I am not calling them communities for a reason because in we did community division in one of the previous videos. So, you should not get confused with that algorithm and this method, because community is basically; community detection is completely different algorithm depending on the number of links. Here the network is complete, here we are concerned with the signs of the edges, so that is why we are not calling them division in two committees recalling them division to coalitions or groups.

So, in the next video we are going to divide this network into two groups given that it has become stable now.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Homophily (Continued) & Positive and Negative Relationships
Lecture – 72
Forming two coalitions

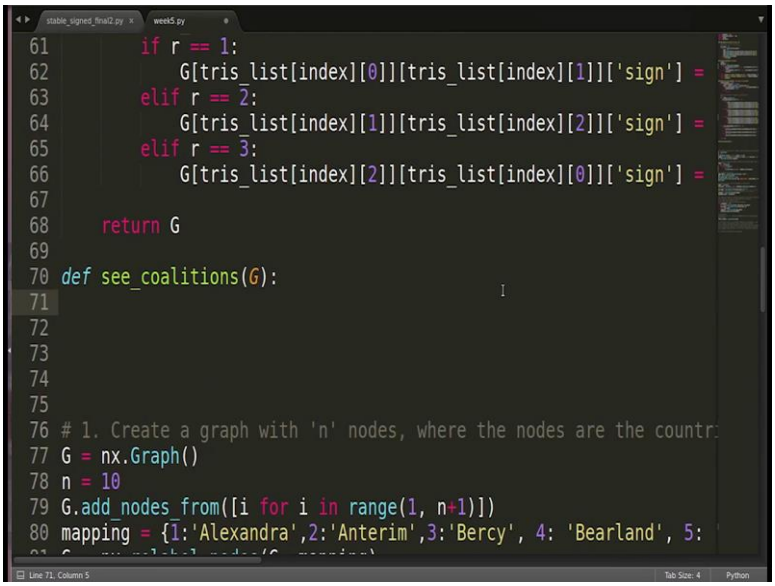
(Refer Slide Time: 00:05)



In the previous video we moved the network from an unstable state to a stable state, where there are no unstable triangles. So, once that happens it is possible to divide the nodes of the network into two groups. So, that is what we are going to do here now.

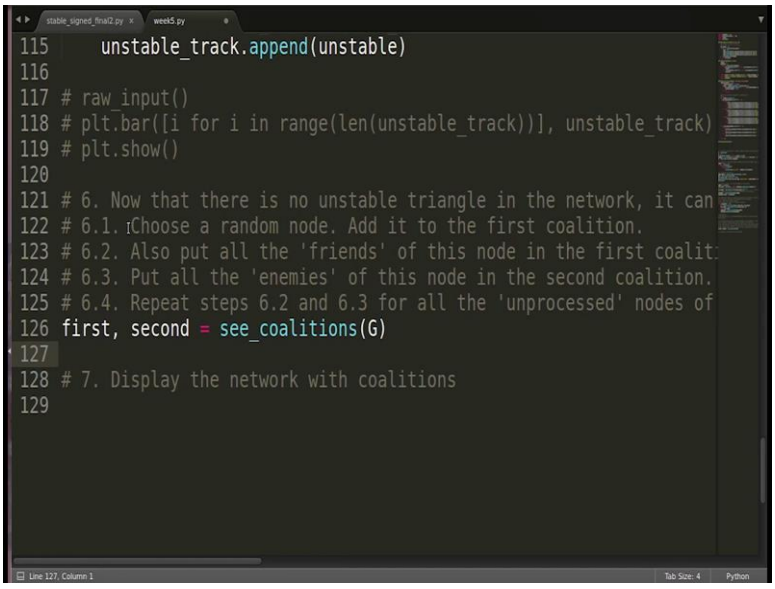
Let us create a function for that and let the function return to lists, where the first list will contain the nodes of the first row and second list will contain the nodes of the second row. So, let us create this function and let us call it see coalitions and let us pass the graph here.

(Refer Slide Time: 00:53)



So, let us create this function here let us go up and we will create this function. So, what do we have to do here? As I told you previously the strategy that we are going to follow as we explained here with me.

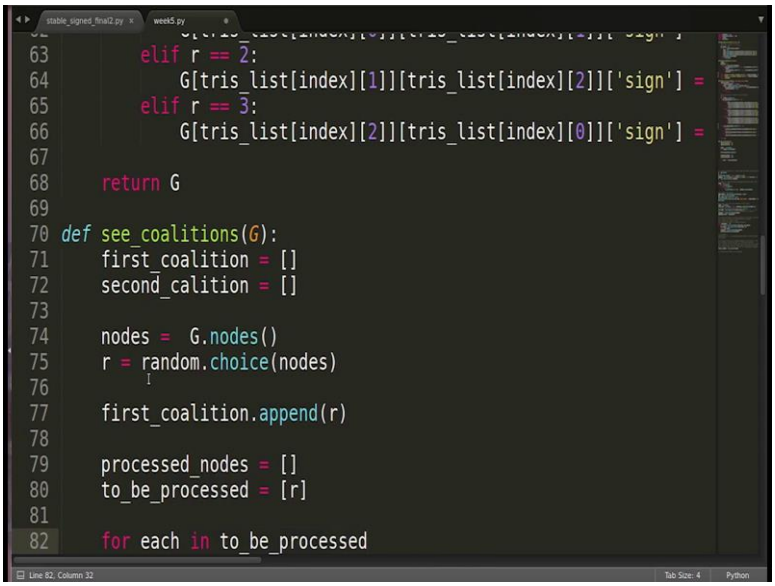
(Refer Slide Time: 01:04)



Let me write it here because this is the sixth step the second last step.

So, we are going to choose a random node and we will add it to the first coalition.

(Refer Slide Time: 01:18)



Now, how do we choose a random node? Very simple, we will first access the list of nodes; nodes is equal to G.nodes. So, we got a list of nodes now what we have to do is we have to choose one node randomly out of it. The function that we can use for that is ran int that we have previously used as well or we can simply use the function random.choice which gives us one entry out of a given list. So, we have this list as nodes. So, we can use any of the functions.

Let us use the function choice r is the random node. So, we will write random.choice; the choice should be out of this list nodes. So, we got a random node here now since we have return also and we have to keep track also let us create two lists for each coalition, first coalition empty list and similarly second coalition another entry list. So, we are going to add nodes to this coalition.

Now, as I told you we will choose a random node and we will add it to the first coalition. So, we will write first I am sorry first coalition.append r right now basically what we have to do is; we have to run a sort of a (Refer Time: 02:49) first search on this node we can added condition that all the enemies of this node are not going to be a part of the bfs tree right.

So, as I told you we will look at the neighbours of this node and some of these neighbours will be positive some of these neighbours will be its friends and some of its neighbours will be its enemies. So, the neighbours which are its friends are going to be a

part of the bfs tree, they will be explored further. However, the nodes which are going to be which are its enemies are not going to be part of the bfs tree they will not be explored they will simply be put in the second coalition list. So, that is the strategy you can use any other strategy as well. So, this is one simple method that works well. So, as we do in bfs also we keep a track of the nodes which have been processed already the nodes which are yet to be processed. So, we are going in the same fashion.

So, let us create for that purpose two lists processed nodes initialized to empty to be processed same. So, this list will keep track of all the nodes which are to be processed. Now since r is the node which has to be processed, we will put r into it ok. Now we will we have to keep running the loop of bfs until this to be processed list becomes empty right you will write for each in to be processed.

(Refer Slide Time: 04:32)

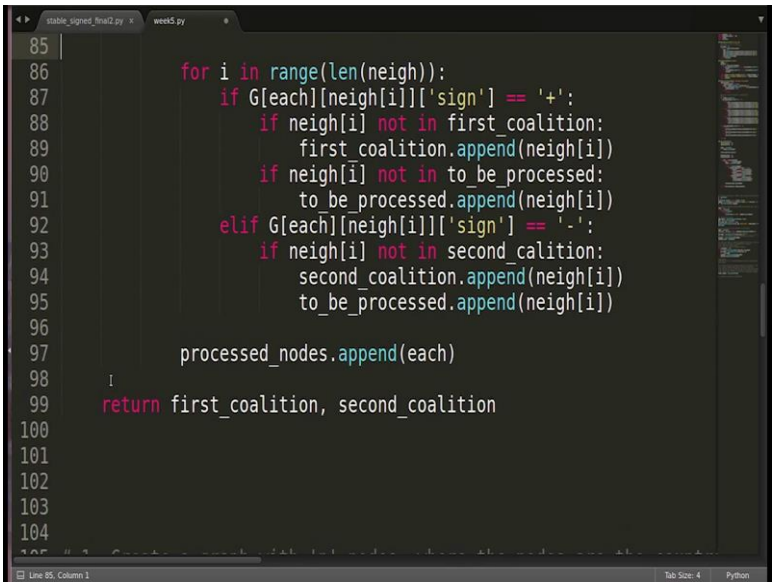
```
74      nodes =  G.nodes()
75      r = random.choice(nodes)
76
77      first_coalition.append(r)
78
79      processed_nodes = []
80      to_be_processed = [r]
81
82      for each in to_be_processed:
83          if each not in processed_nodes:
84              neigh = G.neighbors(each)
85
86              for i in range(len(neigh)):
87                  if G[each][neigh[i]]['sign'] == '+':
88                      if neigh[i] not in first_coalition:
89                          first_coalition.append(neigh[i])
90                      if neigh[i] not in to_be_processed:
91                          to_be_processed.append(neigh[i])
92
93
```

Now, in case the node is an enemy what has to be done? It has to be added to the second coalition and it does not have to be processed. So, we will write elif let me copy paste if it is a negative sign, we will add it to second coalition if it is not already there. So, we will check if neigh[i] not in second_coalition we will add it there. So, we will write second coalition.append(neigh[i]) that is one thing. Secondly, it does not have to be process. So, we are going to add it to be processed list, we will write to be processed.append(neigh[i]) right, ok.

Now, by this time r node that is each which we started with has been completely processed. So, we will write here, we will write since it does that it has been processed we will write processed nodes.append each. So, whatever we just processed, we will added to this list. So, that we have done.

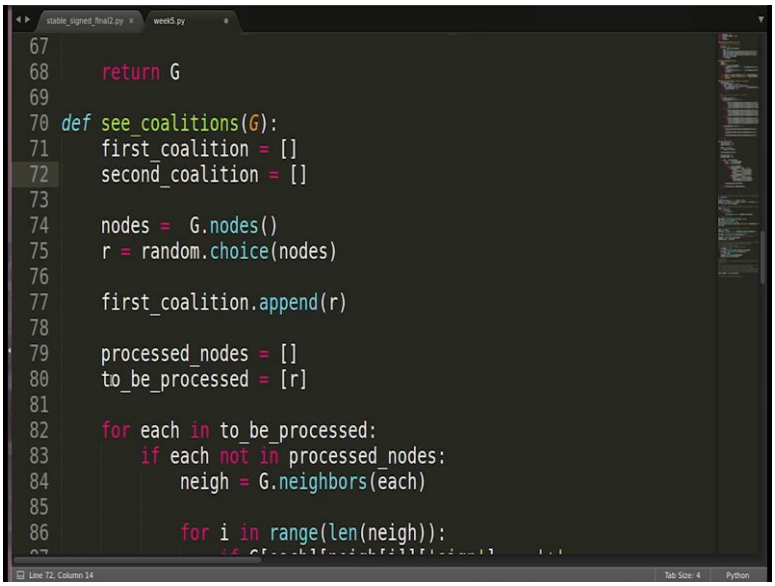
So, this was the method to divide into first_coalition and second_coalition now this function must return these two lists.

(Refer Slide Time: 05:57)



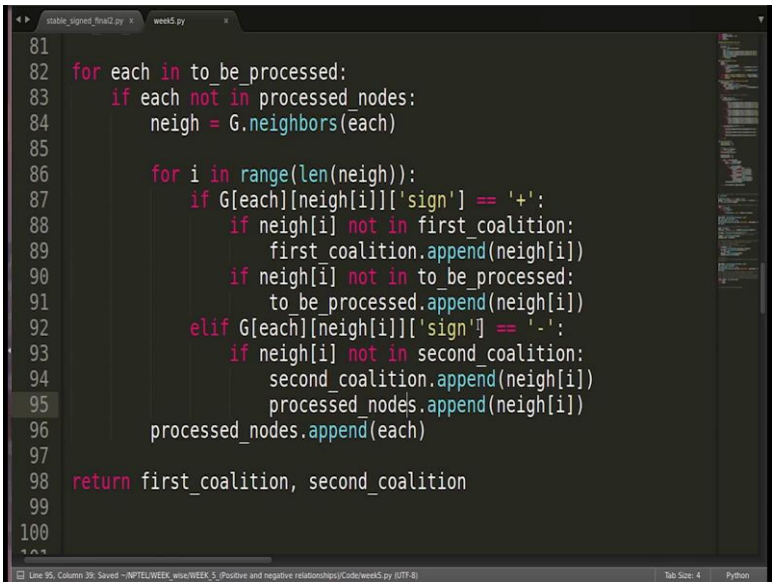
So, we will make it return here ok. We will write return first coalition second coalition if the spelling is wrong is it; oh, I am sorry second coalition its wrong everywhere I am sorry where else here ok.

(Refer Slide Time: 06:23)



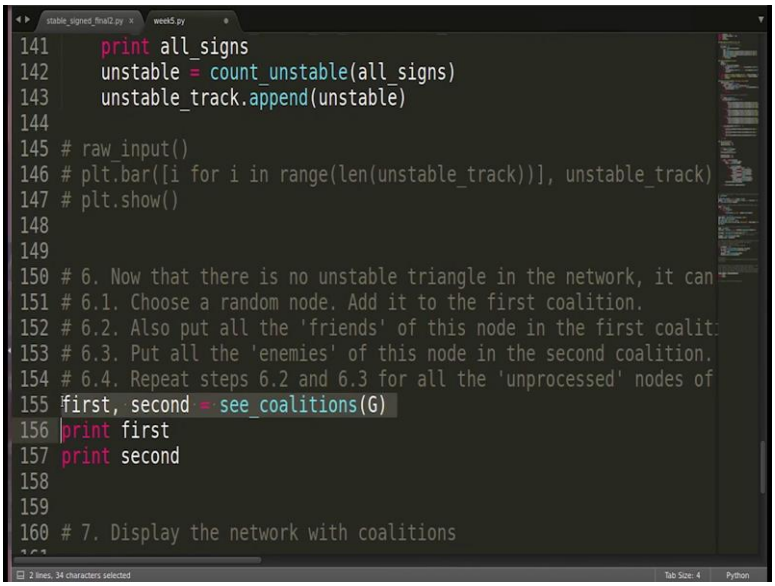
I think we are done (Refer Time: 06:28).

(Refer Slide Time: 06:28)



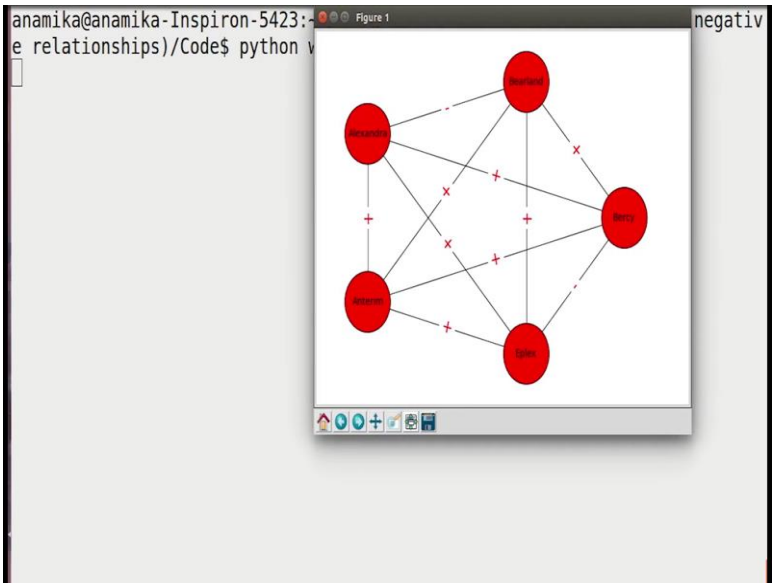
We have added the friends of a given node to the first coalition and the enemies of the given node to the second coalition, and we are not processing the nodes which are going to go in the second coalition. Here, I am sorry we do not need to process the nodes which are which are added to the second coalition. So, they will go to processed nodes here, because they are not going to be processed again ok.

(Refer Slide Time: 07:00)



So, this is the function and let us go back here and see if it works well. So, we call this function see coalitions and its returning the two lists of nodes first and second and then we are printing these two lists.

(Refer Slide Time: 07:15)



Let us see how it functions. So, let us run this, this is the nodes this is the networks with five countries I am closing it. So, that it can go ahead.

(Refer Slide Time: 07:30)

```
Number of unstable triangles out of  10 are 5
[['+', '-', '+'], ['+', '+', '+'], ['+', '+', '+'], ['+', '+', '+'], ['+', '
-', '+'], ['+', '+', '+'], ['-', '+', '+'], ['-', '-', '+'], ['+', '+', '+']
, ['+', '+', '-']]
Number of stable triangles out of  10 are 6
Number of unstable triangles out of  10 are 4
[['+', '+', '+'], ['+', '+', '+'], ['+', '+', '+'], ['+', '+', '+'], ['+', '
-', '+'], ['+', '+', '+'], ['+', '+', '+'], ['+', '-', '+'], ['+', '+', '+']
, ['+', '+', '-']]
Number of stable triangles out of  10 are 7
Number of unstable triangles out of  10 are 3
[['+', '+', '+'], ['+', '+', '+'], ['+', '+', '+'], ['+', '+', '+'], ['+', '
+', '+'], ['+', '+', '+'], ['+', '+', '+'], ['+', '+', '+'], ['+', '+', '+']
, ['+', '+', '+']]
Number of stable triangles out of  10 are 10
Number of unstable triangles out of  10 are 0
Eplex is random
['Eplex', 'Alexandra', 'Bercy', 'Anterim', 'Bearland']
[]
anamika@anamika-Inspiron-5423:~/NPTEL/WEEK_wise/WEEK_5_(Positive and negativ
e relationships)/Code$
```

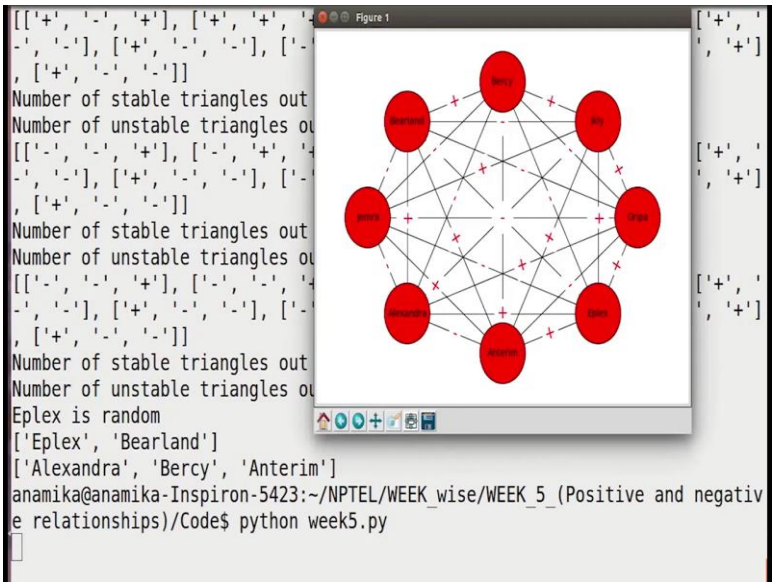
Let me show you one some more country.

(Refer Slide Time: 07:34)

```
 98     return first_coalition, second_coalition
 99
100
101
102
103
104 # 1. Create a graph with 'n' nodes, where the nodes are the countr
105 G = nx.Graph()
106 n = 5
107 G.add_nodes_from([i for i in range(1, n+1)])
108 mapping = {1:'Alexandra',2:'Anterim',3:'Bercy', 4: 'Bearland', 5:
109 G = nx.relabel_nodes(G, mapping)
110
111 # 2. Make it a complete graph by adding all possible edges. Also, a
112 signs = ['+', '-']
113 for i in G.nodes():
114     for j in G.nodes():
115         if i != j:
116             G.add_edge(i, j, sign = random.choice(signs))
117
```

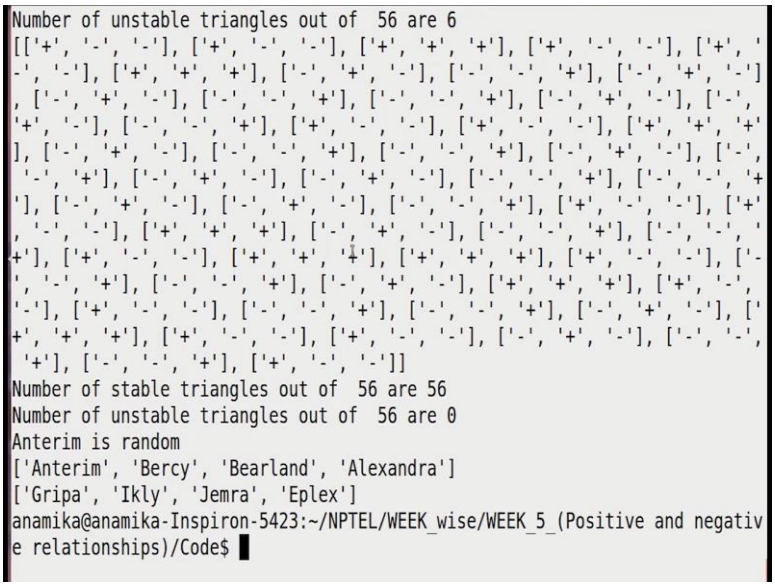
So, that we are able to nicely see the division into two groups: let me increase this to 8 and in c.

(Refer Slide Time: 07:44)



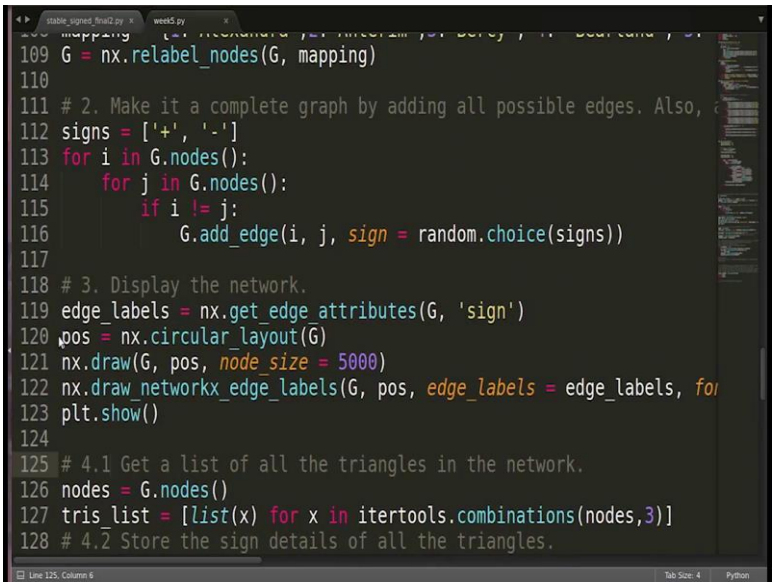
So, these are the 8 countries let me close this.

(Refer Slide Time: 07:46)



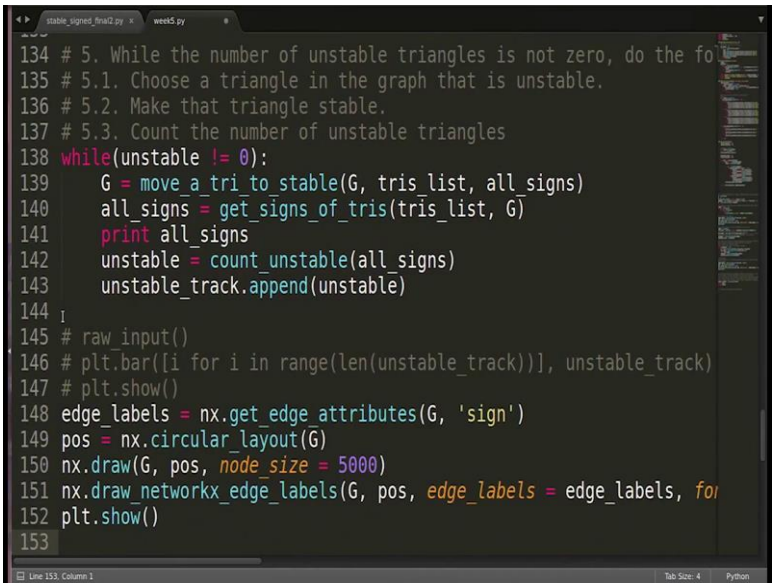
So, it moves the triangle to stable state and then it divided into two groups. As you can see out of the 8 countries four are in first group and the rest four are in the second group if we want to visualize as we can see.

(Refer Slide Time: 08:08)



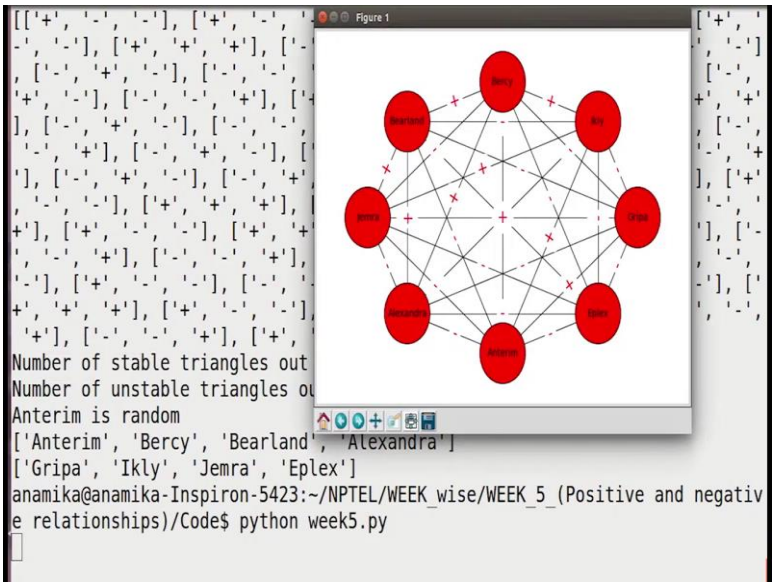
Let me add code of displaying the communities as well the coalitions ok. This is how we displayed the graph previously; I am just copying the code; so that we can display it once again.

(Refer Slide Time: 08:21)



So, I am executing it again.

(Refer Slide Time: 08:24)



So, this is the initial network; let us take one example triangle. So, Bearland, Jenera and Bercy they have an unstable relation, see there are two positive relationships. So, basically Jenera and Bercy are enemies to each other, but they have a common friend which is Bearland. So, let us see what happens to this. I am closing it and its doing its thing. And yes, this is what we had to see. Bearland, Jenera and Bercy they all are friends now. So, over time what happens? Jenera and Bercy which were initially enemies they become friends to each other. So, you can check more examples as well. So, overall, this network is completely stable.

In the next video we are going to visualize these two groups that get formed.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Homophily (Continued) & Positive and Negative Relationships
Lecture – 73
Forming two coalitions (Continued)

(Refer Slide Time: 00:05)

```
72      second_coalition = []
73
74      # nodes =  G.nodes()
75      r = random.choice(nodes)
76      print r, 'is random'
77      first_coalition.append(r)
78
79      processed_nodes = []
80      to_be_processed = [r]
81
82      for each in to_be_processed:
83          if each not in processed_nodes:
84              neigh = G.neighbors(each)
85              for i in range(len(neigh)):
86                  if G[each][neigh[i]]['sign'] == '+':
87                      if neigh[i] not in first_coalition:
88                          first_coalition.append(neigh[i])
89                      if neigh[i] not in to_be_processed:
90                          to_be_processed.append(neigh[i])
91
```

We will take nodes from this list to be processed and we will keep checking their neighbors. So, we will start a loop for each in to be processed, if each not in process node. So, if this node has not already been processed, if each not in processed node then we will look at his neighbors; so let us have this list neigh[i]s equal to G dot neighbors we have to look at the neighbors of each. So, we will write like this.

Now we have take these neighbors one by one and we have to see whether these neighbors are friends or enemies. If they are friends, they will be added to the first list, if they are enemies they will be added to the second list. Apart from that if they are friends and they are being added to the first list, they have to be processed afterwards. So, they will be added to the list to be processed.

And if they are enemies and they are being added to the second list, then they will not be processed further. So, they will be added to the list process nodes. So, we will write for i in range, length of this list because to get to know how many neighbors are there of

course, in this particular example there are n - 1 precisely n - 1 neighbors, but we can generalize this and we can write up to length of neighbors.

So, once we take the neighbor, we have to check the relationship of this neighbor with the node that is each. So, we will write G each to neigh[i] and how do we check the relation by using this attribute sign. So, we will start this if this is equal to plus which means it is a friend, as I told you what has to be done it has to be added to the list of first coalition.

So, if it is not already there. So, we will check if neigh[i] not in first coalition, then it will be added there. So, you write first coalition.append(neigh[i]) done. And second thing that has to be done is it has to be added to the list of to be process nodes if it is not all be there. You will write if neigh[i] not in to be processed and it will appended there to be processed dot append neigh[i] alright

And in the other case when the neighbor is an enemy, what we are going to do is.

(Refer Slide Time: 02:55)

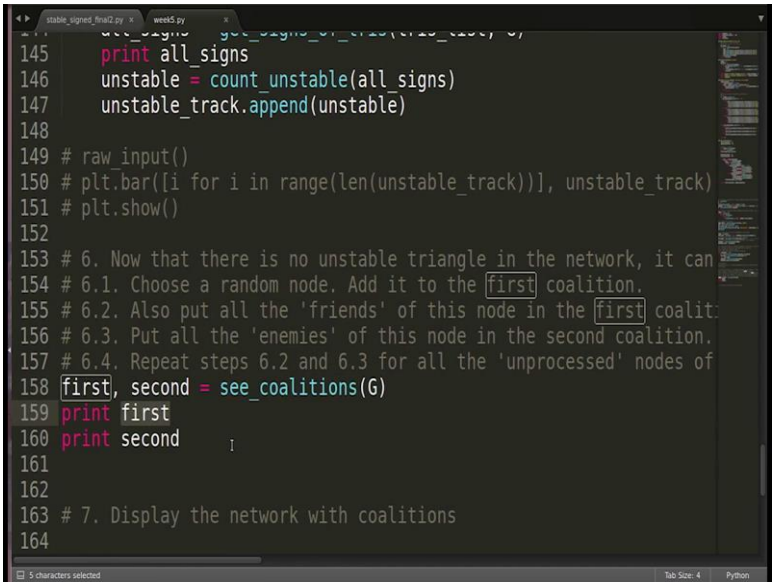
```
81
82      for each in to_be_processed:
83          if each not in processed_nodes:
84              neigh = G.neighbors(each)
85              for i in range(len(neigh)):
86                  if G[each][neigh[i]]['sign'] == '+':
87                      if neigh[i] not in first_coalition:
88                          first_coalition.append(neigh[i])
89                      if neigh[i] not in to_be_processed:
90                          to_be_processed.append(neigh[i])
91                  elif G[each][neigh[i]]['sign'] == '-':
92                      if neigh[i] not in second_coalition:
93                          second_coalition.append(neigh[i])
94                          processed_nodes.append(neigh[i])
95              processed_nodes.append(each)
96
97      return first_coalition, second_coalition
98
99
100
```

So, will write elif and this thing which means this is minus, so its an enemy. What has to be done? It has to be added to the second coalition list if it is not already there. So, we will write if neigh[i] not in second_coalition, it will be appended neigh[i] ok. Second in that has to be done is as I told you it does not have to be processed again. So, it has to be added to the list of processed nodes.

So, we can directly just add it there, processed_nodes.append(neigh[i]). Next thing that has to be done is this each the node which is called each has been process now. So, it has to be added to the list of process nodes. So, we will write here processed nodes dot append each ok. So, that is done

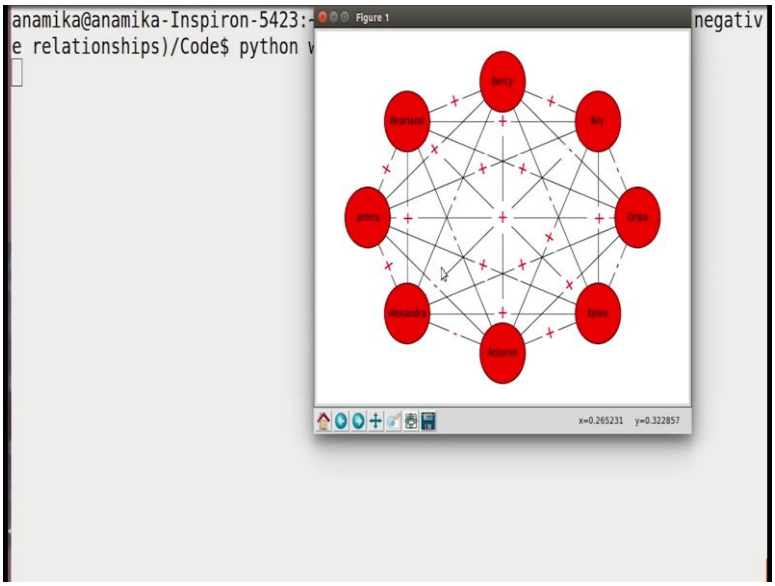
So, there this loop should basically divide the nodes in the two lists that is first coalition and second coalition and that is what this function will return. So, we are going to return this. So, we will write return first coalition, second coalition this should work.

(Refer Slide Time: 04:25)



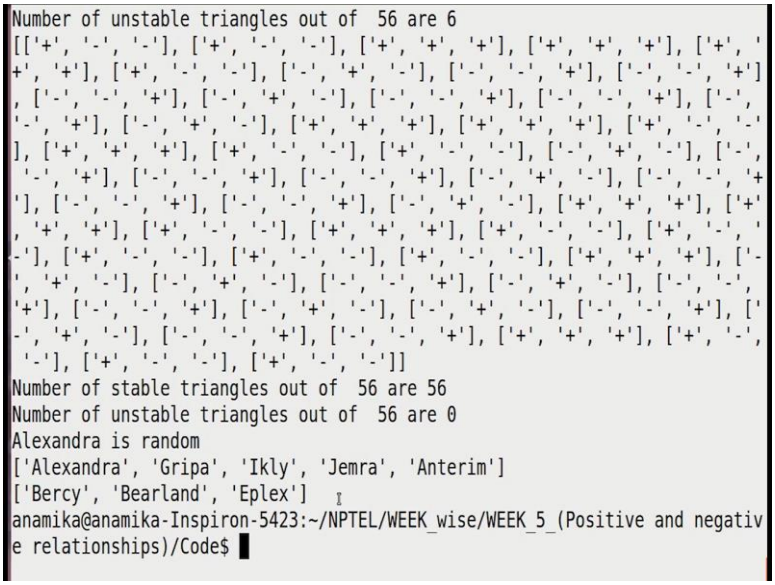
Let us go down and see this is where we call this function and after it returned the two lists, we are just spending those two lists. So, let us see how it works ok.

(Refer Slide Time: 04:32)



These are the air cities and you see the relationships amongst them as well positive and negative relationships I am closing it.

(Refer Slide Time: 04:41)



So, it has processed, and you see these are the two coming the two coalitions formed in the first one there are 5 countries and in the second one there are 3 countries. So, this is the division

In the next video we are going to visualize this network and we are going to compare it with the initial configuration that was entered initially.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Homophily (Continued) & Positive and Negative Relationships
Lecture – 74
Visualizing coalitions and the evolution

(Refer Slide Time: 00:00)

```
154 # 6.1. Choose a random node. Add it to the first coalition.
155 # 6.2. Also put all the 'friends' of this node in the first coalit
156 # 6.3. Put all the 'enemies' of this node in the second coalition.
157 # 6.4. Repeat steps 6.2 and 6.3 for all the 'unprocessed' nodes of
158 first, second = see_coalitions(G)
159 print first
160 print second
161
162
163 # 7. Display the network with coalitions
164 pos = nx.circular_layout(G)
165 nx.draw_networkx_nodes(G, pos, nodelist = first, node_color = 'red
166 nx.draw_networkx_nodes(G, pos, nodelist = second, node_color = 'blu
167
168 nx.draw_networkx_labels(G, pos, font_size = 10, font_family = 'sans
169
170 nx.draw_networkx_edges(G, pos)
171 nx.draw_networkx_edge_labels(G, pos, edge_labels = edge_labels, for
```

Of the implementation where we are going to visualize the communities that is the coalitions that we informed in the previous video. So, one thing that we want to do here is that we want to display the nodes in one coalition by a different color and the nodes in the second coalition by different color; so that we can clearly see the kinds of relationships amongst the nodes inside; the inside the coalition and across the coalition we want to visualize that.

So, what we can do is we can use the same layout circular layout; so we will write nx.circular layout G. Now if we write nx.draw, it will just draw the network simply without any added features there. Since we want to display the nodes in a different in two different colors, we will have to use separate commands.

So, what we can use here is nx.draw in networkx nodes; in that function we will pass the lists list of nodes which have to be colored with the single color. So what we can write is

nx.draw networkx nodes the parameters will be first the graph and then the layout and the third parameter which is the most important is the node list.

So, this list will contain the nodes which have to be colored in one color. So, here we want the nodes which are there in the first list this list to be colored in the same colors. So, you will write node list is equal to first we can specify the color that you want node color is equal to say red. And we can also give the size node size is equal to say 5000; so, that was the displaying the nodes in the first list.

Similarly, we have to display the nodes in the second list we will write nx.drawn networkx nodes G and then pos then node list is equal to second. Now node color is equal to say we will change the color here say blue and node size is equal to 1 second (Refer Time: 02:33) node size is equal to 5000, there is one more parameters which we can use that is alpha which basically gives the transparency value for the for the node color.

So, we can keep it and we can change it later if we require; so, as of now I am giving it 0.8. So, now we have drawn the nodes since we have not made use of nx.draw the rest of the things will not be drawn by default we have only; so we are manually doing it in a way. So, we have drawn the nodes we have not drawn the edges, we have not drawn the the edge labels, we have not drawn the node label; so we are going to do that one by one ok.

So, let us first write the labels of the nodes, so we will write nx.draw networkx labels. So, again the parameters will be G and the layout and then the font size, font size is equal to 10. And then font we can give the font as well by this parameter font family is equal to say sans serif; I am sorry we will put hyphen here.

Now, we have drawn the nodes and the labels on the nodes; now next thing is the edges. So, we will write nx.draw networkx edges; again, the parameters will be G and pos you can give more parameters as well; as of now I think this is enough. Now we have not yet drawn the labels on the edges; so for that we will use nx.draw networkx edge labels; so this is the command, I think we used it at here as well. So, the parameters again will be G and then the layout and then edge labels is equal to edge labels; we are yet to retrieve the labels firstly, we are going to do that after writing this command.

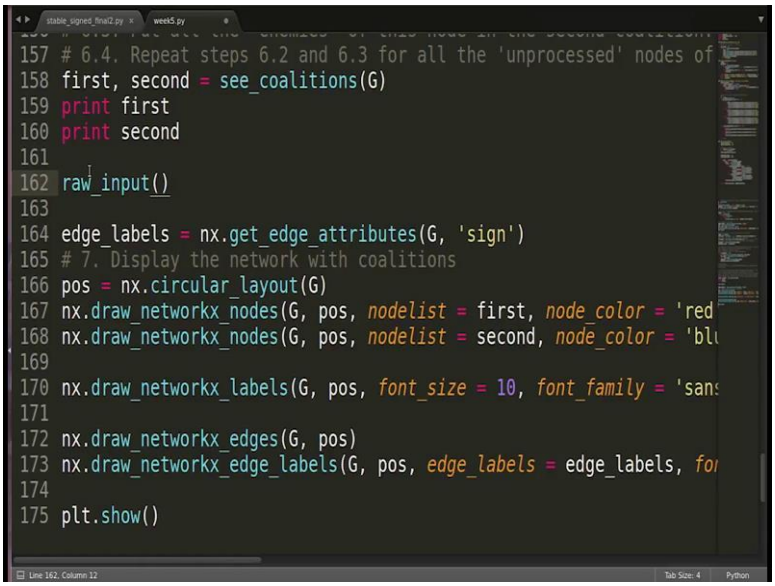
(Refer Slide Time: 04:47)



So, as we did in the previous representation of the graph; we have to retrieve the labels. If you do not do that by default it displays the attribute name as well for example, sign is equal to plus, sign is equal to minus we do not want that, we only want to display plus and minus there. So, we are going to retrieve the edge labels in the next command and apart from that we can display the font size as well; font size is equal to say 20 and in font color is equal to say green maybe.

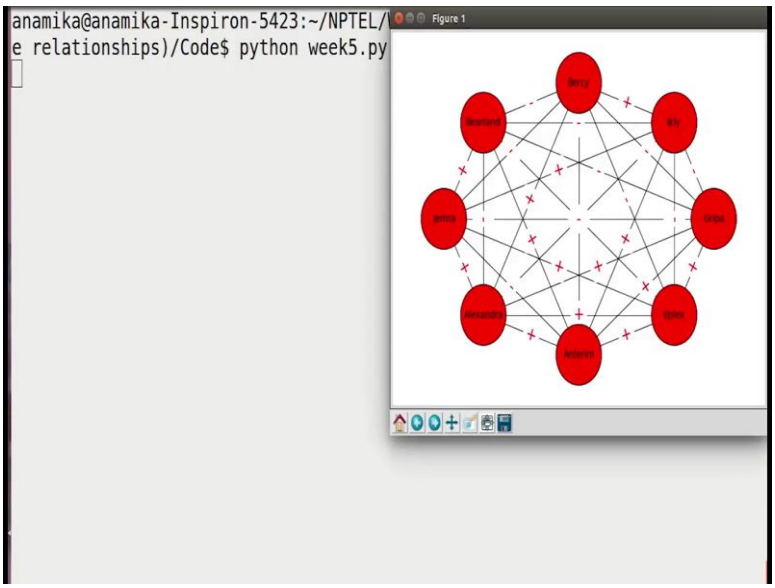
So, the only thing left is retrieving the edge labels ok; for that we will go up here, we will write this command edge labels is equal to nx.get_edge_attributes. And the column address will be the graph and the attribute for which we want to retrieve the values. So, this should fetch the latest attributes of sign for all the edges. So, I think we have drawn pretty much everything, and it should work.

(Refer Slide Time: 05:56)



So, we can write plt.show let me also add one more thing; let me add a function here draw input. Now what this function does is; it will ask you for an enter before it proceeds. So, we want it to pos for a few moments and then we will press enter and then will display the graph the network ok. Let us save it and we will check here.

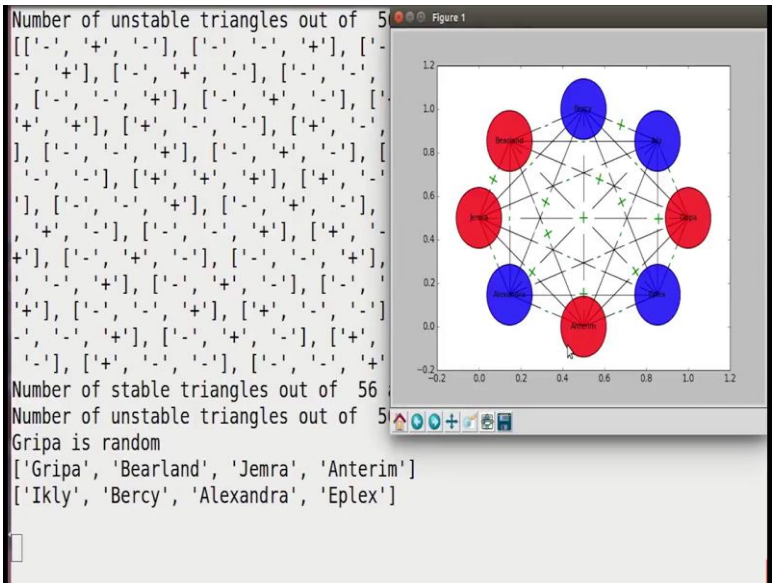
(Refer Slide Time: 06:23)



Let us run this; so, this is the initial graph; the initial network. Let us take an example of an unstable relationship here, let me see Bearland, Alexandra and Anterim this is a triangle which has two positive edges and one negative edge.

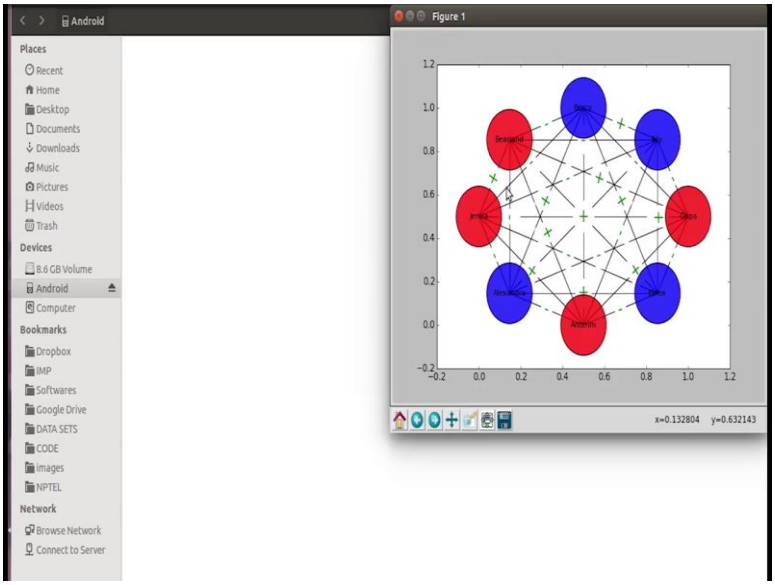
So, this is an unstable relationship this should not stay for a long time in the network. So, let us note down Bearland, Alexandra and Anterim; so, between Bearland and Alexandra there is negative and this two are positive.

(Refer Slide Time: 07:10)



So, let us see what happens to this triangle let me close it and it is now I am pressing enter; this is the final network with the different coalitions colored in a different color.

(Refer Slide Time: 07:19)



So here you see blue color nodes are in the first come in one of the communities I do not I do not remember which community; so, the second coalition nodes are in the red color.

Now, if we look at Bearland, Alexandra and Anterim; this has now become stable by; so, now, there is only one positive edge here.

So, what exactly is happened Alexandra and Anterim which were already friends; they have now turned enemies. So, this is as per our initial configurations that this is the kind of relationship that exists for a long time, but not the previous one. So, that previous one has led to the animosity between the Alexandra and Anterim that is one thing to see. Second thing is, if you look at all the red nodes there are all positive edges amongst them right Bearland and Gripa is positive Gripa and Anterim is positive you can check all the cases.

Similarly, if you took look at all the blue nodes; there are all positive edges amongst them Alexandra, Bercy, Eplex and Icly you see all positive edges are there; they are all friends. However, if you look at the inter edges which is the edges from one coalition to the other coalition; they are all negative which is quite an interesting pattern to see. So, Bercy to Bearland there is negative, Bercy to Gripa is negative Bercy to Jemra is negative; and similarly, Alexandra to Anterim is negative Alexandra to Bearland is negative.

So, precisely we have divided the network into two parts: where people in the first part are all friends, people in the second part are all friends, but these the people in these two parts hate each other; they are all enemies of each other. So, this is a nice and interesting phenomena to observe.